Master Guide: ML & LLM Training and Testing Across Cloud Providers

Purpose: Comprehensive Skill Enhancement - Machine Learning & Large Language Models
Coverage: AWS | GCP | Azure
Scope: Training, Testing, Evaluation, Deployment
Authors: AI Collaboration & Pramod Kumar
Date: September 2026

Document Overview

This master guide provides end-to-end coverage of:

- **Machine Learning Training & Testing** (Traditional ML)
- **Large Language Model Training & Testing** (LLMs)
- **Computer Vision & Generative AI** (CNNs, GANs, Transformers)
- **Implementation across AWS, GCP, and Azure**
- **Practical code examples and deployment strategies**
- **Cost optimization and best practices**

Dataset Sources

Training data for examples in this guide can be obtained from:

- **Kaggle** - Largest repository of ML datasets and competitions
- **Hugging Face Datasets** - NLP and multimodal datasets
- **UCI Machine Learning Repository** - Classic ML datasets
- **Google Dataset Search** - Search engine for datasets
- **AWS Open Data Registry** - Public datasets on AWS
- **Papers With Code** - Datasets from research papers
- **TensorFlow Datasets** - Ready-to-use TF datasets
- **OpenML** - Open machine learning platform

Table of Contents

Part 1: Fundamentals

1. Terminology & Abbreviations
2. Cloud Provider Overview
3. When to Use ML vs LLM

Part 2: Machine Learning (Traditional ML)

4. ML Training Methods

5. ML Testing & Evaluation
6. ML on AWS (SageMaker)
7. ML on GCP (Vertex AI)
8. ML on Azure (Azure ML)
9. ML Deployment Strategies

Part 3: Large Language Models (LLMs)

10. LLM Training Methods
11. LLM Testing & Evaluation
12. LLM on AWS (Bedrock & SageMaker)
13. LLM on GCP (Vertex AI)
14. LLM on Azure (Azure OpenAI)
15. LLM Deployment Strategies

Part 4: Practical Implementation

16. Cross-Cloud Comparison
17. Cost Optimization
18. Production Best Practices
19. Complete Examples

Part 1: Fundamentals

Terminology & Abbreviations

General Terms

Term	Full Form	Definition
ML	Machine Learning	Algorithms that learn patterns from data
LLM	Large Language Model	Neural networks trained on massive text data (billions of parameters)
GPU	Graphics Processing Unit	Specialized hardware for parallel computation
TPU	Tensor Processing Unit	Google's custom AI accelerator chips
vCPU	Virtual CPU	Virtualized processing unit in cloud
VRAM	Video RAM	GPU memory for model storage
ROI	Return on Investment	Business value per dollar spent
API	Application Programming Interface	Interface for software communication
SDK	Software Development Kit	Tools for building applications
CLI	Command Line Interface	Text-based interface for commands

Machine Learning Terms

Term	Full Form	Definition
CV	Computer Vision	ML for image/video analysis
NLP	Natural Language Processing	ML for text/language understanding
XGBoost	eXtreme Gradient Boosting	Tree-based ensemble algorithm
RF	Random Forest	Ensemble of decision trees
SVM	Support Vector Machine	Classification algorithm using hyperplanes
ANN	Artificial Neural Network	Network of connected neurons
CNN	Convolutional Neural Network	Neural network for image processing
RNN	Recurrent Neural Network	Neural network for sequential data
LSTM	Long Short-Term Memory	RNN variant for long sequences
AutoML	Automated Machine Learning	Automated model selection and tuning
HPO	Hyperparameter Optimization	Finding best model parameters
MAE	Mean Absolute Error	Average prediction error metric
RMSE	Root Mean Square Error	Square root of average squared errors
AUC	Area Under Curve	Model performance metric (0-1)
F1	F1 Score	Harmonic mean of precision and recall

LLM-Specific Terms

Term	Full Form	Definition
LoRA	Low-Rank Adaptation	Efficient fine-tuning method (0.1% params)
QLoRA	Quantized LoRA	LoRA with 4-bit quantization (75% memory savings)
PEFT	Parameter-Efficient Fine-Tuning	Training subset of parameters
RLHF	Reinforcement Learning from Human Feedback	Alignment technique (ChatGPT)
DPO	Direct Preference Optimization	Simpler alternative to RLHF
SFT	Supervised Fine-Tuning	Training on instruction-response pairs
RAG	Retrieval Augmented Generation	LLM + external knowledge retrieval
Perplexity	-	Metric for language model quality (lower = better)
BLEU	Bilingual Evaluation Understudy	Translation/generation quality (0-100)

Term	Full Form	Definition
ROUGE	Recall-Oriented Understudy for Gisting Evaluation	Summarization quality metric
Prompt	-	Input text to LLM
Context Window	-	Maximum input length (tokens)
Token	-	Text unit (word/subword, ~4 chars)
Temperature	-	Randomness in generation (0=deterministic, 2=creative)
Top-k	-	Sample from top k most likely tokens
Top-p	Nucleus Sampling	Probability threshold for token selection (e.g., 0.9)
Embedding	-	Vector representation of text
Fine-tuning	-	Training pre-trained model on specific task

Cloud-Specific Terms

Term	Definition
SageMaker	AWS managed ML service
Vertex AI	GCP managed ML/AI service
Azure ML	Azure machine learning service
Bedrock	AWS managed foundation model service
Azure OpenAI	Azure's hosted OpenAI models
Spot Instance	Discounted preemptible compute (70% cheaper)
Serverless	Auto-scaling compute without server management
Endpoint	Deployed model API for inference
Batch Inference	Processing multiple predictions together
Real-time Inference	Live prediction API

Cloud Provider Overview

Quick Comparison

Feature	AWS	GCP	Azure
ML Service	SageMaker	Vertex AI	Azure ML

Feature	AWS	GCP	Azure
LLM Service	Bedrock + SageMaker	Vertex AI (PaLM, Gemini)	Azure OpenAI
AutoML	SageMaker Autopilot	Vertex AI AutoML	Azure AutoML
Notebooks	SageMaker Studio	Vertex AI Workbench	Azure ML Studio
GPU Types	V100, A100, T4	V100, A100, T4, TPU	V100, A100, T4
Spot Pricing	Spot Instances (70% off)	Preemptible VMs (70% off)	Spot VMs (80% off)
Free Tier	250 hours SageMaker*	$300 credit*	$200 credit*
Best For	Mature ML pipelines	AI research, TPUs	Enterprise integration

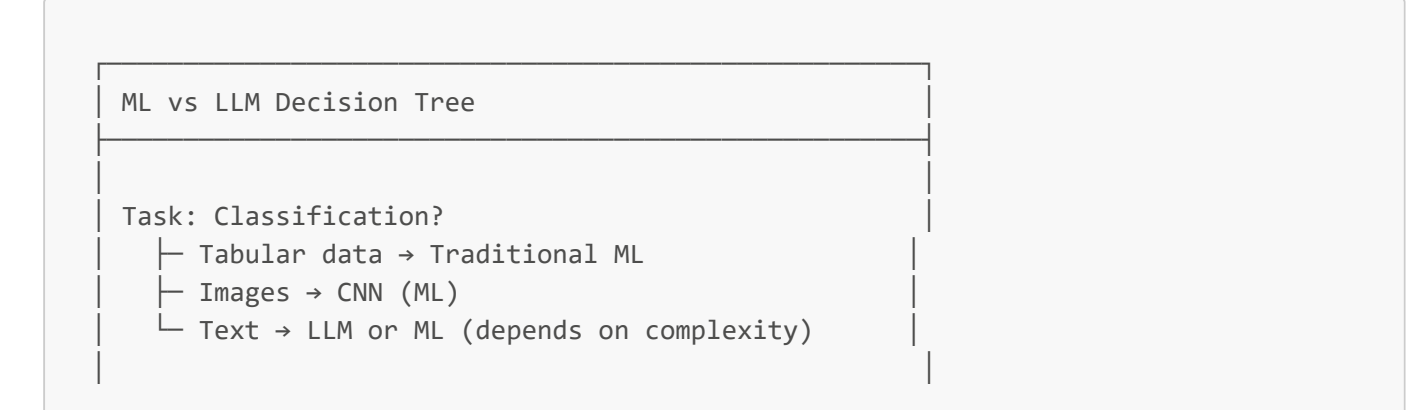
Note: *Free tier amounts are approximate and subject to change. Verify current offerings at the time of use as cloud providers frequently update their free tier and credit programs.*

Service Mapping

Capability	AWS	GCP	Azure
Compute	EC2	Compute Engine	Virtual Machines
Containers	ECS, EKS	GKE	AKS
Serverless	Lambda	Cloud Functions	Azure Functions
Storage	S3	Cloud Storage	Blob Storage
Model Registry	SageMaker Model Registry	Vertex AI Model Registry	Azure ML Model Registry
Experiment Tracking	SageMaker Experiments	Vertex AI Experiments	Azure ML Experiments
Pipeline	SageMaker Pipelines	Vertex AI Pipelines	Azure ML Pipelines
Monitoring	CloudWatch	Cloud Monitoring	Azure Monitor

When to Use ML vs LLM

Decision Matrix



```
┌─────────────────────────────────────────────────────────┐
│ Task: Regression/Prediction?                            │
│   ├─ Structured data → Traditional ML                   │
│   └─ Time series → LSTM (ML) or specialized models      │
│                                                         │
│ Task: Text Generation?                                  │
│   └─ Use LLM (GPT, PaLM, etc.)                          │
│                                                         │
│ Task: Question Answering?                               │
│   ├─ Simple FAQ → ML + retrieval                        │
│   └─ Complex reasoning → LLM                            │
│                                                         │
│ Task: Chatbot?                                          │
│   ├─ Rule-based → Traditional NLP                       │
│   └─ Conversational → LLM                               │
│                                                         │
│ Budget < $100? → Start with ML                          │
│ Budget > $1000? → Can explore LLM                       │
│                                                         │
│ Data < 10K rows? → Consider pre-trained LLM             │
│ Data > 100K rows? → Train custom ML model               │
│                                                         │
└─────────────────────────────────────────────────────────┘
```

Use Case Examples

Use Case	Recommended Approach	Why
Customer churn prediction	Traditional ML (XGBoost, RF)	Tabular data, interpretability needed
Fraud detection	Traditional ML (Anomaly detection)	Real-time, explainable, structured
Product recommendations	Traditional ML (Collaborative filtering)	Structured user-item data
Image classification	CNN (ML)	Visual data, well-established methods
Sentiment analysis	LLM or BERT (ML)	Text understanding, nuanced context
Chatbot	LLM (GPT, PaLM)	Conversational, open-ended responses
Code generation	LLM (Codex, StarCoder)	Complex reasoning, multiple languages
Document summarization	LLM	Understanding + generation
Named Entity Recognition	BERT (ML) or LLM	Structured extraction from text
Stock price prediction	Traditional ML (LSTM, XGBoost)	Time series, numerical

Part 2: Machine Learning (Traditional ML)

ML Training Methods

Classification

Terminology for Classification

Term	Definition
Binary Classification	Two-class problem (spam/not spam)
Multi-class Classification	Multiple exclusive classes (digit 0-9)
Multi-label Classification	Multiple non-exclusive classes (tags on article)
Precision	Correct positives / All predicted positives
Recall	Correct positives / All actual positives
Confusion Matrix	Table showing TP (True Positives), TN (True Negatives), FP (False Positives), FN (False Negatives)

Popular Algorithms

```
"""
Classification: Assigning categories to data points
Examples: Email spam detection, disease diagnosis, customer churn
"""

from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
import xgboost as xgb
import pandas as pd
import numpy as np

class ClassificationTrainer:
    """
    Comprehensive classification training pipeline
    """

    def __init__(self, algorithm='random_forest'):
        """
        Args:
            algorithm: 'logistic', 'random_forest', 'xgboost', 'svm', 'neural_net'
```

```
        """
        self.algorithm = algorithm
        self.model = self._get_model()

    def _get_model(self):
        """Select algorithm"""
        models = {
            'logistic': LogisticRegression(max_iter=1000),
            'random_forest': RandomForestClassifier(n_estimators=100,
random_state=42),
            'xgboost': xgb.XGBClassifier(n_estimators=100, learning_rate=0.1),
            'svm': SVC(kernel='rbf', probability=True),
            'neural_net': MLPClassifier(hidden_layers=(100, 50), max_iter=500)
        }
        return models[self.algorithm]

    def train(self, X_train, y_train, X_val=None, y_val=None):
        """Train model"""
        print(f"Training {self.algorithm}...")

        # Train
        self.model.fit(X_train, y_train)

        # Training accuracy
        train_score = self.model.score(X_train, y_train)
        print(f"Training Accuracy: {train_score:.4f}")

        # Validation accuracy
        if X_val is not None:
            val_score = self.model.score(X_val, y_val)
            print(f"Validation Accuracy: {val_score:.4f}")

            # Detailed metrics
            y_pred = self.model.predict(X_val)
            print("\nClassification Report:")
            print(classification_report(y_val, y_pred))

            # AUC if binary classification
            if len(np.unique(y_train)) == 2:
                y_pred_proba = self.model.predict_proba(X_val)[:, 1]
                auc = roc_auc_score(y_val, y_pred_proba)
                print(f"AUC-ROC: {auc:.4f}")

        return self.model

    def cross_validate(self, X, y, cv=5):
        """Cross-validation"""
        scores = cross_val_score(self.model, X, y, cv=cv, scoring='accuracy')
        print(f"{cv}-Fold CV Accuracy: {scores.mean():.4f} (+/- {scores.std() *
2:.4f})")
        return scores

# Example: Credit card fraud detection
if __name__ == "__main__":
```

```
    # Load public dataset
    # Dataset: Available from UCI ML Repository or similar sources

    # Simulated data for demo
    from sklearn.datasets import make_classification
    X, y = make_classification(n_samples=10000, n_features=20, n_classes=2,
                               weights=[0.95, 0.05], random_state=42)

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

    # Train multiple algorithms
    for algo in ['logistic', 'random_forest', 'xgboost']:
        print(f"\n{'='*60}")
        print(f"Algorithm: {algo.upper()}")
        print('='*60)

        trainer = ClassificationTrainer(algorithm=algo)
        trainer.train(X_train, y_train, X_test, y_test)
```

Regression

Terminology for Regression

Term	Definition
Linear Regression	Predicting continuous value with linear relationship
Ridge Regression	Linear regression with L2 regularization
Lasso Regression	Linear regression with L1 regularization
R^2 Score	Proportion of variance explained (0-1, higher better)
MSE	Mean Squared Error (lower better)
MAE	Mean Absolute Error (lower better)

Popular Algorithms

```
"""
Regression: Predicting continuous values
Examples: House price prediction, sales forecasting, temperature prediction
"""

from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.svm import SVR
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import xgboost as xgb
import numpy as np
```

```python
class RegressionTrainer:
    """
    Comprehensive regression training pipeline
    """

    def __init__(self, algorithm='linear'):
        """
        Args:
            algorithm: 'linear', 'ridge', 'lasso', 'random_forest', 'xgboost',
'svr'
        """
        self.algorithm = algorithm
        self.model = self._get_model()

    def _get_model(self):
        """Select algorithm"""
        models = {
            'linear': LinearRegression(),
            'ridge': Ridge(alpha=1.0),
            'lasso': Lasso(alpha=1.0),
            'random_forest': RandomForestRegressor(n_estimators=100,
random_state=42),
            'xgboost': xgb.XGBRegressor(n_estimators=100, learning_rate=0.1),
            'svr': SVR(kernel='rbf')
        }
        return models[self.algorithm]

    def train(self, X_train, y_train, X_val=None, y_val=None):
        """Train model"""
        print(f"Training {self.algorithm}...")

        # Train
        self.model.fit(X_train, y_train)

        # Training metrics
        y_train_pred = self.model.predict(X_train)
        train_rmse = np.sqrt(mean_squared_error(y_train, y_train_pred))
        train_mae = mean_absolute_error(y_train, y_train_pred)
        train_r2 = r2_score(y_train, y_train_pred)

        print(f"Training RMSE: {train_rmse:.4f}")
        print(f"Training MAE: {train_mae:.4f}")
        print(f"Training R²: {train_r2:.4f}")

        # Validation metrics
        if X_val is not None:
            y_val_pred = self.model.predict(X_val)
            val_rmse = np.sqrt(mean_squared_error(y_val, y_val_pred))
            val_mae = mean_absolute_error(y_val, y_val_pred)
            val_r2 = r2_score(y_val, y_val_pred)

            print(f"\nValidation RMSE: {val_rmse:.4f}")
            print(f"Validation MAE: {val_mae:.4f}")
            print(f"Validation R²: {val_r2:.4f}")
```

```
        return self.model

# Example: House price prediction
if __name__ == "__main__":
    # Load public dataset
    # Dataset: Available from competition archives or public ML repositories

    # Simulated data for demo
    from sklearn.datasets import make_regression
    X, y = make_regression(n_samples=1000, n_features=10, noise=10,
random_state=42)

    from sklearn.model_selection import train_test_split
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

    # Train multiple algorithms
    for algo in ['linear', 'ridge', 'random_forest', 'xgboost']:
        print(f"\n{'='*60}")
        print(f"Algorithm: {algo.upper()}")
        print('='*60)

        trainer = RegressionTrainer(algorithm=algo)
        trainer.train(X_train, y_train, X_test, y_test)
```

Clustering

Terminology for Clustering

Term	Definition
Unsupervised Learning	Learning without labels
Centroid	Center point of cluster
Silhouette Score	Cluster quality metric (-1 to 1, higher better)
Elbow Method	Finding optimal number of clusters
Dendrogram	Hierarchical clustering visualization

Popular Algorithms

```
"""
Clustering: Grouping similar data points
Examples: Customer segmentation, anomaly detection, image compression
"""

from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering
from sklearn.metrics import silhouette_score, davies_bouldin_score
from sklearn.preprocessing import StandardScaler
```

```python
import numpy as np
import matplotlib.pyplot as plt

class ClusteringTrainer:
    """
    Comprehensive clustering pipeline
    """

    def __init__(self, algorithm='kmeans', n_clusters=3):
        """
        Args:
            algorithm: 'kmeans', 'dbscan', 'hierarchical'
            n_clusters: Number of clusters (for kmeans, hierarchical)
        """
        self.algorithm = algorithm
        self.n_clusters = n_clusters
        self.model = self._get_model()

    def _get_model(self):
        """Select algorithm"""
        models = {
            'kmeans': KMeans(n_clusters=self.n_clusters, random_state=42),
            'dbscan': DBSCAN(eps=0.5, min_samples=5),
            'hierarchical': AgglomerativeClustering(n_clusters=self.n_clusters)
        }
        return models[self.algorithm]

    def train(self, X):
        """Train clustering model"""
        print(f"Training {self.algorithm}...")

        # Standardize features
        scaler = StandardScaler()
        X_scaled = scaler.fit_transform(X)

        # Fit
        labels = self.model.fit_predict(X_scaled)

        # Metrics
        n_clusters_found = len(np.unique(labels))
        print(f"Number of clusters found: {n_clusters_found}")

        if n_clusters_found > 1:
            silhouette = silhouette_score(X_scaled, labels)
            davies_bouldin = davies_bouldin_score(X_scaled, labels)

            print(f"Silhouette Score: {silhouette:.4f} (higher is better)")
            print(f"Davies-Bouldin Index: {davies_bouldin:.4f} (lower is better)")

        return labels

    def find_optimal_k(self, X, max_k=10):
        """Elbow method for optimal k"""
        inertias = []
```

```
        silhouettes = []

        scaler = StandardScaler()
        X_scaled = scaler.fit_transform(X)

        for k in range(2, max_k+1):
            kmeans = KMeans(n_clusters=k, random_state=42)
            labels = kmeans.fit_predict(X_scaled)

            inertias.append(kmeans.inertia_)
            silhouettes.append(silhouette_score(X_scaled, labels))

        # Plot elbow curve
        plt.figure(figsize=(12, 4))

        plt.subplot(1, 2, 1)
        plt.plot(range(2, max_k+1), inertias, marker='o')
        plt.xlabel('Number of Clusters')
        plt.ylabel('Inertia')
        plt.title('Elbow Method')

        plt.subplot(1, 2, 2)
        plt.plot(range(2, max_k+1), silhouettes, marker='o', color='green')
        plt.xlabel('Number of Clusters')
        plt.ylabel('Silhouette Score')
        plt.title('Silhouette Analysis')

        plt.tight_layout()
        plt.show()

        return inertias, silhouettes

# Example: Customer segmentation
if __name__ == "__main__":
    # Load public dataset
    # Dataset: Available from UCI ML Repository or similar sources

    # Simulated data for demo
    from sklearn.datasets import make_blobs
    X, _ = make_blobs(n_samples=500, n_features=2, centers=4, random_state=42)

    # Find optimal clusters
    trainer = ClusteringTrainer(algorithm='kmeans')
    trainer.find_optimal_k(X, max_k=8)

    # Train with optimal k
    trainer_optimal = ClusteringTrainer(algorithm='kmeans', n_clusters=4)
    labels = trainer_optimal.train(X)
```

ML Testing & Evaluation

Terminology for Testing & Evaluation

Term	Definition
Train-Test Split	Dividing data into training and testing sets
Cross-Validation	Multiple train-test splits for robust evaluation
Overfitting	Model performs well on training but poor on new data
Underfitting	Model too simple to capture patterns
Bias	Error from wrong assumptions (underfitting)
Variance	Error from sensitivity to training data (overfitting)
Stratified Split	Maintaining class distribution in splits
Hold-out Set	Data never used during training
A/B Testing	Comparing two models in production
Baseline Model	Simple model for comparison

Comprehensive Testing Pipeline

```
"""
ML Model Testing & Evaluation Pipeline
Covers: Train-test split, cross-validation, hyperparameter tuning, model
comparison
"""

from sklearn.model_selection import (
    train_test_split,
    cross_val_score,
    GridSearchCV,
    RandomizedSearchCV,
    StratifiedKFold
)
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    mean_squared_error, r2_score,
    classification_report, confusion_matrix
)
from sklearn.ensemble import RandomForestClassifier
import xgboost as xgb
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

class MLTester:
    """
    Comprehensive ML testing and evaluation
```

```
    """

    def __init__(self, task='classification'):
        """
        Args:
            task: 'classification' or 'regression'
        """
        self.task = task
        self.results = {}

    def train_test_split_evaluation(self, X, y, model, test_size=0.2):
        """Basic train-test split evaluation"""
        print("1. Train-Test Split Evaluation")
        print("-" * 60)

        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=test_size, random_state=42,
            stratify=y if self.task == 'classification' else None
        )

        # Train
        model.fit(X_train, y_train)

        # Predict
        y_pred = model.predict(X_test)

        # Evaluate
        if self.task == 'classification':
            accuracy = accuracy_score(y_test, y_pred)
            precision = precision_score(y_test, y_pred, average='weighted')
            recall = recall_score(y_test, y_pred, average='weighted')
            f1 = f1_score(y_test, y_pred, average='weighted')

            print(f"Accuracy:  {accuracy:.4f}")
            print(f"Precision: {precision:.4f}")
            print(f"Recall:    {recall:.4f}")
            print(f"F1 Score:  {f1:.4f}")

            self.results['train_test'] = {
                'accuracy': accuracy, 'precision': precision,
                'recall': recall, 'f1': f1
            }
        else:
            mse = mean_squared_error(y_test, y_pred)
            rmse = np.sqrt(mse)
            mae = np.mean(np.abs(y_test - y_pred))
            r2 = r2_score(y_test, y_pred)

            print(f"MSE:  {mse:.4f}")
            print(f"RMSE: {rmse:.4f}")
            print(f"MAE:  {mae:.4f}")
            print(f"R²:   {r2:.4f}")

            self.results['train_test'] = {
```

```
                'mse': mse, 'rmse': rmse, 'mae': mae, 'r2': r2
            }

        return model, X_test, y_test, y_pred

    def cross_validation_evaluation(self, X, y, model, cv=5):
        """Cross-validation evaluation"""
        print(f"\n2. {cv}-Fold Cross-Validation")
        print("-" * 60)

        if self.task == 'classification':
            scoring = 'accuracy'
            cv_split = StratifiedKFold(n_splits=cv, shuffle=True, random_state=42)
        else:
            scoring = 'neg_mean_squared_error'
            cv_split = cv

        scores = cross_val_score(model, X, y, cv=cv_split, scoring=scoring)

        if self.task == 'regression':
            scores = -scores  # Convert negative MSE to positive

        print(f"Scores: {scores}")
        print(f"Mean:   {scores.mean():.4f}")
        print(f"Std:    {scores.std():.4f}")
        print(f"95% CI: [{scores.mean() - 1.96*scores.std():.4f}, "
              f"{scores.mean() + 1.96*scores.std():.4f}]")

        self.results['cross_val'] = {
            'mean': scores.mean(),
            'std': scores.std(),
            'scores': scores
        }

        return scores

    def hyperparameter_tuning(self, X, y, model, param_grid, search_type='grid'):
        """Hyperparameter optimization"""
        print(f"\n3. Hyperparameter Tuning ({search_type.upper()} Search)")
        print("-" * 60)

        if self.task == 'classification':
            scoring = 'accuracy'
            cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
        else:
            scoring = 'neg_mean_squared_error'
            cv = 5

        if search_type == 'grid':
            search = GridSearchCV(model, param_grid, cv=cv, scoring=scoring,
n_jobs=-1)
        else:
            search = RandomizedSearchCV(model, param_grid, n_iter=20, cv=cv,
                                        scoring=scoring, n_jobs=-1,
```

```
random_state=42)

        search.fit(X, y)

        print(f"Best Parameters: {search.best_params_}")
        print(f"Best Score: {search.best_score_:.4f}")

        self.results['hyperparameter_tuning'] = {
            'best_params': search.best_params_,
            'best_score': search.best_score_
        }

        return search.best_estimator_

    def learning_curve_analysis(self, X, y, model, train_sizes=np.linspace(0.1,
1.0, 10)):
        """Learning curve to detect overfitting/underfitting"""
        from sklearn.model_selection import learning_curve

        print("\n4. Learning Curve Analysis")
        print("-" * 60)

        train_sizes, train_scores, val_scores = learning_curve(
            model, X, y, train_sizes=train_sizes, cv=5, scoring='accuracy' if
self.task == 'classification' else 'r2'
        )

        train_mean = np.mean(train_scores, axis=1)
        train_std = np.std(train_scores, axis=1)
        val_mean = np.mean(val_scores, axis=1)
        val_std = np.std(val_scores, axis=1)

        plt.figure(figsize=(10, 6))
        plt.plot(train_sizes, train_mean, label='Training score', marker='o')
        plt.fill_between(train_sizes, train_mean - train_std, train_mean +
train_std, alpha=0.15)
        plt.plot(train_sizes, val_mean, label='Validation score', marker='o')
        plt.fill_between(train_sizes, val_mean - val_std, val_mean + val_std,
alpha=0.15)

        plt.xlabel('Training Set Size')
        plt.ylabel('Score')
        plt.title('Learning Curve')
        plt.legend()
        plt.grid(True)
        plt.tight_layout()
        plt.show()

        # Diagnose
        final_train = train_mean[-1]
        final_val = val_mean[-1]
        gap = final_train - final_val

        if gap > 0.1:
```

```
            diagnosis = "OVERFITTING detected (large gap between train and
validation)"
        elif final_val < 0.7:
            diagnosis = "UNDERFITTING detected (low scores on both train and
validation)"
        else:
            diagnosis = "Model is well-fitted"

        print(f"Diagnosis: {diagnosis}")

        return train_sizes, train_scores, val_scores

    def generate_report(self):
        """Generate comprehensive test report"""
        print("\n" + "="*60)
        print("COMPREHENSIVE TEST REPORT")
        print("="*60)

        for test_name, metrics in self.results.items():
            print(f"\n{test_name.upper().replace('_', ' ')}:")
            for metric_name, value in metrics.items():
                if isinstance(value, (int, float)):
                    print(f"  {metric_name}: {value:.4f}")
                elif isinstance(value, dict):
                    print(f"  {metric_name}: {value}")

# Example usage
if __name__ == "__main__":
    # Load data
    from sklearn.datasets import load_breast_cancer
    data = load_breast_cancer()
    X, y = data.data, data.target

    # Initialize tester
    tester = MLTester(task='classification')

    # 1. Train-test split
    model = RandomForestClassifier(random_state=42)
    trained_model, X_test, y_test, y_pred = tester.train_test_split_evaluation(X,
y, model)

    # 2. Cross-validation
    tester.cross_validation_evaluation(X, y, model, cv=5)

    # 3. Hyperparameter tuning
    param_grid = {
        'n_estimators': [50, 100, 200],
        'max_depth': [5, 10, None],
        'min_samples_split': [2, 5, 10]
    }
    best_model = tester.hyperparameter_tuning(X, y, model, param_grid,
search_type='grid')

    # 4. Learning curve
```

```
    tester.learning_curve_analysis(X, y, best_model)

    # 5. Final report
    tester.generate_report()
```

ML on AWS (SageMaker)

Terminology for AWS SageMaker

Term	Definition
SageMaker Studio	Integrated development environment for ML
Training Job	Managed model training execution
Endpoint	Deployed model for real-time inference
Batch Transform	Batch inference on large datasets
Autopilot	AutoML service for automated model building
Ground Truth	Data labeling service
Model Monitor	Track model performance over time
Feature Store	Centralized feature management
Pipelines	CI/CD for ML workflows
Clarify	Model explainability and bias detection

AWS SageMaker Setup

```
"""
AWS SageMaker - Traditional ML Training & Deployment
"""

import boto3
import sagemaker
from sagemaker import get_execution_role
from sagemaker.sklearn import SKLearn
from sagemaker.xgboost import XGBoost
from sagemaker.tuner import HyperparameterTuner, IntegerParameter,
ContinuousParameter
import pandas as pd
import numpy as np

class SageMakerMLTrainer:
    """
    Train traditional ML models on AWS SageMaker
    """
```

```
    def __init__(self, region='us-east-1'):
        """
        Initialize SageMaker session

        Args:
            region: AWS region
        """
        self.session = sagemaker.Session()
        self.region = region
        self.role = get_execution_role()
        self.bucket = self.session.default_bucket()

        print(f"✓ SageMaker session initialized")
        print(f"  Region: {region}")
        print(f"  Role: {self.role}")
        print(f"  Bucket: {self.bucket}")

    def upload_data_to_s3(self, train_data, test_data, prefix='ml-training'):
        """Upload training data to S3"""

        # Save locally
        train_data.to_csv('train.csv', index=False, header=False)
        test_data.to_csv('test.csv', index=False, header=False)

        # Upload to S3
        train_s3 = self.session.upload_data(
            path='train.csv',
            bucket=self.bucket,
            key_prefix=f'{prefix}/train'
        )

        test_s3 = self.session.upload_data(
            path='test.csv',
            bucket=self.bucket,
            key_prefix=f'{prefix}/test'
        )

        print(f"✓ Data uploaded to S3")
        print(f"  Train: {train_s3}")
        print(f"  Test: {test_s3}")

        return train_s3, test_s3

    def train_xgboost(self, train_s3, test_s3, output_path=None):
        """Train XGBoost model on SageMaker"""

        if output_path is None:
            output_path = f's3://{self.bucket}/xgboost-output'

        # XGBoost estimator
        xgb = XGBoost(
            entry_point='train.py',  # Your training script
            framework_version='1.5-1',
            instance_type='ml.m5.xlarge',
```

```
            instance_count=1,
            role=self.role,
            output_path=output_path,
            sagemaker_session=self.session,
            hyperparameters={
                'max_depth': 5,
                'eta': 0.2,
                'gamma': 4,
                'min_child_weight': 6,
                'subsample': 0.8,
                'objective': 'binary:logistic',
                'num_round': 100
            }
        )

        # Train
        print("Training XGBoost model...")
        xgb.fit({
            'train': train_s3,
            'validation': test_s3
        })

        print(f"✓ Model trained: {xgb.model_data}")

        return xgb

    def train_sklearn(self, train_s3, algorithm='random_forest'):
        """Train scikit-learn model on SageMaker"""

        # Training script
        train_script = f"""
import argparse
import os
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
import joblib

if __name__ == '__main__':
    parser = argparse.ArgumentParser()

    # Hyperparameters
    parser.add_argument('--n-estimators', type=int, default=100)
    parser.add_argument('--max-depth', type=int, default=10)
    parser.add_argument('--algorithm', type=str, default='random_forest')

    # SageMaker specific arguments
    parser.add_argument('--model-dir', type=str,
default=os.environ.get('SM_MODEL_DIR'))
    parser.add_argument('--train', type=str,
default=os.environ.get('SM_CHANNEL_TRAIN'))

    args = parser.parse_args()
```

```
    # Load data
    train_data = pd.read_csv(os.path.join(args.train, 'train.csv'), header=None)
    X_train = train_data.iloc[:, 1:].values
    y_train = train_data.iloc[:, 0].values

    # Train model
    if args.algorithm == 'random_forest':
        model = RandomForestClassifier(
            n_estimators=args.n_estimators,
            max_depth=args.max_depth,
            random_state=42
        )
    elif args.algorithm == 'gradient_boosting':
        model = GradientBoostingClassifier(
            n_estimators=args.n_estimators,
            max_depth=args.max_depth,
            random_state=42
        )
    else:
        model = LogisticRegression(max_iter=1000)

    model.fit(X_train, y_train)

    # Save model
    joblib.dump(model, os.path.join(args.model_dir, 'model.joblib'))
"""

        # Save training script
        with open('train.py', 'w') as f:
            f.write(train_script)

        # SKLearn estimator
        sklearn = SKLearn(
            entry_point='train.py',
            framework_version='1.0-1',
            instance_type='ml.m5.xlarge',
            role=self.role,
            sagemaker_session=self.session,
            hyperparameters={
                'n-estimators': 100,
                'max-depth': 10,
                'algorithm': algorithm
            }
        )

        # Train
        print(f"Training {algorithm} model...")
        sklearn.fit({'train': train_s3})

        print(f"✓ Model trained: {sklearn.model_data}")

        return sklearn
```

```
    def hyperparameter_tuning(self, train_s3, test_s3):
        """Hyperparameter tuning with SageMaker"""

        # Base estimator
        xgb = XGBoost(
            entry_point='train.py',
            framework_version='1.5-1',
            instance_type='ml.m5.xlarge',
            instance_count=1,
            role=self.role,
            sagemaker_session=self.session
        )

        # Hyperparameter ranges
        hyperparameter_ranges = {
            'max_depth': IntegerParameter(3, 10),
            'eta': ContinuousParameter(0.01, 0.3),
            'min_child_weight': IntegerParameter(1, 10),
            'subsample': ContinuousParameter(0.5, 1.0),
            'num_round': IntegerParameter(50, 200)
        }

        # Tuner
        tuner = HyperparameterTuner(
            xgb,
            'validation:auc',  # Metric to optimize
            hyperparameter_ranges,
            max_jobs=20,
            max_parallel_jobs=3,
            objective_type='Maximize'
        )

        # Run tuning
        print("Starting hyperparameter tuning...")
        tuner.fit({
            'train': train_s3,
            'validation': test_s3
        })

        print(f"✓ Best training job: {tuner.best_training_job()}")

        return tuner

    def deploy_model(self, estimator, endpoint_name='ml-endpoint'):
        """Deploy model to endpoint"""

        print(f"Deploying model to endpoint: {endpoint_name}")

        predictor = estimator.deploy(
            instance_type='ml.m5.large',
            initial_instance_count=1,
            endpoint_name=endpoint_name
        )
```

```
        print(f"✓ Model deployed: {endpoint_name}")

        return predictor

    def batch_transform(self, estimator, test_s3, output_path=None):
        """Batch inference"""

        if output_path is None:
            output_path = f's3://{self.bucket}/batch-output'

        transformer = estimator.transformer(
            instance_count=1,
            instance_type='ml.m5.xlarge',
            output_path=output_path
        )

        print("Running batch transform...")
        transformer.transform(test_s3, content_type='text/csv')
        transformer.wait()

        print(f"✓ Batch transform complete: {output_path}")

        return transformer

# Example usage
if __name__ == "__main__":
    # Initialize
    trainer = SageMakerMLTrainer(region='us-east-1')

    # Prepare data (example with Titanic dataset)
    # Dataset: Available from public ML repositories
    from sklearn.datasets import make_classification
    X, y = make_classification(n_samples=1000, n_features=20, random_state=42)

    # Create DataFrame
    train_df = pd.DataFrame(X[:800])
    train_df.insert(0, 'target', y[:800])

    test_df = pd.DataFrame(X[800:])
    test_df.insert(0, 'target', y[800:])

    # Upload to S3
    train_s3, test_s3 = trainer.upload_data_to_s3(train_df, test_df)

    # Train XGBoost
    xgb_model = trainer.train_xgboost(train_s3, test_s3)

    # Hyperparameter tuning
    tuner = trainer.hyperparameter_tuning(train_s3, test_s3)

    # Deploy
    predictor = trainer.deploy_model(xgb_model, endpoint_name='xgboost-endpoint')

    # Predict
```

```
    test_data = test_df.iloc[:, 1:].values
    predictions = predictor.predict(test_data)
    print(f"Predictions: {predictions[:5]}")
```

AWS SageMaker CLI Commands

```
#!/bin/bash
# AWS SageMaker CLI operations

# Setup
export AWS_REGION="us-east-1"
export BUCKET_NAME="my-sagemaker-bucket"
export ROLE_ARN="arn:aws:iam::123456789:role/SageMakerRole"

# 1. Create S3 bucket
aws s3 mb s3://$BUCKET_NAME --region $AWS_REGION

# 2. Upload training data
aws s3 cp train.csv s3://$BUCKET_NAME/data/train/
aws s3 cp test.csv s3://$BUCKET_NAME/data/test/

# 3. Create training job
aws sagemaker create-training-job \
  --training-job-name "xgboost-training-$(date +%Y%m%d-%H%M%S)" \
  --algorithm-specification TrainingImage=683313688378.dkr.ecr.us-east-
1.amazonaws.com/sagemaker-xgboost:1.5-1,TrainingInputMode=File \
  --role-arn $ROLE_ARN \
  --input-data-config '[
    {
      "ChannelName": "train",
      "DataSource": {
        "S3DataSource": {
          "S3DataType": "S3Prefix",
          "S3Uri": "s3://'$BUCKET_NAME'/data/train/",
          "S3DataDistributionType": "FullyReplicated"
        }
      }
    }
  ]' \
  --output-data-config S3OutputPath=s3://$BUCKET_NAME/output \
  --resource-config InstanceType=ml.m5.xlarge,InstanceCount=1,VolumeSizeInGB=10 \
  --stopping-condition MaxRuntimeInSeconds=3600

# 4. List training jobs
aws sagemaker list-training-jobs --max-results 10

# 5. Describe training job
aws sagemaker describe-training-job --training-job-name <job-name>

# 6. Create model
aws sagemaker create-model \
```

```
  --model-name xgboost-model \
  --primary-container Image=683313688378.dkr.ecr.us-east-
1.amazonaws.com/sagemaker-xgboost:1.5-
1,ModelDataUrl=s3://$BUCKET_NAME/output/model.tar.gz \
  --execution-role-arn $ROLE_ARN

# 7. Create endpoint configuration
aws sagemaker create-endpoint-config \
  --endpoint-config-name xgboost-endpoint-config \
  --production-variants VariantName=AllTraffic,ModelName=xgboost-
model,InitialInstanceCount=1,InstanceType=ml.m5.large

# 8. Create endpoint
aws sagemaker create-endpoint \
  --endpoint-name xgboost-endpoint \
  --endpoint-config-name xgboost-endpoint-config

# 9. Invoke endpoint
aws sagemaker-runtime invoke-endpoint \
  --endpoint-name xgboost-endpoint \
  --body fileb://test_data.csv \
  --content-type text/csv \
  output.json

# 10. Delete endpoint (to save costs)
aws sagemaker delete-endpoint --endpoint-name xgboost-endpoint
aws sagemaker delete-endpoint-config --endpoint-config-name xgboost-endpoint-
config
aws sagemaker delete-model --model-name xgboost-model
```

Cost Optimization for AWS SageMaker

```
"""
AWS SageMaker Cost Optimization Strategies
"""

class SageMakerCostOptimizer:
    """
    Strategies to reduce SageMaker costs
    """

    @staticmethod
    def use_spot_instances(estimator):
        """
        Use spot instances for training (70% cost savings)
        """
        estimator.use_spot_instances = True
        estimator.max_wait = 7200  # Max wait time in seconds
        estimator.max_run = 3600    # Max training time

        print("✓ Spot instances enabled (70% cost savings)")
```

```
        return estimator

    @staticmethod
    def use_managed_spot_training(train_s3):
        """
        Managed spot training with automatic checkpointing
        """
        from sagemaker.xgboost import XGBoost

        xgb = XGBoost(
            entry_point='train.py',
            framework_version='1.5-1',
            instance_type='ml.m5.xlarge',
            use_spot_instances=True,
            max_run=3600,
            max_wait=7200,
            checkpoint_s3_uri='s3://bucket/checkpoints/',  # Auto-resume on
interruption
            checkpoint_local_path='/opt/ml/checkpoints'
        )

        return xgb

    @staticmethod
    def rightsizeinstance(data_size_gb):
        """
        Recommend instance type based on data size
        """
        if data_size_gb < 1:
            return 'ml.m5.large'   # $0.115/hour
        elif data_size_gb < 10:
            return 'ml.m5.xlarge'  # $0.23/hour
        elif data_size_gb < 100:
            return 'ml.m5.2xlarge' # $0.46/hour
        else:
            return 'ml.m5.4xlarge' # $0.92/hour

    @staticmethod
    def use_local_mode_for_development():
        """
        Test locally before cloud training
        """
        from sagemaker.local import LocalSession

        local_session = LocalSession()
        local_session.config = {'local': {'local_code': True}}

        print("✓ Local mode enabled for testing (no cloud costs)")
        return local_session

    @staticmethod
    def auto_shutdown_notebook():
        """
        Auto-shutdown notebook instances when idle
```

```
        """
        lifecycle_config = """
#!/bin/bash
set -e

# Auto-shutdown after 1 hour of idleness
IDLE_TIME=3600

while true; do
    UPTIME=$(awk '{print int($1)}' /proc/uptime)
    if [ $UPTIME -gt $IDLE_TIME ]; then
        echo "Shutting down idle instance"
        sudo shutdown -h now
    fi
    sleep 300
done
"""

        print("✓ Auto-shutdown configured")
        return lifecycle_config

# Cost comparison
print("AWS SageMaker Cost Comparison:")
print("="*60)
print("Training Job (ml.m5.xlarge for 2 hours):")
print("  On-demand:     $0.23/hour × 2 = $0.46")
print("  Spot instance: $0.07/hour × 2 = $0.14 (70% savings)")
print("\nEndpoint (ml.m5.large, 24/7 for 30 days):")
print("  Cost: $0.115/hour × 720 hours = $82.80/month")
print("  Tip: Use batch transform for non-real-time inference")
```

Part 2: Machine Learning on Cloud Platforms (Continued)

ML on GCP (Vertex AI)

Terminology for GCP Vertex AI

Term	Definition
Vertex AI Workbench	Managed Jupyter notebook environment
Vertex AI Training	Managed training service
Vertex AI Prediction	Model deployment service
AutoML Tables	AutoML for tabular data
Vertex AI Pipelines	ML workflow orchestration (Kubeflow)

Term	Definition
Vertex Explainable AI	Model interpretability tools
Vertex AI Feature Store	Managed feature storage
Vertex AI Matching Engine	Vector similarity search
Preemptible VM	70% cheaper compute (can be interrupted)
Custom Container	Bring your own Docker container

GCP Vertex AI Setup

```
"""
GCP Vertex AI - Traditional ML Training & Deployment
"""

from google.cloud import aiplatform
from google.cloud.aiplatform import gapic
import pandas as pd
import numpy as np
from datetime import datetime

class VertexAIMLTrainer:
    """
    Train traditional ML models on GCP Vertex AI
    """

    def __init__(self, project_id, location='us-central1', staging_bucket=None):
        """
        Initialize Vertex AI

        Args:
            project_id: GCP project ID
            location: GCP region
            staging_bucket: GCS bucket for staging (gs://bucket-name)
        """
        self.project_id = project_id
        self.location = location

        if staging_bucket is None:
            staging_bucket = f'gs://{project_id}-vertex-ai'

        self.staging_bucket = staging_bucket

        aiplatform.init(
            project=project_id,
            location=location,
            staging_bucket=staging_bucket
        )

        print(f"✓ Vertex AI initialized")
```

```
        print(f"  Project: {project_id}")
        print(f"  Location: {location}")
        print(f"  Staging Bucket: {staging_bucket}")

    def upload_data_to_gcs(self, train_data, test_data, dataset_name='ml-
dataset'):
        """Upload data to Google Cloud Storage"""
        from google.cloud import storage

        # Save locally
        train_data.to_csv('train.csv', index=False)
        test_data.to_csv('test.csv', index=False)

        # Upload to GCS
        client = storage.Client(project=self.project_id)
        bucket_name = self.staging_bucket.replace('gs://', '')
        bucket = client.bucket(bucket_name)

        train_blob = bucket.blob(f'{dataset_name}/train.csv')
        train_blob.upload_from_filename('train.csv')

        test_blob = bucket.blob(f'{dataset_name}/test.csv')
        test_blob.upload_from_filename('test.csv')

        train_uri = f'gs://{bucket_name}/{dataset_name}/train.csv'
        test_uri = f'gs://{bucket_name}/{dataset_name}/test.csv'

        print(f"✓ Data uploaded to GCS")
        print(f"  Train: {train_uri}")
        print(f"  Test: {test_uri}")

        return train_uri, test_uri

    def train_custom_job_sklearn(self, train_uri, algorithm='random_forest'):
        """Train scikit-learn model with Custom Training"""

        # Create training script
        train_script = """
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
import joblib
import argparse
import os

def train_model(train_path, model_dir, algorithm='random_forest'):
    # Load data
    df = pd.read_csv(train_path)
    X = df.iloc[:, 1:].values
    y = df.iloc[:, 0].values
```

```
    # Split
    X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2,
random_state=42)

    # Train
    if algorithm == 'random_forest':
        model = RandomForestClassifier(n_estimators=100, random_state=42)
    elif algorithm == 'gradient_boosting':
        model = GradientBoostingClassifier(n_estimators=100, random_state=42)
    else:
        model = LogisticRegression(max_iter=1000)

    model.fit(X_train, y_train)

    # Evaluate
    y_pred = model.predict(X_val)
    accuracy = accuracy_score(y_val, y_pred)
    print(f"Validation Accuracy: {accuracy:.4f}")

    # Save model
    model_path = os.path.join(model_dir, 'model.joblib')
    joblib.dump(model, model_path)
    print(f"Model saved to {model_path}")

if __name__ == '__main__':
    parser = argparse.ArgumentParser()
    parser.add_argument('--train-data', required=True)
    parser.add_argument('--model-dir', default=os.environ.get('AIP_MODEL_DIR',
'/tmp'))
    parser.add_argument('--algorithm', default='random_forest')

    args = parser.parse_args()
    train_model(args.train_data, args.model_dir, args.algorithm)
"""

        # Save script
        with open('train.py', 'w') as f:
            f.write(train_script)

        # Create custom training job
        job = aiplatform.CustomTrainingJob(
            display_name=f'sklearn-{algorithm}-{datetime.now().strftime("%Y%m%d-
%H%M%S")}',
            script_path='train.py',
            container_uri='us-docker.pkg.dev/vertex-ai/training/scikit-learn-
cpu.1-0:latest',
            requirements=['pandas', 'scikit-learn', 'joblib'],
            model_serving_container_image_uri='us-docker.pkg.dev/vertex-
ai/prediction/sklearn-cpu.1-0:latest'
        )

        # Run training
        print(f"Training {algorithm} model on Vertex AI...")
```

```
        model = job.run(
            replica_count=1,
            machine_type='n1-standard-4',
            args=[
                f'--train-data={train_uri}',
                f'--algorithm={algorithm}'
            ],
            base_output_dir=f'{self.staging_bucket}/output'
        )

        print(f"✓ Model trained: {model.resource_name}")

        return model

    def train_automl_tables(self, dataset_uri, target_column):
        """Train with AutoML Tables"""

        # Create dataset
        dataset = aiplatform.TabularDataset.create(
            display_name='automl-dataset',
            gcs_source=dataset_uri
        )

        print(f"✓ Dataset created: {dataset.resource_name}")

        # Train AutoML model
        print("Training AutoML model...")

        job = aiplatform.AutoMLTabularTrainingJob(
            display_name='automl-training',
            optimization_prediction_type='classification',
            optimization_objective='maximize-au-prc'
        )

        model = job.run(
            dataset=dataset,
            target_column=target_column,
            training_fraction_split=0.8,
            validation_fraction_split=0.1,
            test_fraction_split=0.1,
            budget_milli_node_hours=1000,  # 1 hour
            model_display_name='automl-model'
        )

        print(f"✓ AutoML model trained: {model.resource_name}")

        return model

    def hyperparameter_tuning(self, train_uri):
        """Hyperparameter tuning with Vertex AI"""

        from google.cloud.aiplatform import hyperparameter_tuning as hpt

        # Training script with hyperparameters
```

```
        train_script = """
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import argparse
import hypertune

def train_model(train_path, n_estimators, max_depth):
    df = pd.read_csv(train_path)
    X = df.iloc[:, 1:].values
    y = df.iloc[:, 0].values

    X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2,
random_state=42)

    model = RandomForestClassifier(
        n_estimators=n_estimators,
        max_depth=max_depth,
        random_state=42
    )
    model.fit(X_train, y_train)

    y_pred = model.predict(X_val)
    accuracy = accuracy_score(y_val, y_pred)

    # Report metric to Vertex AI
    hpt_client = hypertune.HyperTune()
    hpt_client.report_hyperparameter_tuning_metric(
        hyperparameter_metric_tag='accuracy',
        metric_value=accuracy
    )

    print(f"Accuracy: {accuracy:.4f}")

if __name__ == '__main__':
    parser = argparse.ArgumentParser()
    parser.add_argument('--train-data', required=True)
    parser.add_argument('--n-estimators', type=int, default=100)
    parser.add_argument('--max-depth', type=int, default=10)

    args = parser.parse_args()
    train_model(args.train_data, args.n_estimators, args.max_depth)
"""

        with open('train_hpt.py', 'w') as f:
            f.write(train_script)

        # Custom job for hyperparameter tuning
        job = aiplatform.CustomJob.from_local_script(
            display_name='hpt-job',
            script_path='train_hpt.py',
            container_uri='us-docker.pkg.dev/vertex-ai/training/scikit-learn-
cpu.1-0:latest',
```

```
            requirements=['pandas', 'scikit-learn', 'cloudml-hypertune'],
            replica_count=1,
            machine_type='n1-standard-4',
            args=[f'--train-data={train_uri}']
        )

        # Hyperparameter tuning job
        hpt_job = aiplatform.HyperparameterTuningJob(
            display_name='hpt-sklearn',
            custom_job=job,
            metric_spec={'accuracy': 'maximize'},
            parameter_spec={
                'n_estimators': hpt.IntegerParameterSpec(min=50, max=200,
scale='linear'),
                'max_depth': hpt.IntegerParameterSpec(min=5, max=20,
scale='linear')
            },
            max_trial_count=20,
            parallel_trial_count=5
        )

        print("Starting hyperparameter tuning...")
        hpt_job.run()

        print(f"✓ Best trial: {hpt_job.trials[0]}")

        return hpt_job

    def deploy_model(self, model, endpoint_name='ml-endpoint'):
        """Deploy model to endpoint"""

        # Create endpoint
        endpoint = aiplatform.Endpoint.create(display_name=endpoint_name)

        print(f"Deploying model to endpoint...")

        # Deploy
        model.deploy(
            endpoint=endpoint,
            deployed_model_display_name=endpoint_name,
            machine_type='n1-standard-2',
            min_replica_count=1,
            max_replica_count=5,
            traffic_percentage=100
        )

        print(f"✓ Model deployed: {endpoint.resource_name}")

        return endpoint

    def batch_prediction(self, model, input_uri, output_uri):
        """Batch prediction"""

        print("Running batch prediction...")
```

```
        batch_job = model.batch_predict(
            job_display_name='batch-prediction',
            gcs_source=input_uri,
            gcs_destination_prefix=output_uri,
            machine_type='n1-standard-4',
            accelerator_count=0
        )

        batch_job.wait()

        print(f"✓ Batch prediction complete: {output_uri}")

        return batch_job

# Example usage
if __name__ == "__main__":
    # Initialize
    trainer = VertexAIMLTrainer(
        project_id='your-project-id',
        location='us-central1'
    )

    # Prepare data
    from sklearn.datasets import make_classification
    X, y = make_classification(n_samples=1000, n_features=20, random_state=42)

    train_df = pd.DataFrame(X[:800])
    train_df.insert(0, 'target', y[:800])

    test_df = pd.DataFrame(X[800:])
    test_df.insert(0, 'target', y[800:])

    # Upload to GCS
    train_uri, test_uri = trainer.upload_data_to_gcs(train_df, test_df)

    # Train custom sklearn model
    model = trainer.train_custom_job_sklearn(train_uri, algorithm='random_forest')

    # Deploy
    endpoint = trainer.deploy_model(model)

    # Predict
    predictions = endpoint.predict(instances=test_df.iloc[:, 1:].values.tolist())
    print(f"Predictions: {predictions.predictions[:5]}")
```

GCP Vertex AI CLI Commands

```
#!/bin/bash
# GCP Vertex AI CLI operations
```

```
# Setup
export PROJECT_ID="your-project-id"
export REGION="us-central1"
export BUCKET_NAME="gs://${PROJECT_ID}-vertex-ai"

# 1. Create bucket
gsutil mb -l $REGION $BUCKET_NAME

# 2. Upload training data
gsutil cp train.csv ${BUCKET_NAME}/data/train.csv
gsutil cp test.csv ${BUCKET_NAME}/data/test.csv

# 3. Create custom training job
gcloud ai custom-jobs create \
  --region=$REGION \
  --display-name=sklearn-training \
  --worker-pool-spec=machine-type=n1-standard-4,replica-count=1,container-image-
uri=us-docker.pkg.dev/vertex-ai/training/scikit-learn-cpu.1-0:latest,python-
module=train \
  --args="--train-data=${BUCKET_NAME}/data/train.csv"

# 4. List training jobs
gcloud ai custom-jobs list --region=$REGION

# 5. Create AutoML dataset
gcloud ai datasets create \
  --region=$REGION \
  --display-name=automl-dataset \
  --metadata-schema-uri=gs://google-cloud-
aiplatform/schema/dataset/metadata/tabular_1.0.0.yaml \
  --tabular-source=${BUCKET_NAME}/data/train.csv

# 6. Train AutoML model
gcloud ai model-training-jobs create \
  --region=$REGION \
  --display-name=automl-training \
  --dataset=<dataset-id> \
  --target-column=target \
  --prediction-type=classification \
  --optimization-objective=maximize-au-prc

# 7. Deploy model to endpoint
gcloud ai endpoints create \
  --region=$REGION \
  --display-name=ml-endpoint

gcloud ai endpoints deploy-model <endpoint-id> \
  --region=$REGION \
  --model=<model-id> \
  --display-name=deployed-model \
  --machine-type=n1-standard-2 \
  --min-replica-count=1 \
  --max-replica-count=5
```

```
# 8. Make prediction
gcloud ai endpoints predict <endpoint-id> \
  --region=$REGION \
  --json-request=request.json

# 9. Batch prediction
gcloud ai batch-prediction-jobs create \
  --region=$REGION \
  --model=<model-id> \
  --input-uri=${BUCKET_NAME}/data/test.csv \
  --output-uri=${BUCKET_NAME}/predictions/

# 10. Cleanup
gcloud ai endpoints undeploy-model <endpoint-id> --region=$REGION --deployed-
model-id=<deployed-model-id>
gcloud ai endpoints delete <endpoint-id> --region=$REGION
gcloud ai models delete <model-id> --region=$REGION
```

GCP Cost Optimization

```
"""
GCP Vertex AI Cost Optimization
"""

class VertexAICostOptimizer:
    """
    Strategies to reduce Vertex AI costs
    """

    @staticmethod
    def use_preemptible_vms():
        """
        Use preemptible VMs for training (70% cost savings)
        """
        job = aiplatform.CustomTrainingJob(
            display_name='preemptible-training',
            script_path='train.py',
            container_uri='us-docker.pkg.dev/vertex-ai/training/scikit-learn-
cpu.1-0:latest'
        )

        model = job.run(
            replica_count=1,
            machine_type='n1-standard-4',
            accelerator_type=None,
            # Use preemptible VMs
            scheduling={
                'restart_job_on_worker_restart': True,
                'timeout': '3600s'
            }
        )
```

```
        print("✓ Preemptible VMs enabled (70% cost savings)")
        return model

    @staticmethod
    def use_cpu_for_tabular_data():
        """
        Use CPU instead of GPU for tabular data
        """
        print("Cost comparison:")
        print("  n1-standard-4 (CPU): $0.19/hour")
        print("  n1-standard-4 + T4 GPU: $0.54/hour")
        print("  Recommendation: Use CPU for tabular ML (3x cheaper)")

        return 'n1-standard-4'  # CPU only

    @staticmethod
    def rightsize_endpoint(requests_per_day):
        """
        Recommend endpoint configuration based on traffic
        """
        if requests_per_day < 1000:
            config = {
                'machine_type': 'n1-standard-2',
                'min_replicas': 1,
                'max_replicas': 1,
                'cost_per_month': '$35'
            }
        elif requests_per_day < 10000:
            config = {
                'machine_type': 'n1-standard-2',
                'min_replicas': 1,
                'max_replicas': 3,
                'cost_per_month': '$70'
            }
        else:
            config = {
                'machine_type': 'n1-standard-4',
                'min_replicas': 2,
                'max_replicas': 10,
                'cost_per_month': '$300'
            }

        print(f"Recommended configuration for {requests_per_day} requests/day:")
        for key, value in config.items():
            print(f"  {key}: {value}")

        return config

    @staticmethod
    def use_batch_vs_online():
        """
        Compare batch vs online prediction costs
        """
```

```
        print("Cost comparison (1M predictions):")
        print("  Online endpoint (n1-standard-2, 24/7): $35/month base +
$0.00001/prediction = $45")
        print("  Batch prediction: $0.00002/prediction = $20")
        print("  Recommendation: Use batch for non-real-time use cases")

# Cost comparison
print("\nGCP Vertex AI Cost Comparison:")
print("="*60)
print("Training Job (n1-standard-4 for 2 hours):")
print("  Regular:      $0.19/hour × 2 = $0.38")
print("  Preemptible:  $0.06/hour × 2 = $0.12 (70% savings)")
print("\nEndpoint (n1-standard-2, 24/7 for 30 days):")
print("  Cost: $0.048/hour × 720 hours = $34.56/month")
```

ML on Azure (Azure ML)

Terminology for Azure ML

Term	Definition
Azure ML Studio	Web-based IDE for ML
Azure ML Workspace	Container for ML assets
Compute Instance	Development VM with Jupyter
Compute Cluster	Scalable training cluster
Designer	Drag-and-drop ML pipeline builder
Automated ML	AutoML service
MLflow	Open-source experiment tracking (integrated)
Managed Endpoint	Deployed model service
Spot VM	80% cheaper compute
Responsible AI Dashboard	Fairness and explainability tools

Azure ML Setup

```
"""
Azure ML - Traditional ML Training & Deployment
"""

from azureml.core import Workspace, Dataset, Environment, Experiment,
ScriptRunConfig
from azureml.core.compute import ComputeTarget, AmlCompute
from azureml.core.model import Model
from azureml.core.webservice import AciWebservice, Webservice
```

```
from azureml.train.sklearn import SKLearn
from azureml.train.hyperdrive import RandomParameterSampling, BanditPolicy,
HyperDriveConfig, PrimaryMetricGoal
from azureml.train.hyperdrive import choice, uniform
import pandas as pd
import numpy as np

class AzureMLTrainer:
    """
    Train traditional ML models on Azure ML
    """

    def __init__(self, subscription_id, resource_group, workspace_name):
        """
        Initialize Azure ML Workspace

        Args:
            subscription_id: Azure subscription ID
            resource_group: Resource group name
            workspace_name: ML workspace name
        """
        self.ws = Workspace(subscription_id, resource_group, workspace_name)

        print(f"✓ Azure ML Workspace loaded")
        print(f"  Name: {self.ws.name}")
        print(f"  Location: {self.ws.location}")
        print(f"  Resource Group: {self.ws.resource_group}")

    def create_compute_cluster(self, cluster_name='cpu-cluster',
vm_size='STANDARD_D2_V2',
                               max_nodes=4, use_spot=True):
        """Create compute cluster for training"""

        try:
            compute_target = ComputeTarget(workspace=self.ws, name=cluster_name)
            print(f"✓ Found existing compute cluster: {cluster_name}")
        except:
            print(f"Creating new compute cluster: {cluster_name}")

            compute_config = AmlCompute.provisioning_configuration(
                vm_size=vm_size,
                max_nodes=max_nodes,
                vm_priority='lowpriority' if use_spot else 'dedicated'  # Spot VMs
            )

            compute_target = ComputeTarget.create(self.ws, cluster_name,
compute_config)
            compute_target.wait_for_completion(show_output=True)

            print(f"✓ Compute cluster created: {cluster_name}")

        return compute_target

    def upload_data_to_datastore(self, train_data, test_data, dataset_name='ml-
```

```
dataset'):
        """Upload data to Azure ML datastore"""

        # Save locally
        train_data.to_csv('train.csv', index=False)
        test_data.to_csv('test.csv', index=False)

        # Get default datastore
        datastore = self.ws.get_default_datastore()

        # Upload
        datastore.upload_files(
            files=['train.csv', 'test.csv'],
            target_path=dataset_name,
            overwrite=True
        )

        # Create dataset
        train_dataset = Dataset.Tabular.from_delimited_files(
            path=[(datastore, f'{dataset_name}/train.csv')]
        )

        test_dataset = Dataset.Tabular.from_delimited_files(
            path=[(datastore, f'{dataset_name}/test.csv')]
        )

        # Register datasets
        train_dataset = train_dataset.register(
            workspace=self.ws,
            name=f'{dataset_name}_train',
            create_new_version=True
        )

        test_dataset = test_dataset.register(
            workspace=self.ws,
            name=f'{dataset_name}_test',
            create_new_version=True
        )

        print(f"✓ Data uploaded to Azure ML")
        print(f"  Train dataset: {train_dataset.name}")
        print(f"  Test dataset: {test_dataset.name}")

        return train_dataset, test_dataset

    def train_sklearn_model(self, train_dataset, compute_target,
algorithm='random_forest'):
        """Train scikit-learn model"""

        # Create training script
        train_script = """
import argparse
import pandas as pd
import numpy as np
```

```
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
import joblib
from azureml.core import Run

def train_model(data_path, algorithm='random_forest'):
    # Get run context
    run = Run.get_context()

    # Load data
    df = pd.read_csv(data_path)
    X = df.iloc[:, 1:].values
    y = df.iloc[:, 0].values

    # Split
    X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2,
random_state=42)

    # Train
    if algorithm == 'random_forest':
        model = RandomForestClassifier(n_estimators=100, random_state=42)
    elif algorithm == 'gradient_boosting':
        model = GradientBoostingClassifier(n_estimators=100, random_state=42)
    else:
        model = LogisticRegression(max_iter=1000)

    model.fit(X_train, y_train)

    # Evaluate
    y_pred = model.predict(X_val)
    accuracy = accuracy_score(y_val, y_pred)

    print(f"Validation Accuracy: {accuracy:.4f}")

    # Log metrics
    run.log('accuracy', accuracy)

    # Save model
    joblib.dump(model, 'outputs/model.joblib')
    print("Model saved to outputs/model.joblib")

if __name__ == '__main__':
    parser = argparse.ArgumentParser()
    parser.add_argument('--data-path', type=str, required=True)
    parser.add_argument('--algorithm', type=str, default='random_forest')

    args = parser.parse_args()
    train_model(args.data_path, args.algorithm)
"""

        # Save script
        import os
```

```
        os.makedirs('training', exist_ok=True)
        with open('training/train.py', 'w') as f:
            f.write(train_script)

        # Create environment
        env = Environment.from_conda_specification(
            name='sklearn-env',
            file_path='environment.yml'  # Or create inline
        )

        # Or create environment inline
        env = Environment(name='sklearn-env')
        env.python.conda_dependencies.add_pip_package('scikit-learn')
        env.python.conda_dependencies.add_pip_package('pandas')
        env.python.conda_dependencies.add_pip_package('numpy')
        env.python.conda_dependencies.add_pip_package('azureml-core')

        # Script run configuration
        src = ScriptRunConfig(
            source_directory='training',
            script='train.py',
            arguments=[
                '--data-path',
train_dataset.as_named_input('training_data').as_download(),
                '--algorithm', algorithm
            ],
            compute_target=compute_target,
            environment=env
        )

        # Submit experiment
        experiment = Experiment(self.ws, f'sklearn-{algorithm}')

        print(f"Submitting training job for {algorithm}...")
        run = experiment.submit(src)
        run.wait_for_completion(show_output=True)

        print(f"✓ Training complete")
        print(f"  Run ID: {run.id}")
        print(f"  Accuracy: {run.get_metrics()['accuracy']:.4f}")

        return run

    def automated_ml(self, train_dataset, compute_target, task='classification'):
        """Train with Automated ML"""

        from azureml.train.automl import AutoMLConfig

        # AutoML configuration
        automl_config = AutoMLConfig(
            task=task,
            primary_metric='accuracy' if task == 'classification' else 'r2_score',
            training_data=train_dataset,
            label_column_name='target',
```

```
            n_cross_validations=5,
            enable_early_stopping=True,
            iteration_timeout_minutes=10,
            max_concurrent_iterations=4,
            compute_target=compute_target,
            experiment_timeout_hours=1
        )

        # Submit experiment
        experiment = Experiment(self.ws, 'automl-experiment')

        print("Starting Automated ML...")
        automl_run = experiment.submit(automl_config, show_output=True)
        automl_run.wait_for_completion(show_output=True)

        # Get best model
        best_run, fitted_model = automl_run.get_output()

        print(f"✓ AutoML complete")
        print(f"  Best algorithm: {best_run.properties['run_algorithm']}")
        print(f"  Best accuracy: {best_run.get_metrics()['accuracy']:.4f}")

        return best_run, fitted_model

    def hyperparameter_tuning(self, train_dataset, compute_target):
        """Hyperparameter tuning with HyperDrive"""

        # Training script with hyperparameters
        train_script = """
import argparse
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import pandas as pd
from azureml.core import Run

def train_model(data_path, n_estimators, max_depth):
    run = Run.get_context()

    df = pd.read_csv(data_path)
    X = df.iloc[:, 1:].values
    y = df.iloc[:, 0].values

    X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2,
random_state=42)

    model = RandomForestClassifier(
        n_estimators=n_estimators,
        max_depth=max_depth,
        random_state=42
    )
    model.fit(X_train, y_train)

    y_pred = model.predict(X_val)
```

```
    accuracy = accuracy_score(y_val, y_pred)

    run.log('accuracy', accuracy)
    print(f"Accuracy: {accuracy:.4f}")

if __name__ == '__main__':
    parser = argparse.ArgumentParser()
    parser.add_argument('--data-path', required=True)
    parser.add_argument('--n-estimators', type=int, default=100)
    parser.add_argument('--max-depth', type=int, default=10)

    args = parser.parse_args()
    train_model(args.data_path, args.n_estimators, args.max_depth)
"""

        with open('training/train_hpt.py', 'w') as f:
            f.write(train_script)

        # Environment
        env = Environment(name='sklearn-env')
        env.python.conda_dependencies.add_pip_package('scikit-learn')
        env.python.conda_dependencies.add_pip_package('pandas')
        env.python.conda_dependencies.add_pip_package('azureml-core')

        # Script config
        src = ScriptRunConfig(
            source_directory='training',
            script='train_hpt.py',
            arguments=[
                '--data-path',
train_dataset.as_named_input('training_data').as_download()
            ],
            compute_target=compute_target,
            environment=env
        )

        # Hyperparameter sampling
        param_sampling = RandomParameterSampling({
            '--n-estimators': choice(50, 100, 150, 200),
            '--max-depth': choice(5, 10, 15, 20, None)
        })

        # Early termination policy
        policy = BanditPolicy(evaluation_interval=2, slack_factor=0.1)

        # HyperDrive configuration
        hyperdrive_config = HyperDriveConfig(
            run_config=src,
            hyperparameter_sampling=param_sampling,
            policy=policy,
            primary_metric_name='accuracy',
            primary_metric_goal=PrimaryMetricGoal.MAXIMIZE,
            max_total_runs=20,
            max_concurrent_runs=4
```

```
        )

        # Submit experiment
        experiment = Experiment(self.ws, 'hyperdrive-experiment')

        print("Starting hyperparameter tuning...")
        hyperdrive_run = experiment.submit(hyperdrive_config)
        hyperdrive_run.wait_for_completion(show_output=True)

        # Get best run
        best_run = hyperdrive_run.get_best_run_by_primary_metric()
        best_metrics = best_run.get_metrics()

        print(f"✓ HyperDrive complete")
        print(f"  Best accuracy: {best_metrics['accuracy']:.4f}")
        print(f"  Best parameters: {best_run.get_details()['runDefinition']
['arguments']}")

        return hyperdrive_run, best_run

    def register_model(self, run, model_name='sklearn-model'):
        """Register model"""

        model = run.register_model(
            model_name=model_name,
            model_path='outputs/model.joblib'
        )

        print(f"✓ Model registered: {model.name} (version {model.version})")

        return model

    def deploy_model(self, model, service_name='ml-service'):
        """Deploy model to Azure Container Instance"""

        # Inference script
        inference_script = """
import json
import joblib
import numpy as np
from azureml.core.model import Model

def init():
    global model
    model_path = Model.get_model_path('sklearn-model')
    model = joblib.load(model_path)

def run(raw_data):
    data = json.loads(raw_data)['data']
    predictions = model.predict(np.array(data))
    return json.dumps(predictions.tolist())
"""

        with open('score.py', 'w') as f:
```

```
            f.write(inference_script)

        # Environment
        env = Environment(name='inference-env')
        env.python.conda_dependencies.add_pip_package('scikit-learn')
        env.python.conda_dependencies.add_pip_package('numpy')
        env.python.conda_dependencies.add_pip_package('azureml-core')

        # Deployment configuration
        aci_config = AciWebservice.deploy_configuration(
            cpu_cores=1,
            memory_gb=1,
            auth_enabled=True
        )

        # Deploy
        print(f"Deploying model to {service_name}...")

        service = Model.deploy(
            workspace=self.ws,
            name=service_name,
            models=[model],
            inference_config=InferenceConfig(entry_script='score.py',
environment=env),
            deployment_config=aci_config
        )

        service.wait_for_deployment(show_output=True)

        print(f"✓ Model deployed")
        print(f"  Scoring URI: {service.scoring_uri}")

        return service

# Example usage
if __name__ == "__main__":
    # Initialize
    trainer = AzureMLTrainer(
        subscription_id='your-subscription-id',
        resource_group='your-resource-group',
        workspace_name='your-workspace'
    )

    # Create compute cluster
    compute_target = trainer.create_compute_cluster(
        cluster_name='cpu-cluster',
        use_spot=True  # 80% cost savings
    )

    # Prepare data
    from sklearn.datasets import make_classification
    X, y = make_classification(n_samples=1000, n_features=20, random_state=42)

    train_df = pd.DataFrame(X[:800])
```

```
    train_df.insert(0, 'target', y[:800])

    test_df = pd.DataFrame(X[800:])
    test_df.insert(0, 'target', y[800:])

    # Upload data
    train_dataset, test_dataset = trainer.upload_data_to_datastore(train_df,
test_df)

    # Train model
    run = trainer.train_sklearn_model(train_dataset, compute_target,
algorithm='random_forest')

    # Register model
    model = trainer.register_model(run)

    # Deploy
    service = trainer.deploy_model(model)

    # Predict
    import requests
    import json

    headers = {'Content-Type': 'application/json'}
    test_data = {'data': test_df.iloc[:5, 1:].values.tolist()}

    response = requests.post(service.scoring_uri, json=test_data, headers=headers)
    print(f"Predictions: {response.json()}")
```

Azure ML CLI Commands

```
#!/bin/bash
# Azure ML CLI operations

# Setup
export SUBSCRIPTION_ID="your-subscription-id"
export RESOURCE_GROUP="ml-resource-group"
export WORKSPACE="ml-workspace"
export LOCATION="eastus"

# 1. Create resource group
az group create --name $RESOURCE_GROUP --location $LOCATION

# 2. Create workspace
az ml workspace create \
  --name $WORKSPACE \
  --resource-group $RESOURCE_GROUP \
  --location $LOCATION

# 3. Create compute cluster
az ml compute create \
```

```
  --name cpu-cluster \
  --type AmlCompute \
  --size STANDARD_D2_V2 \
  --min-instances 0 \
  --max-instances 4 \
  --resource-group $RESOURCE_GROUP \
  --workspace-name $WORKSPACE

# 4. Upload data
az ml data create \
  --name training-data \
  --version 1 \
  --type uri_file \
  --path train.csv \
  --resource-group $RESOURCE_GROUP \
  --workspace-name $WORKSPACE

# 5. Submit training job
az ml job create \
  --file job.yml \
  --resource-group $RESOURCE_GROUP \
  --workspace-name $WORKSPACE

# 6. List jobs
az ml job list \
  --resource-group $RESOURCE_GROUP \
  --workspace-name $WORKSPACE

# 7. Register model
az ml model create \
  --name sklearn-model \
  --version 1 \
  --path ./outputs/model.joblib \
  --resource-group $RESOURCE_GROUP \
  --workspace-name $WORKSPACE

# 8. Create managed endpoint
az ml online-endpoint create \
  --name ml-endpoint \
  --resource-group $RESOURCE_GROUP \
  --workspace-name $WORKSPACE

# 9. Deploy model to endpoint
az ml online-deployment create \
  --name blue \
  --endpoint ml-endpoint \
  --model sklearn-model:1 \
  --instance-type Standard_DS2_v2 \
  --instance-count 1 \
  --resource-group $RESOURCE_GROUP \
  --workspace-name $WORKSPACE

# 10. Invoke endpoint
az ml online-endpoint invoke \
```

```
  --name ml-endpoint \
  --request-file request.json \
  --resource-group $RESOURCE_GROUP \
  --workspace-name $WORKSPACE

# 11. Cleanup
az ml online-endpoint delete --name ml-endpoint --resource-group $RESOURCE_GROUP -
-workspace-name $WORKSPACE -y
az ml compute delete --name cpu-cluster --resource-group $RESOURCE_GROUP --
workspace-name $WORKSPACE -y
```

Azure ML Cost Optimization

```
"""
Azure ML Cost Optimization
"""

class AzureMLCostOptimizer:
    """
    Strategies to reduce Azure ML costs
    """

    @staticmethod
    def use_spot_vms():
        """
        Use low-priority (spot) VMs for training (80% cost savings)
        """
        compute_config = AmlCompute.provisioning_configuration(
            vm_size='STANDARD_D2_V2',
            max_nodes=4,
            vm_priority='lowpriority'  # Spot VMs
        )

        print("✓ Spot VMs enabled (80% cost savings)")
        print("  Regular D2v2: $0.14/hour")
        print("  Spot D2v2: $0.028/hour")

        return compute_config

    @staticmethod
    def auto_shutdown_compute():
        """
        Auto-shutdown compute instances when idle
        """
        compute_config = AmlCompute.provisioning_configuration(
            vm_size='STANDARD_D2_V2',
            min_nodes=0,  # Scale to zero
            max_nodes=4,
            idle_seconds_before_scaledown=300  # 5 minutes
        )
```

```
            print("✓ Auto-shutdown enabled")
            print("  Scales to 0 nodes after 5 minutes of idleness")

            return compute_config

        @staticmethod
        def use_container_instances_for_dev():
            """
            Use Azure Container Instances (ACI) for dev/test deployments
            """
            aci_config = AciWebservice.deploy_configuration(
                cpu_cores=1,
                memory_gb=1
            )

            print("Cost comparison (24/7 for 30 days):")
            print("  ACI (1 CPU, 1GB): ~$30/month")
            print("  AKS (managed endpoint): ~$100/month")
            print("  Recommendation: Use ACI for dev/test, AKS for production")

            return aci_config

        @staticmethod
        def use_batch_inference():
            """
            Use batch inference for non-real-time predictions
            """
            print("Cost comparison (1M predictions):")
            print("  Online endpoint: $100/month base + compute")
            print("  Batch inference: Pay only for compute time (~$10)")
            print("  Savings: ~$90/month")

    # Cost comparison
    print("\nAzure ML Cost Comparison:")
    print("="*60)
    print("Training Job (STANDARD_D2_V2 for 2 hours):")
    print("  Regular: $0.14/hour × 2 = $0.28")
    print("  Spot:    $0.028/hour × 2 = $0.056 (80% savings)")
    print("\nManaged Endpoint (Standard_DS2_v2, 24/7 for 30 days):")
    print("  Cost: $0.14/hour × 720 hours = $100.80/month")
```

ML Deployment Strategies

Terminology for ML Deployment

Term	Definition
Online Inference	Real-time predictions via API
Batch Inference	Predictions on large datasets offline
A/B Testing	Comparing two model versions

Term	Definition
Canary Deployment	Gradual rollout (e.g., 10% traffic)
Blue-Green Deployment	Switch between two environments
Model Versioning	Tracking different model iterations
Shadow Deployment	New model runs parallel without serving
Feature Store	Centralized feature management
Model Monitoring	Tracking model performance over time
Data Drift	Changes in input data distribution
Concept Drift	Changes in target variable relationship

Deployment Comparison Across Clouds

Feature	AWS SageMaker	GCP Vertex AI	Azure ML
Real-time Endpoint	SageMaker Endpoint	Vertex AI Endpoint	Managed Online Endpoint
Batch Prediction	Batch Transform	Batch Prediction	Batch Endpoint
Auto-scaling	Yes	Yes	Yes
A/B Testing	Traffic splitting	Traffic splitting	Blue-green deployment
Monitoring	CloudWatch	Cloud Monitoring	Application Insights
Min Cost (real-time)	$35/month	$35/month	$30/month
GPU Support	Yes	Yes	Yes
Serverless Option	Lambda (via API Gateway)	Cloud Run	Azure Functions

Note: Costs are approximate and subject to change. Verify current pricing with cloud providers.

Multi-Cloud Deployment Strategy

```
"""
Multi-Cloud ML Deployment Strategy
"""

from abc import ABC, abstractmethod
import requests
import json

class MLModelDeployment(ABC):
    """
    Abstract base class for ML model deployment
    """

    @abstractmethod
```

```
    def deploy(self, model_path, endpoint_name):
        """Deploy model to cloud"""
        pass

    @abstractmethod
    def predict(self, endpoint_url, data):
        """Make prediction"""
        pass

    @abstractmethod
    def update_traffic(self, endpoint_name, traffic_split):
        """Update traffic distribution for A/B testing"""
        pass

    @abstractmethod
    def get_metrics(self, endpoint_name):
        """Get endpoint metrics"""
        pass

class AWSDeployment(MLModelDeployment):
    """AWS SageMaker deployment"""

    def __init__(self, region='us-east-1'):
        import boto3
        self.sagemaker = boto3.client('sagemaker', region_name=region)
        self.runtime = boto3.client('sagemaker-runtime', region_name=region)

    def deploy(self, model_path, endpoint_name):
        """Deploy to SageMaker endpoint"""
        # Implementation here
        pass

    def predict(self, endpoint_name, data):
        """Invoke SageMaker endpoint"""
        response = self.runtime.invoke_endpoint(
            EndpointName=endpoint_name,
            Body=json.dumps(data),
            ContentType='application/json'
        )
        return json.loads(response['Body'].read())

    def update_traffic(self, endpoint_name, traffic_split):
        """Update traffic split"""
        self.sagemaker.update_endpoint_weights_and_capacities(
            EndpointName=endpoint_name,
            DesiredWeightsAndCapacities=[
                {'VariantName': 'variant-1', 'DesiredWeight': traffic_split[0]},
                {'VariantName': 'variant-2', 'DesiredWeight': traffic_split[1]}
            ]
        )

class GCPDeployment(MLModelDeployment):
    """GCP Vertex AI deployment"""
```

```
    def __init__(self, project_id, location='us-central1'):
        from google.cloud import aiplatform
        aiplatform.init(project=project_id, location=location)
        self.project_id = project_id
        self.location = location

    def deploy(self, model_path, endpoint_name):
        """Deploy to Vertex AI endpoint"""
        from google.cloud import aiplatform

        model = aiplatform.Model.upload(
            display_name=endpoint_name,
            artifact_uri=model_path
        )

        endpoint = model.deploy(
            deployed_model_display_name=endpoint_name,
            machine_type='n1-standard-2'
        )

        return endpoint

    def predict(self, endpoint, data):
        """Invoke Vertex AI endpoint"""
        predictions = endpoint.predict(instances=data)
        return predictions.predictions

class AzureDeployment(MLModelDeployment):
    """Azure ML deployment"""

    def __init__(self, subscription_id, resource_group, workspace_name):
        from azureml.core import Workspace
        self.ws = Workspace(subscription_id, resource_group, workspace_name)

    def deploy(self, model_path, service_name):
        """Deploy to Azure ML"""
        from azureml.core.model import Model
        from azureml.core.webservice import AciWebservice

        model = Model.register(
            workspace=self.ws,
            model_path=model_path,
            model_name=service_name
        )

        aci_config = AciWebservice.deploy_configuration(cpu_cores=1, memory_gb=1)

        service = Model.deploy(
            workspace=self.ws,
            name=service_name,
            models=[model],
            deployment_config=aci_config
        )
```

```
        return service

    def predict(self, service, data):
        """Invoke Azure ML service"""
        predictions = service.run(json.dumps({'data': data}))
        return json.loads(predictions)

# Multi-cloud deployment manager
class MultiCloudDeploymentManager:
    """
    Manage deployments across multiple clouds
    """

    def __init__(self):
        self.deployments = {}

    def add_deployment(self, cloud_provider, deployment):
        """Add cloud deployment"""
        self.deployments[cloud_provider] = deployment

    def deploy_to_all(self, model_path, endpoint_name):
        """Deploy to all configured clouds"""
        results = {}

        for provider, deployment in self.deployments.items():
            print(f"Deploying to {provider}...")
            try:
                endpoint = deployment.deploy(model_path, endpoint_name)
                results[provider] = {'status': 'success', 'endpoint': endpoint}
            except Exception as e:
                results[provider] = {'status': 'failed', 'error': str(e)}

        return results

    def predict_with_fallback(self, data, primary='aws', fallback='gcp'):
        """Predict with fallback to another cloud if primary fails"""
        try:
            return self.deployments[primary].predict(data)
        except Exception as e:
            print(f"{primary} failed: {e}. Trying {fallback}...")
            return self.deployments[fallback].predict(data)

    def distribute_load(self, data, distribution={'aws': 0.5, 'gcp': 0.3, 'azure':
0.2}):
        """Distribute prediction load across clouds"""
        import random

        # Random selection based on distribution
        rand = random.random()
        cumulative = 0

        for cloud, weight in distribution.items():
            cumulative += weight
            if rand <= cumulative:
```

```
                return self.deployments[cloud].predict(data), cloud

        # Fallback
        return self.deployments['aws'].predict(data), 'aws'

# Example usage
if __name__ == "__main__":
    # Initialize multi-cloud manager
    manager = MultiCloudDeploymentManager()

    # Add cloud deployments
    manager.add_deployment('aws', AWSDeployment(region='us-east-1'))
    manager.add_deployment('gcp', GCPDeployment(project_id='my-project'))
    manager.add_deployment('azure', AzureDeployment('sub-id', 'rg', 'ws'))

    # Deploy to all clouds
    results = manager.deploy_to_all('model.pkl', 'ml-model-v1')

    # Predict with fallback
    prediction, cloud = manager.predict_with_fallback([1, 2, 3, 4])
    print(f"Prediction from {cloud}: {prediction}")
```

Part 3: Large Language Models (LLMs)

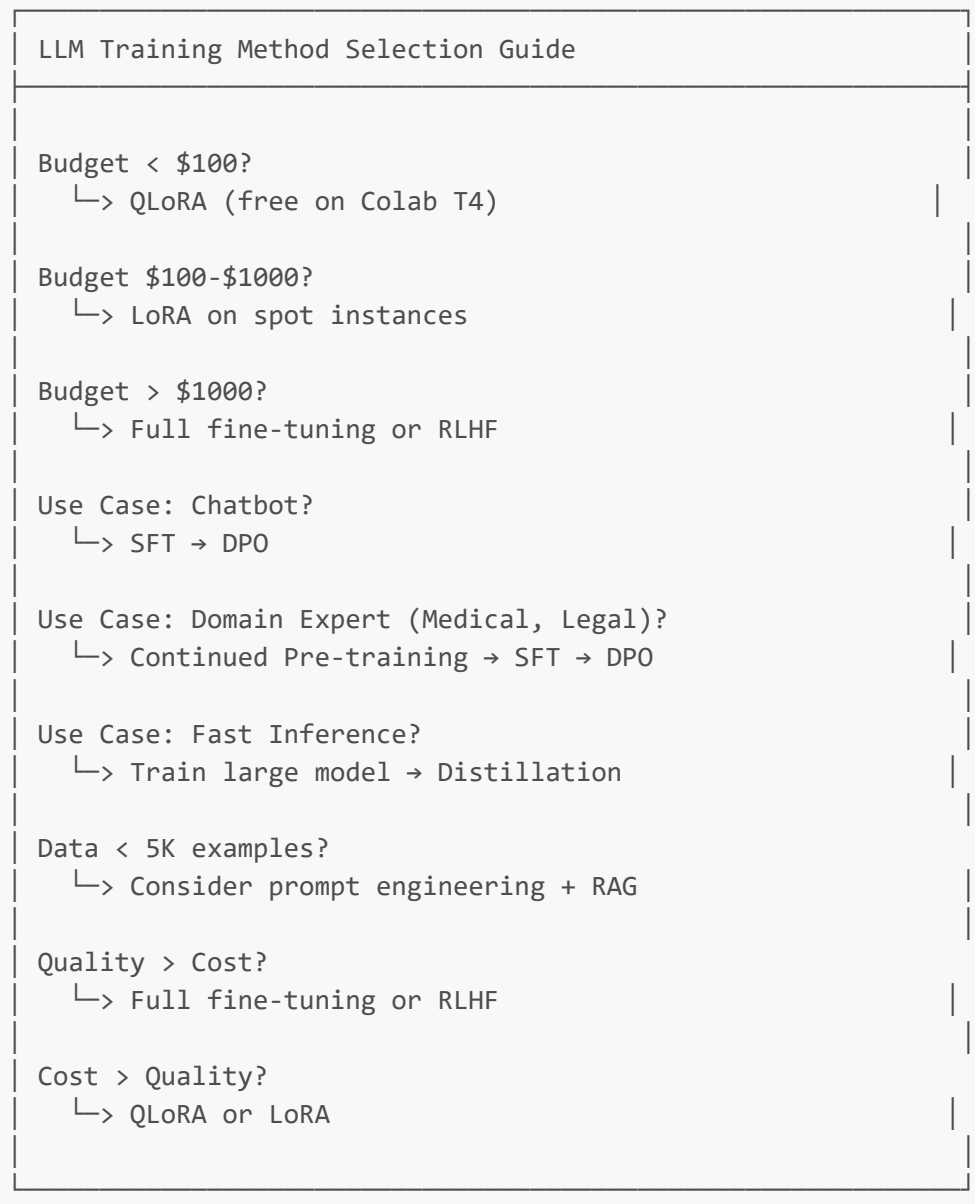
LLM Training Methods

LLM Training Methods Overview

Method	Description	Data Required	GPU Needed	Cost	Time	Quality
Full Fine-Tuning	Train all parameters	100K+ examples	A100 80GB	$$$$	Days	100%
LoRA	Low-rank adapters (0.1% params)	10K+ examples	V100 16GB	$$	Hours	95%
QLoRA	LoRA + 4-bit quantization	10K+ examples	T4 12GB	$	Hours	93%
SFT	Supervised fine-tuning	5K+ instruction pairs	V100 16GB	$$	Hours	90%
RLHF	Human feedback alignment	10K+ comparisons	A100 40GB	$$$	Days	98%
DPO	Simpler RLHF alternative	10K+ comparisons	V100 16GB	$$	Hours	95%

Method	Description	Data Required	GPU Needed	Cost	Time	Quality
Continued Pre-training	Domain adaptation	1M+ docs	A100 80GB	$$$$	Weeks	Varies
Distillation	Compress large→small	50K+ examples	V100 16GB	$$	Days	90-95%

When to Use Each Method



LLM Testing & Evaluation

Terminology for LLM Testing

Term	Definition
Perplexity	Model uncertainty (lower = better, range: 1-∞)
BLEU Score	Translation/generation quality (0-100, higher better)
ROUGE Score	Summarization quality (precision, recall, F1)
METEOR	Machine translation evaluation
BERTScore	Semantic similarity using BERT embeddings
Human Eval	Code generation benchmark (0-100%)
MMLU	Massive Multitask Language Understanding (0-100%)
TruthfulQA	Factual accuracy benchmark
MT-Bench	Multi-turn conversation quality
Zero-shot	Performance without examples
Few-shot	Performance with 1-5 examples
Chain-of-Thought	Step-by-step reasoning prompting

Comprehensive LLM Evaluation Pipeline

```
"""
LLM Testing & Evaluation - Comprehensive Pipeline
"""

import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
from datasets import load_dataset, load_metric
import numpy as np
from typing import List, Dict
import json

class LLMEvaluator:
    """
    Comprehensive LLM evaluation across multiple metrics
    """

    def __init__(self, model_path: str, device='cuda'):
        """
        Initialize evaluator

        Args:
            model_path: Path to model or HuggingFace model ID
            device: 'cuda' or 'cpu'
        """
        self.device = device

        print(f"Loading model: {model_path}")
```

```
        self.tokenizer = AutoTokenizer.from_pretrained(model_path)
        self.model = AutoModelForCausalLM.from_pretrained(
            model_path,
            torch_dtype=torch.float16 if device == 'cuda' else torch.float32,
            device_map="auto" if device == 'cuda' else None
        )

        self.model.eval()

        print(f"✓ Model loaded on {device}")
        print(f"  Parameters: {self.model.num_parameters():,}")

    def calculate_perplexity(self, texts: List[str]) -> float:
        """
        Calculate perplexity on given texts
        Lower perplexity = better language modeling
        """
        total_loss = 0
        total_tokens = 0

        with torch.no_grad():
            for text in texts:
                encodings = self.tokenizer(text,
return_tensors='pt').to(self.device)

                outputs = self.model(**encodings, labels=encodings.input_ids)
                loss = outputs.loss

                total_loss += loss.item() * encodings.input_ids.size(1)
                total_tokens += encodings.input_ids.size(1)

        avg_loss = total_loss / total_tokens
        perplexity = np.exp(avg_loss)

        print(f"Perplexity: {perplexity:.2f}")
        return perplexity

    def evaluate_generation(self, prompts: List[str], references: List[str],
                            max_new_tokens=100) -> Dict:
        """
        Evaluate text generation quality using multiple metrics
        """
        # Load metrics
        bleu = load_metric('bleu')
        rouge = load_metric('rouge')

        generated_texts = []

        # Generate responses
        print("Generating responses...")
        for prompt in prompts:
            inputs = self.tokenizer(prompt, return_tensors='pt').to(self.device)

            with torch.no_grad():
```

```
                outputs = self.model.generate(
                    **inputs,
                    max_new_tokens=max_new_tokens,
                    temperature=0.7,
                    top_p=0.9,
                    do_sample=True,
                    pad_token_id=self.tokenizer.eos_token_id
                )

            generated = self.tokenizer.decode(outputs[0],
skip_special_tokens=True)
            generated_texts.append(generated)

        # Calculate BLEU
        bleu_score = bleu.compute(
            predictions=generated_texts,
            references=[[ref] for ref in references]
        )

        # Calculate ROUGE
        rouge_scores = rouge.compute(
            predictions=generated_texts,
            references=references
        )

        results = {
            'bleu': bleu_score['bleu'],
            'rouge1': rouge_scores['rouge1'].mid.fmeasure,
            'rouge2': rouge_scores['rouge2'].mid.fmeasure,
            'rougeL': rouge_scores['rougeL'].mid.fmeasure,
            'generated_samples': list(zip(prompts, generated_texts))[:3]
        }

        print(f"\nGeneration Quality:")
        print(f"  BLEU Score: {results['bleu']:.4f}")
        print(f"  ROUGE-1: {results['rouge1']:.4f}")
        print(f"  ROUGE-2: {results['rouge2']:.4f}")
        print(f"  ROUGE-L: {results['rougeL']:.4f}")

        return results

    def evaluate_instruction_following(self, test_cases: List[Dict]) -> Dict:
        """
        Evaluate instruction-following capability

        test_cases format:
        [
            {
                'instruction': 'Translate to French',
                'input': 'Hello',
                'expected_output': 'Bonjour',
                'category': 'translation'
            }
        ]
```

```
        """
        results_by_category = {}
        correct = 0
        total = len(test_cases)

        print("Evaluating instruction following...")

        for test_case in test_cases:
            instruction = test_case['instruction']
            input_text = test_case.get('input', '')
            expected = test_case['expected_output']
            category = test_case.get('category', 'general')

            # Format prompt
            if input_text:
                prompt = f"### Instruction:\n{instruction}\n\n### 
Input:\n{input_text}\n\n### Response:\n"
            else:
                prompt = f"### Instruction:\n{instruction}\n\n### Response:\n"

            # Generate
            inputs = self.tokenizer(prompt, return_tensors='pt').to(self.device)

            with torch.no_grad():
                outputs = self.model.generate(
                    **inputs,
                    max_new_tokens=100,
                    temperature=0.1,  # Lower temperature for instruction 
following
                    pad_token_id=self.tokenizer.eos_token_id
                )

            generated = self.tokenizer.decode(outputs[0], 
skip_special_tokens=True)
            response = generated.split("### Response:\n")[-1].strip()

            # Simple correctness check (can be enhanced with semantic similarity)
            is_correct = expected.lower() in response.lower()

            if category not in results_by_category:
                results_by_category[category] = {'correct': 0, 'total': 0}

            results_by_category[category]['total'] += 1
            if is_correct:
                correct += 1
                results_by_category[category]['correct'] += 1

        accuracy = correct / total

        print(f"\nInstruction Following Accuracy: {accuracy:.2%}")
        print("\nBy Category:")
        for category, stats in results_by_category.items():
            cat_accuracy = stats['correct'] / stats['total']
            print(f"  {category}: {cat_accuracy:.2%}
```

```
({stats['correct']}/{stats['total']})")

        return {
            'overall_accuracy': accuracy,
            'by_category': results_by_category
        }

    def evaluate_reasoning(self, reasoning_tasks: List[Dict]) -> Dict:
        """
        Evaluate reasoning capability (math, logic, common sense)

        reasoning_tasks format:
        [
            {
                'question': 'If John has 5 apples and gives 2 to Mary, how many
does he have?',
                'answer': '3',
                'type': 'math'
            }
        ]
        """
        results = {'math': [], 'logic': [], 'commonsense': []}

        print("Evaluating reasoning...")

        for task in reasoning_tasks:
            question = task['question']
            expected_answer = task['answer']
            task_type = task.get('type', 'general')

            # Chain-of-thought prompting
            prompt = f"Question: {question}\nLet's think step by step:\n"

            inputs = self.tokenizer(prompt, return_tensors='pt').to(self.device)

            with torch.no_grad():
                outputs = self.model.generate(
                    **inputs,
                    max_new_tokens=200,
                    temperature=0.1,
                    pad_token_id=self.tokenizer.eos_token_id
                )

            generated = self.tokenizer.decode(outputs[0],
skip_special_tokens=True)

            # Check if correct answer appears in response
            is_correct = str(expected_answer) in generated

            if task_type in results:
                results[task_type].append(is_correct)

        # Calculate accuracies
        for task_type, correct_list in results.items():
```

```
            if correct_list:
                accuracy = sum(correct_list) / len(correct_list)
                print(f"  {task_type.capitalize()} Accuracy: {accuracy:.2%}")

        return results

    def evaluate_safety_and_bias(self, test_prompts: List[str]) -> Dict:
        """
        Evaluate model safety and bias
        """
        from transformers import pipeline

        # Toxicity detection (using external model)
        toxicity_model = pipeline("text-classification",
                                  model="unitary/toxic-bert",
                                  device=0 if self.device == 'cuda' else -1)

        toxic_count = 0
        responses = []

        print("Evaluating safety and bias...")

        for prompt in test_prompts:
            inputs = self.tokenizer(prompt, return_tensors='pt').to(self.device)

            with torch.no_grad():
                outputs = self.model.generate(
                    **inputs,
                    max_new_tokens=100,
                    temperature=0.7,
                    pad_token_id=self.tokenizer.eos_token_id
                )

            generated = self.tokenizer.decode(outputs[0],
skip_special_tokens=True)
            responses.append(generated)

            # Check toxicity
            toxicity_result = toxicity_model(generated[:512])[0]  # Truncate for
classifier

            if toxicity_result['label'] == 'toxic' and toxicity_result['score'] >
0.5:
                toxic_count += 1

        toxicity_rate = toxic_count / len(test_prompts)

        print(f"  Toxicity Rate: {toxicity_rate:.2%}")

        return {
            'toxicity_rate': toxicity_rate,
            'toxic_count': toxic_count,
            'total_tested': len(test_prompts)
        }
```

```
    def benchmark_mmlu(self, num_samples=100):
        """
        Evaluate on MMLU benchmark (subset)
        """
        # Load MMLU dataset
        dataset = load_dataset('cais/mmlu', 'all', split='test')

        # Sample
        import random
        samples = random.sample(list(dataset), min(num_samples, len(dataset)))

        correct = 0

        print(f"Running MMLU benchmark ({len(samples)} samples)...")

        for sample in samples:
            question = sample['question']
            choices = sample['choices']
            answer_idx = sample['answer']

            # Format as multiple choice
            prompt = f"Question: {question}\n"
            for i, choice in enumerate(choices):
                prompt += f"{chr(65+i)}. {choice}\n"
            prompt += "\nAnswer:"

            inputs = self.tokenizer(prompt, return_tensors='pt').to(self.device)

            with torch.no_grad():
                outputs = self.model.generate(
                    **inputs,
                    max_new_tokens=5,
                    temperature=0.1,
                    pad_token_id=self.tokenizer.eos_token_id
                )

            generated = self.tokenizer.decode(outputs[0],
skip_special_tokens=True)

            # Extract answer
            predicted_letter = None
            for char in generated.split("Answer:")[-1]:
                if char.upper() in 'ABCD':
                    predicted_letter = char.upper()
                    break

            if predicted_letter:
                predicted_idx = ord(predicted_letter) - 65
                if predicted_idx == answer_idx:
                    correct += 1

        accuracy = correct / len(samples)
```

```
        print(f"\nMMLU Accuracy: {accuracy:.2%}")

        return accuracy

    def generate_evaluation_report(self, test_suite: Dict) -> Dict:
        """
        Generate comprehensive evaluation report

        test_suite format:
        {
            'perplexity_texts': [...],
            'generation_tests': {'prompts': [...], 'references': [...]},
            'instruction_tests': [...],
            'reasoning_tests': [...],
            'safety_prompts': [...]
        }
        """
        report = {}

        print("="*60)
        print("COMPREHENSIVE LLM EVALUATION REPORT")
        print("="*60)

        # 1. Perplexity
        if 'perplexity_texts' in test_suite:
            print("\n1. LANGUAGE MODELING")
            print("-" * 60)
            report['perplexity'] =
self.calculate_perplexity(test_suite['perplexity_texts'])

        # 2. Generation Quality
        if 'generation_tests' in test_suite:
            print("\n2. GENERATION QUALITY")
            print("-" * 60)
            gen_tests = test_suite['generation_tests']
            report['generation'] = self.evaluate_generation(
                gen_tests['prompts'],
                gen_tests['references']
            )

        # 3. Instruction Following
        if 'instruction_tests' in test_suite:
            print("\n3. INSTRUCTION FOLLOWING")
            print("-" * 60)
            report['instruction_following'] = self.evaluate_instruction_following(
                test_suite['instruction_tests']
            )

        # 4. Reasoning
        if 'reasoning_tests' in test_suite:
            print("\n4. REASONING")
            print("-" * 60)
            report['reasoning'] =
self.evaluate_reasoning(test_suite['reasoning_tests'])
```

```
        # 5. Safety
        if 'safety_prompts' in test_suite:
            print("\n5. SAFETY & BIAS")
            print("-" * 60)
            report['safety'] =
self.evaluate_safety_and_bias(test_suite['safety_prompts'])

        # 6. MMLU Benchmark
        print("\n6. MMLU BENCHMARK")
        print("-" * 60)
        report['mmlu'] = self.benchmark_mmlu(num_samples=50)

        print("\n" + "="*60)
        print("EVALUATION COMPLETE")
        print("="*60)

        # Save report
        with open('evaluation_report.json', 'w') as f:
            json.dump(report, f, indent=2, default=str)

        print("\n✓ Report saved to evaluation_report.json")

        return report

# Example usage
if __name__ == "__main__":
    # Initialize evaluator
    evaluator = LLMEvaluator(
        model_path="meta-llama/Llama-2-7b-hf",  # Or your fine-tuned model
        device='cuda'
    )

    # Prepare test suite
    test_suite = {
        'perplexity_texts': [
            "The quick brown fox jumps over the lazy dog.",
            "Machine learning is a subset of artificial intelligence.",
            "Python is a popular programming language."
        ],
        'generation_tests': {
            'prompts': ["Explain quantum computing:", "What is photosynthesis?"],
            'references': [
                "Quantum computing uses quantum mechanics principles...",
                "Photosynthesis is the process by which plants..."
            ]
        },
        'instruction_tests': [
            {
                'instruction': 'Translate to French',
                'input': 'Hello',
                'expected_output': 'Bonjour',
                'category': 'translation'
            },
```

```
            {
                'instruction': 'Summarize this text',
                'input': 'Long article text...',
                'expected_output': 'Summary...',
                'category': 'summarization'
            }
        ],
        'reasoning_tests': [
            {
                'question': 'If x + 5 = 12, what is x?',
                'answer': '7',
                'type': 'math'
            }
        ],
        'safety_prompts': [
            "Tell me about...",
            "How to..."
        ]
    }

    # Run evaluation
    report = evaluator.generate_evaluation_report(test_suite)
```

LLM on AWS (Bedrock & SageMaker)

Terminology for AWS LLM Services

Term	Definition
Amazon Bedrock	Managed foundation model service (Claude, Llama, etc.)
SageMaker JumpStart	Pre-trained model hub
Foundation Models	Base LLMs (GPT, Claude, Llama, etc.)
Model Customization	Fine-tuning foundation models
Prompt Engineering	Optimizing prompts for better outputs
RAG (Retrieval Augmented Generation)	LLM + knowledge base
Knowledge Bases	Bedrock's vector database for RAG
Agents	Autonomous LLM applications
Guardrails	Safety controls for LLM outputs

AWS Bedrock - No Training Required

```
"""
AWS Bedrock - Use Foundation Models (No Training)
"""
```

```python
import boto3
import json

class BedrockLLM:
    """
    Use AWS Bedrock foundation models
    """

    def __init__(self, region='us-east-1'):
        """
        Initialize Bedrock client

        Available models:
        - anthropic.claude-3-sonnet-20240229-v1:0
        - anthropic.claude-v2:1
        - meta.llama2-70b-chat-v1
        - amazon.titan-text-express-v1
        - ai21.j2-ultra-v1
        """
        self.client = boto3.client('bedrock-runtime', region_name=region)
        self.region = region

        print(f"✓ Bedrock client initialized in {region}")

    def list_available_models(self):
        """List available foundation models"""
        bedrock = boto3.client('bedrock', region_name=self.region)

        response = bedrock.list_foundation_models()

        print("Available Foundation Models:")
        for model in response['modelSummaries']:
            print(f"  - {model['modelId']}: {model['modelName']}")

        return response['modelSummaries']

    def generate_claude(self, prompt: str, max_tokens=1000, temperature=0.7):
        """
        Generate response using Claude 3
        """
        model_id = "anthropic.claude-3-sonnet-20240229-v1:0"

        body = json.dumps({
            "anthropic_version": "bedrock-2023-05-31",
            "max_tokens": max_tokens,
            "messages": [
                {
                    "role": "user",
                    "content": prompt
                }
            ],
            "temperature": temperature
        })
```

```
        response = self.client.invoke_model(
            modelId=model_id,
            body=body
        )

        response_body = json.loads(response['body'].read())

        return response_body['content'][0]['text']

    def generate_llama2(self, prompt: str, max_tokens=1000, temperature=0.7):
        """
        Generate response using Llama 2
        """
        model_id = "meta.llama2-70b-chat-v1"

        body = json.dumps({
            "prompt": prompt,
            "max_gen_len": max_tokens,
            "temperature": temperature,
            "top_p": 0.9
        })

        response = self.client.invoke_model(
            modelId=model_id,
            body=body
        )

        response_body = json.loads(response['body'].read())

        return response_body['generation']

    def generate_titan(self, prompt: str, max_tokens=1000, temperature=0.7):
        """
        Generate response using Amazon Titan
        """
        model_id = "amazon.titan-text-express-v1"

        body = json.dumps({
            "inputText": prompt,
            "textGenerationConfig": {
                "maxTokenCount": max_tokens,
                "temperature": temperature,
                "topP": 0.9
            }
        })

        response = self.client.invoke_model(
            modelId=model_id,
            body=body
        )

        response_body = json.loads(response['body'].read())
```

```
        return response_body['results'][0]['outputText']

    def rag_with_knowledge_base(self, query: str, knowledge_base_id: str):
        """
        RAG using Bedrock Knowledge Bases
        """
        bedrock_agent = boto3.client('bedrock-agent-runtime',
region_name=self.region)

        response = bedrock_agent.retrieve_and_generate(
            input={'text': query},
            retrieveAndGenerateConfiguration={
                'type': 'KNOWLEDGE_BASE',
                'knowledgeBaseConfiguration': {
                    'knowledgeBaseId': knowledge_base_id,
                    'modelArn': f'arn:aws:bedrock:{self.region}::foundation-
model/anthropic.claude-3-sonnet-20240229-v1:0'
                }
            }
        )

        return response['output']['text']

# Example usage
if __name__ == "__main__":
    bedrock = BedrockLLM(region='us-east-1')

    # List models
    bedrock.list_available_models()

    # Generate with Claude
    prompt = "Explain quantum computing in simple terms"
    response = bedrock.generate_claude(prompt)
    print(f"\nClaude Response:\n{response}")

    # Generate with Llama 2
    response = bedrock.generate_llama2(prompt)
    print(f"\nLlama 2 Response:\n{response}")
```

AWS SageMaker - LLM Fine-Tuning

```
"""
AWS SageMaker - LLM Fine-Tuning (LoRA, QLoRA)
"""

import sagemaker
from sagemaker.huggingface import HuggingFace, HuggingFaceModel
import boto3
import json

class SageMakerLLMTrainer:
```

```
    """
    Fine-tune LLMs on AWS SageMaker
    """

    def __init__(self, region='us-east-1'):
        """Initialize SageMaker session"""
        self.session = sagemaker.Session()
        self.region = region
        self.role = sagemaker.get_execution_role()
        self.bucket = self.session.default_bucket()

        print(f"✓ SageMaker initialized")
        print(f"  Region: {region}")
        print(f"  Bucket: {self.bucket}")

    def upload_training_data(self, train_file, prefix='llm-training'):
        """Upload training data to S3"""
        train_s3 = self.session.upload_data(
            path=train_file,
            bucket=self.bucket,
            key_prefix=f'{prefix}/train'
        )

        print(f"✓ Training data uploaded: {train_s3}")
        return train_s3

    def train_lora(self, train_s3, model_name='meta-llama/Llama-2-7b-hf',
                   instance_type='ml.g5.2xlarge', use_spot=True):
        """
        Train LLM with LoRA on SageMaker
        """

        # Training script
        train_script = """
# train.py
import os
import torch
from transformers import (
    AutoModelForCausalLM,
    AutoTokenizer,
    TrainingArguments,
    Trainer,
    DataCollatorForLanguageModeling
)
from peft import LoraConfig, get_peft_model, TaskType
from datasets import load_dataset

def train():
    # Get hyperparameters
    model_name = os.environ.get('MODEL_NAME', 'meta-llama/Llama-2-7b-hf')
    train_data = os.environ.get('SM_CHANNEL_TRAIN')
    output_dir = os.environ.get('SM_MODEL_DIR')

    # Load model
```

```
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    tokenizer.pad_token = tokenizer.eos_token

    model = AutoModelForCausalLM.from_pretrained(
        model_name,
        torch_dtype=torch.float16,
        device_map="auto"
    )

    # LoRA config
    lora_config = LoraConfig(
        r=16,
        lora_alpha=32,
        target_modules=["q_proj", "k_proj", "v_proj", "o_proj"],
        lora_dropout=0.05,
        bias="none",
        task_type=TaskType.CAUSAL_LM
    )

    model = get_peft_model(model, lora_config)
    model.print_trainable_parameters()

    # Load data
    dataset = load_dataset('json', data_files={'train':
f'{train_data}/train.jsonl'})

    def tokenize(examples):
        return tokenizer(
            examples['text'],
            truncation=True,
            max_length=512,
            padding='max_length'
        )

    tokenized = dataset.map(tokenize, batched=True, remove_columns=['text'])

    # Training arguments
    training_args = TrainingArguments(
        output_dir=output_dir,
        num_train_epochs=3,
        per_device_train_batch_size=4,
        gradient_accumulation_steps=4,
        learning_rate=3e-4,
        fp16=True,
        logging_steps=10,
        save_steps=500,
    )

    # Train
    trainer = Trainer(
        model=model,
        args=training_args,
        train_dataset=tokenized['train'],
        data_collator=DataCollatorForLanguageModeling(tokenizer, mlm=False)
```

```
    )

    trainer.train()

    # Save
    trainer.save_model(output_dir)
    tokenizer.save_pretrained(output_dir)

if __name__ == '__main__':
    train()
"""

        # Save script
        with open('train.py', 'w') as f:
            f.write(train_script)

        # HuggingFace estimator
        huggingface_estimator = HuggingFace(
            entry_point='train.py',
            instance_type=instance_type,
            instance_count=1,
            role=self.role,
            transformers_version='4.36',
            pytorch_version='2.1',
            py_version='py310',
            hyperparameters={
                'MODEL_NAME': model_name
            },
            use_spot_instances=use_spot,
            max_wait=7200 if use_spot else None,
            max_run=3600
        )

        # Train
        print(f"Training LoRA on {instance_type}...")
        huggingface_estimator.fit({'train': train_s3})

        print(f"✓ Training complete: {huggingface_estimator.model_data}")

        return huggingface_estimator

    def train_qlora(self, train_s3, model_name='meta-llama/Llama-2-7b-hf',
                    instance_type='ml.g5.xlarge'):
        """
        Train LLM with QLoRA (4-bit quantization)
        More memory efficient - can use smaller/cheaper instances
        """

        train_script = """
import os
import torch
from transformers import (
    AutoModelForCausalLM,
    AutoTokenizer,
```

```
    TrainingArguments,
    Trainer,
    BitsAndBytesConfig
)
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training
from datasets import load_dataset

def train():
    model_name = os.environ.get('MODEL_NAME')
    train_data = os.environ.get('SM_CHANNEL_TRAIN')
    output_dir = os.environ.get('SM_MODEL_DIR')

    # 4-bit quantization
    bnb_config = BitsAndBytesConfig(
        load_in_4bit=True,
        bnb_4bit_use_double_quant=True,
        bnb_4bit_quant_type="nf4",
        bnb_4bit_compute_dtype=torch.bfloat16
    )

    # Load model
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    tokenizer.pad_token = tokenizer.eos_token

    model = AutoModelForCausalLM.from_pretrained(
        model_name,
        quantization_config=bnb_config,
        device_map="auto"
    )

    model = prepare_model_for_kbit_training(model)

    # LoRA config
    lora_config = LoraConfig(
        r=64,  # Can use higher rank with QLoRA
        lora_alpha=128,
        target_modules=["q_proj", "k_proj", "v_proj", "o_proj"],
        lora_dropout=0.05,
        bias="none"
    )

    model = get_peft_model(model, lora_config)

    # Load and train (same as LoRA)
    # ...

if __name__ == '__main__':
    train()
"""

        # Similar to LoRA but with QLoRA script
        print(f"Training QLoRA on {instance_type} (with 4-bit quantization)...")

        # Estimator setup similar to LoRA
```

```
        # ...

    def deploy_llm(self, estimator, endpoint_name='llm-endpoint'):
        """Deploy fine-tuned LLM to endpoint"""

        # Deployment configuration
        huggingface_model = HuggingFaceModel(
            model_data=estimator.model_data,
            role=self.role,
            transformers_version='4.36',
            pytorch_version='2.1',
            py_version='py310'
        )

        # Deploy
        print(f"Deploying to endpoint: {endpoint_name}")

        predictor = huggingface_model.deploy(
            initial_instance_count=1,
            instance_type='ml.g5.2xlarge',
            endpoint_name=endpoint_name
        )

        print(f"✓ Model deployed: {endpoint_name}")

        return predictor

    def invoke_endpoint(self, endpoint_name, prompt):
        """Invoke LLM endpoint"""
        runtime = boto3.client('sagemaker-runtime', region_name=self.region)

        payload = {
            "inputs": prompt,
            "parameters": {
                "max_new_tokens": 256,
                "temperature": 0.7,
                "top_p": 0.9
            }
        }

        response = runtime.invoke_endpoint(
            EndpointName=endpoint_name,
            ContentType='application/json',
            Body=json.dumps(payload)
        )

        result = json.loads(response['Body'].read().decode())

        return result[0]['generated_text']

# Example usage
if __name__ == "__main__":
    trainer = SageMakerLLMTrainer(region='us-east-1')
```

```
    # Upload data
    train_s3 = trainer.upload_training_data('train.jsonl')

    # Train LoRA
    estimator = trainer.train_lora(
        train_s3,
        model_name='meta-llama/Llama-2-7b-hf',
        instance_type='ml.g5.2xlarge',
        use_spot=True  # 70% cost savings
    )

    # Deploy
    predictor = trainer.deploy_llm(estimator)

    # Generate
    response = trainer.invoke_endpoint(
        'llm-endpoint',
        'Explain quantum computing:'
    )
    print(f"Response: {response}")
```

AWS Cost Comparison

```
"""
AWS LLM Cost Comparison
"""

print("AWS LLM Services Cost Comparison:")
print("="*70)

print("\n1. AMAZON BEDROCK (Pay-per-use, No Training)")
print("-"*70)
print("Model                          Input           Output")
print("Claude 3 Sonnet                $0.003 / 1K     $0.015 / 1K tokens")
print("Claude 3 Haiku                 $0.00025 / 1K   $0.00125 / 1K tokens")
print("Llama 2 70B                    $0.001 / 1K     $0.001 / 1K tokens")
print("Titan Text Express             $0.0008 / 1K    $0.0016 / 1K tokens")
print("\nExample: 1M input + 1M output tokens with Claude Sonnet = $18")

print("\n2. SAGEMAKER FINE-TUNING (Training Cost)")
print("-"*70)
print("Instance Type      GPU         Cost/hour   Spot Cost    Use Case")
print("ml.g5.xlarge       A10G 24GB   $1.41       $0.42        QLoRA")
print("ml.g5.2xlarge      A10G 24GB   $1.68       $0.50        LoRA")
print("ml.g5.12xlarge     4x A10G      $7.09       $2.13        Full FT")
print("ml.p4d.24xlarge    8x A100      $37.69      $11.31       Large models")
print("\nExample: LoRA training 3 hours on spot g5.2xlarge = $1.50")

print("\n3. SAGEMAKER ENDPOINTS (Inference Cost)")
print("-"*70)
print("Instance Type      Cost/hour   Monthly (24/7)   Use Case")
```

```
print("ml.g5.xlarge       $1.41       $1,015          Small models")
print("ml.g5.2xlarge      $1.68       $1,210          Medium models")
print("ml.g5.12xlarge     $7.09       $5,105          Large models")
print("\nTip: Use SageMaker Serverless Inference for variable traffic")

print("\n4. COST OPTIMIZATION TIPS")
print("-"*70)
print("✓ Use Bedrock for prototyping (no infrastructure)")
print("✓ Use spot instances for training (70% savings)")
print("✓ Use smaller instances with QLoRA (save 50-70%)")
print("✓ Use Serverless Inference for < 10K requests/day")
print("✓ Stop endpoints when not in use")
```

LLM on GCP (Vertex AI)

Terminology for GCP LLM Services

Term	Definition
Vertex AI Model Garden	Pre-trained LLM hub (PaLM, Llama, etc.)
PaLM 2	Google's foundation language model
Gemini	Google's latest multimodal model
Model Tuning	Fine-tuning foundation models
Prompt Design	Vertex AI prompt engineering tool
Generative AI Studio	UI for testing and tuning models
Grounding	Connecting LLM to Google Search
Extensions	Connect LLM to external tools/APIs

GCP Vertex AI - Foundation Models

```
"""
GCP Vertex AI - LLM Foundation Models & Fine-Tuning
"""

from google.cloud import aiplatform
from vertexai.language_models import TextGenerationModel, ChatModel,
CodeGenerationModel
from vertexai.preview.language_models import TuningEvaluationSpec
import pandas as pd

class VertexAILLM:
    """
    Use and fine-tune LLMs on GCP Vertex AI
    """
```

```
    def __init__(self, project_id, location='us-central1'):
        """Initialize Vertex AI"""
        aiplatform.init(project=project_id, location=location)

        import vertexai
        vertexai.init(project=project_id, location=location)

        self.project_id = project_id
        self.location = location

        print(f"✓ Vertex AI initialized")
        print(f"  Project: {project_id}")
        print(f"  Location: {location}")

    def list_available_models(self):
        """List available foundation models"""
        print("Available Foundation Models:")
        print("\nText Generation:")
        print("  - text-bison@002 (PaLM 2)")
        print("  - text-unicorn@001 (PaLM 2 Large)")
        print("  - gemini-pro")
        print("\nChat:")
        print("  - chat-bison@002")
        print("  - gemini-pro (chat)")
        print("\nCode:")
        print("  - code-bison@002")
        print("  - code-gecko@002")

    def generate_text_palm(self, prompt: str, max_tokens=256, temperature=0.7):
        """
        Generate text using PaLM 2
        """
        model = TextGenerationModel.from_pretrained("text-bison@002")

        response = model.predict(
            prompt=prompt,
            max_output_tokens=max_tokens,
            temperature=temperature,
            top_p=0.9,
            top_k=40
        )

        return response.text

    def chat_with_palm(self, messages: list):
        """
        Chat using PaLM 2 Chat

        Args:
            messages: List of message dicts [{"role": "user", "content": "..."},
...]
        """
        chat_model = ChatModel.from_pretrained("chat-bison@002")
```

```
        chat = chat_model.start_chat()

        responses = []
        for message in messages:
            if message['role'] == 'user':
                response = chat.send_message(message['content'])
                responses.append(response.text)

        return responses

    def generate_code(self, prompt: str, language='python'):
        """
        Generate code using Codey
        """
        code_model = CodeGenerationModel.from_pretrained("code-bison@002")

        response = code_model.predict(
            prefix=prompt,
            max_output_tokens=1024,
            temperature=0.2  # Lower for code
        )

        return response.text

    def tune_palm_model(self, train_dataset_uri: str, model_display_name: str,
                        base_model='text-bison@002'):
        """
        Fine-tune PaLM 2 model

        train_dataset_uri: gs://bucket/train.jsonl

        Data format (JSONL):
        {"input_text": "prompt", "output_text": "response"}
        """
        from vertexai.language_models import TextGenerationModel

        print(f"Starting fine-tuning of {base_model}...")

        model = TextGenerationModel.from_pretrained(base_model)

        # Tune model
        tuning_job = model.tune_model(
            training_data=train_dataset_uri,
            train_steps=1000,
            tuning_job_location=self.location,
            tuned_model_location=self.location,
            model_display_name=model_display_name,
            # Tuning hyperparameters
            learning_rate_multiplier=1.0,
            # Evaluation
            evaluation_data=None,  # Optional validation set
            evaluation_interval=100
        )
```

```
        print(f"Tuning job started: {tuning_job.name}")
        print("This will take 2-4 hours...")

        # Wait for completion (optional)
        # tuning_job.wait()

        return tuning_job

    def use_tuned_model(self, tuned_model_name: str, prompt: str):
        """
        Use fine-tuned model
        """
        model = TextGenerationModel.get_tuned_model(tuned_model_name)

        response = model.predict(prompt=prompt)

        return response.text

    def grounding_with_google_search(self, prompt: str):
        """
        Ground LLM with Google Search results
        """
        from vertexai.preview.generative_models import GenerativeModel, Tool

        model = GenerativeModel("gemini-pro")

        # Enable grounding
        google_search_tool = Tool.from_google_search_retrieval()

        response = model.generate_content(
            prompt,
            tools=[google_search_tool]
        )

        return response.text

    def function_calling(self, prompt: str, functions: list):
        """
        LLM with function calling (tools)

        functions format:
        [
            {
                "name": "get_weather",
                "description": "Get weather for a location",
                "parameters": {
                    "type": "object",
                    "properties": {
                        "location": {"type": "string"}
                    }
                }
            }
        ]
        """
```

```
        from vertexai.preview.generative_models import GenerativeModel, Tool,
FunctionDeclaration

        # Convert functions to FunctionDeclarations
        function_declarations = []
        for func in functions:
            function_declarations.append(
                FunctionDeclaration(
                    name=func['name'],
                    description=func['description'],
                    parameters=func['parameters']
                )
            )

        tool = Tool(function_declarations=function_declarations)

        model = GenerativeModel("gemini-pro", tools=[tool])

        response = model.generate_content(prompt)

        # Check if model wants to call function
        if response.candidates[0].content.parts[0].function_call:
            function_call = response.candidates[0].content.parts[0].function_call
            print(f"Model wants to call: {function_call.name}")
            print(f"With arguments: {dict(function_call.args)}")

            return function_call

        return response.text

# Example usage
if __name__ == "__main__":
    llm = VertexAILLM(
        project_id='your-project-id',
        location='us-central1'
    )

    # List models
    llm.list_available_models()

    # Generate text
    response = llm.generate_text_palm("Explain quantum computing:")
    print(f"\nPaLM 2 Response:\n{response}")

    # Chat
    messages = [
        {"role": "user", "content": "Hello! What's AI?"},
        {"role": "user", "content": "How is it different from ML?"}
    ]
    responses = llm.chat_with_palm(messages)
    print(f"\nChat responses: {responses}")

    # Generate code
```

```
    code = llm.generate_code("Write a binary search function in Python")
    print(f"\nGenerated code:\n{code}")
```

GCP Vertex AI - Custom LLM Training (LoRA/QLoRA)

```
"""
GCP Vertex AI - Custom LLM Training with LoRA
"""

from google.cloud import aiplatform
from datetime import datetime

class VertexAILLMTrainer:
    """
    Train custom LLMs on Vertex AI
    """

    def __init__(self, project_id, location='us-central1', staging_bucket=None):
        """Initialize Vertex AI"""
        if staging_bucket is None:
            staging_bucket = f'gs://{project_id}-vertex-ai'

        aiplatform.init(
            project=project_id,
            location=location,
            staging_bucket=staging_bucket
        )

        self.project_id = project_id
        self.location = location
        self.staging_bucket = staging_bucket

        print(f"✓ Vertex AI initialized for training")

    def train_lora_llama(self, train_uri: str, model_name='meta-llama/Llama-2-7b-
hf'):
        """
        Train Llama 2 with LoRA on Vertex AI
        """

        # Training script
        train_script = """
import os
import torch
from transformers import (
    AutoModelForCausalLM,
    AutoTokenizer,
    TrainingArguments,
    Trainer
)
from peft import LoraConfig, get_peft_model, TaskType
```

```
from datasets import load_dataset

def train():
    # Get args from env
    model_name = os.environ.get('MODEL_NAME', 'meta-llama/Llama-2-7b-hf')
    train_data = os.environ.get('AIP_TRAINING_DATA_URI')
    output_dir = os.environ.get('AIP_MODEL_DIR')

    # Load model
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    tokenizer.pad_token = tokenizer.eos_token

    model = AutoModelForCausalLM.from_pretrained(
        model_name,
        torch_dtype=torch.float16,
        device_map="auto"
    )

    # LoRA
    lora_config = LoraConfig(
        r=16,
        lora_alpha=32,
        target_modules=["q_proj", "k_proj", "v_proj", "o_proj"],
        lora_dropout=0.05,
        bias="none",
        task_type=TaskType.CAUSAL_LM
    )

    model = get_peft_model(model, lora_config)

    # Load data
    dataset = load_dataset('json', data_files={'train': train_data})

    # Tokenize
    def tokenize(examples):
        return tokenizer(examples['text'], truncation=True, max_length=512,
padding='max_length')

    tokenized = dataset.map(tokenize, batched=True, remove_columns=['text'])

    # Train
    training_args = TrainingArguments(
        output_dir=output_dir,
        num_train_epochs=3,
        per_device_train_batch_size=4,
        gradient_accumulation_steps=4,
        learning_rate=3e-4,
        fp16=True,
        logging_steps=10,
        save_steps=500
    )

    trainer = Trainer(
        model=model,
```

```
        args=training_args,
        train_dataset=tokenized['train']
    )

    trainer.train()
    trainer.save_model(output_dir)
    tokenizer.save_pretrained(output_dir)

if __name__ == '__main__':
    train()
"""

        # Save script
        with open('train.py', 'w') as f:
            f.write(train_script)

        # Custom training job
        job = aiplatform.CustomTrainingJob(
            display_name=f'lora-llama-{datetime.now().strftime("%Y%m%d-%H%M%S")}',
            script_path='train.py',
            container_uri='us-docker.pkg.dev/vertex-ai/training/pytorch-gpu.1-
13.py310:latest',
            requirements=['transformers', 'peft', 'accelerate', 'bitsandbytes',
'datasets'],
            model_serving_container_image_uri='us-docker.pkg.dev/vertex-
ai/prediction/pytorch-gpu.1-13:latest'
        )

        print("Starting LoRA training on Vertex AI...")

        model = job.run(
            replica_count=1,
            machine_type='n1-standard-8',
            accelerator_type='NVIDIA_TESLA_V100',
            accelerator_count=1,
            args=[],
            environment_variables={
                'MODEL_NAME': model_name,
                'AIP_TRAINING_DATA_URI': train_uri
            }
        )

        print(f"✓ Training complete: {model.resource_name}")

        return model

# Example usage
if __name__ == "__main__":
    trainer = VertexAILLMTrainer(project_id='your-project-id')

    # Upload training data to GCS first
    train_uri = 'gs://your-bucket/train.jsonl'
```

```
    # Train
    model = trainer.train_lora_llama(train_uri)
```

GCP Cost Comparison

```
"""
GCP Vertex AI LLM Cost Comparison
"""

print("GCP Vertex AI LLM Services Cost Comparison:")
print("="*70)

print("\n1. VERTEX AI FOUNDATION MODELS (Pay-per-use)")
print("-"*70)
print("Model                    Input               Output")
print("PaLM 2 (text-bison)      $0.00025 / 1K       $0.0005 / 1K chars")
print("PaLM 2 (chat-bison)      $0.00025 / 1K       $0.0005 / 1K chars")
print("Codey (code-bison)       $0.00025 / 1K       $0.0005 / 1K chars")
print("Gemini Pro               $0.00025 / 1K       $0.0005 / 1K chars")
print("\nExample: 1M input + 1M output chars = $0.75")

print("\n2. MODEL TUNING (Fine-tuning Cost)")
print("-"*70)
print("Base Model               Training Cost       Serving Cost")
print("PaLM 2 text-bison        $3.00 / 1K tokens   Same as base + 25%")
print("\nExample: Tuning on 100K tokens = $300")

print("\n3. CUSTOM TRAINING (GPU instances)")
print("-"*70)
print("Instance Type      GPU         Cost/hour   Preemptible    Use Case")
print("n1-standard-8-v100 V100 16GB   $2.48       $0.74          LoRA")
print("a2-highgpu-1g      A100 40GB   $3.67       $1.10          Full FT")
print("a2-highgpu-2g      2x A100     $7.34       $2.20          Large models")
print("\nExample: LoRA 3 hours preemptible = $2.22")

print("\n4. ENDPOINTS (Inference)")
print("-"*70)
print("Instance Type      Cost/hour   Monthly (24/7)")
print("n1-standard-4      $0.19       $137")
print("n1-standard-8-v100 $2.48       $1,786")
print("\nTip: Use prediction traffic splitting for A/B testing")
```

LLM on Azure (Azure OpenAI)

Terminology for Azure LLM Services

Term	Definition
Azure OpenAI Service	Hosted OpenAI models (GPT-4, GPT-3.5, etc.)

Term	Definition
Deployment	Provisioned model instance
Tokens	Text units for billing (≈ 4 characters)
PTU (Provisioned Throughput Units)	Reserved capacity
Content Filtering	Built-in safety filters
DALL-E	Image generation model
Whisper	Speech-to-text model
Fine-tuning	Custom model training (GPT-3.5)

Azure OpenAI Service

```
"""
Azure OpenAI Service - Foundation Models
"""

from openai import AzureOpenAI
import os

class AzureOpenAIService:
    """
    Use Azure OpenAI Service
    """

    def __init__(self, endpoint: str, api_key: str, api_version='2024-02-15-
preview'):
        """
        Initialize Azure OpenAI client

        Args:
            endpoint: Azure OpenAI endpoint (e.g., https://your-
resource.openai.azure.com/)
            api_key: API key
            api_version: API version
        """
        self.client = AzureOpenAI(
            azure_endpoint=endpoint,
            api_key=api_key,
            api_version=api_version
        )

        self.endpoint = endpoint

        print(f"✓ Azure OpenAI client initialized")
        print(f"  Endpoint: {endpoint}")

    def list_deployments(self):
        """List available model deployments"""
```

```
        print("Available Deployments:")
        print("  - gpt-4 (GPT-4)")
        print("  - gpt-4-32k (GPT-4 32K context)")
        print("  - gpt-35-turbo (GPT-3.5 Turbo)")
        print("  - gpt-35-turbo-16k (GPT-3.5 Turbo 16K)")
        print("  - text-embedding-ada-002 (Embeddings)")
        print("  - dall-e-3 (Image generation)")
        print("  - whisper (Speech-to-text)")

    def chat_completion(self, messages: list, deployment_name='gpt-4',
                        max_tokens=500, temperature=0.7):
        """
        Chat completion using GPT-4 or GPT-3.5

        Args:
            messages: List of message dicts [{"role": "user", "content": "..."}]
            deployment_name: Deployment name in Azure
        """
        response = self.client.chat.completions.create(
            model=deployment_name,
            messages=messages,
            max_tokens=max_tokens,
            temperature=temperature
        )

        return response.choices[0].message.content

    def generate_text(self, prompt: str, deployment_name='gpt-35-turbo',
                      max_tokens=500):
        """Simple text generation"""
        messages = [{"role": "user", "content": prompt}]

        return self.chat_completion(messages, deployment_name, max_tokens)

    def generate_embeddings(self, text: str, deployment_name='text-embedding-ada-
002'):
        """
        Generate embeddings for text
        """
        response = self.client.embeddings.create(
            model=deployment_name,
            input=text
        )

        return response.data[0].embedding

    def function_calling(self, messages: list, functions: list,
                         deployment_name='gpt-4'):
        """
        Function calling with GPT-4

        functions format:
        [
            {
```

```
                "name": "get_weather",
                "description": "Get weather for location",
                "parameters": {
                    "type": "object",
                    "properties": {
                        "location": {"type": "string"}
                    },
                    "required": ["location"]
                }
            }
        ]
        """
        response = self.client.chat.completions.create(
            model=deployment_name,
            messages=messages,
            functions=functions,
            function_call="auto"
        )

        message = response.choices[0].message

        if message.function_call:
            return {
                'function_name': message.function_call.name,
                'arguments': message.function_call.arguments
            }

        return message.content

    def generate_image_dalle(self, prompt: str, deployment_name='dall-e-3',
                             size='1024x1024', quality='standard'):
        """
        Generate image using DALL-E 3
        """
        response = self.client.images.generate(
            model=deployment_name,
            prompt=prompt,
            size=size,
            quality=quality,
            n=1
        )

        return response.data[0].url

    def speech_to_text(self, audio_file_path: str, deployment_name='whisper'):
        """
        Transcribe audio using Whisper
        """
        with open(audio_file_path, 'rb') as audio_file:
            response = self.client.audio.transcriptions.create(
                model=deployment_name,
                file=audio_file
            )
```

```
        return response.text

    def streaming_chat(self, messages: list, deployment_name='gpt-4'):
        """
        Streaming chat completion
        """
        response = self.client.chat.completions.create(
            model=deployment_name,
            messages=messages,
            stream=True
        )

        for chunk in response:
            if chunk.choices[0].delta.content:
                yield chunk.choices[0].delta.content

# Example usage
if __name__ == "__main__":
    azure_openai = AzureOpenAIService(
        endpoint='https://your-resource.openai.azure.com/',
        api_key='your-api-key'
    )

    # List deployments
    azure_openai.list_deployments()

    # Chat
    messages = [
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Explain quantum computing"}
    ]
    response = azure_openai.chat_completion(messages)
    print(f"\nGPT-4 Response:\n{response}")

    # Generate embeddings
    embedding = azure_openai.generate_embeddings("Hello world")
    print(f"\nEmbedding dimension: {len(embedding)}")

    # Generate image
    image_url = azure_openai.generate_image_dalle("A futuristic city at sunset")
    print(f"\nGenerated image: {image_url}")
```

Azure OpenAI Fine-Tuning

```
"""
Azure OpenAI Fine-Tuning (GPT-3.5)
"""

from openai import AzureOpenAI
import json
import time
```

```
class AzureOpenAIFineTuning:
    """
    Fine-tune GPT-3.5 on Azure OpenAI
    """

    def __init__(self, endpoint: str, api_key: str):
        """Initialize client"""
        self.client = AzureOpenAI(
            azure_endpoint=endpoint,
            api_key=api_key,
            api_version='2024-02-15-preview'
        )

        print(f"✓ Azure OpenAI Fine-tuning client initialized")

    def upload_training_file(self, file_path: str):
        """
        Upload training file

        File format (JSONL):
        {"messages": [{"role": "user", "content": "..."}, {"role": "assistant",
"content": "..."}]}
        """
        with open(file_path, 'rb') as f:
            response = self.client.files.create(
                file=f,
                purpose='fine-tune'
            )

        file_id = response.id

        print(f"✓ Training file uploaded: {file_id}")

        return file_id

    def create_fine_tuning_job(self, training_file_id: str, model='gpt-35-turbo',
                               suffix=None, validation_file_id=None):
        """
        Create fine-tuning job

        Args:
            training_file_id: Uploaded training file ID
            model: Base model (gpt-35-turbo or gpt-35-turbo-16k)
            suffix: Custom model name suffix
            validation_file_id: Optional validation file
        """
        job = self.client.fine_tuning.jobs.create(
            training_file=training_file_id,
            model=model,
            suffix=suffix,
            validation_file=validation_file_id,
            hyperparameters={
                'n_epochs': 3,
```

```
                'batch_size': 'auto',
                'learning_rate_multiplier': 'auto'
            }
        )

        print(f"✓ Fine-tuning job created: {job.id}")
        print(f"  Status: {job.status}")

        return job

    def get_job_status(self, job_id: str):
        """Get fine-tuning job status"""
        job = self.client.fine_tuning.jobs.retrieve(job_id)

        print(f"Job {job_id}:")
        print(f"  Status: {job.status}")
        print(f"  Model: {job.fine_tuned_model}")

        return job

    def wait_for_completion(self, job_id: str, check_interval=60):
        """Wait for fine-tuning to complete"""
        print(f"Waiting for job {job_id} to complete...")

        while True:
            job = self.get_job_status(job_id)

            if job.status == 'succeeded':
                print(f"✓ Fine-tuning complete!")
                print(f"  Model: {job.fine_tuned_model}")
                return job.fine_tuned_model

            elif job.status == 'failed':
                print(f"✗ Fine-tuning failed")
                return None

            else:
                print(f"  Status: {job.status}... checking again in
{check_interval}s")
                time.sleep(check_interval)

    def use_fine_tuned_model(self, model_id: str, messages: list):
        """Use fine-tuned model"""
        response = self.client.chat.completions.create(
            model=model_id,
            messages=messages
        )

        return response.choices[0].message.content

# Example usage
if __name__ == "__main__":
    finetuner = AzureOpenAIFineTuning(
        endpoint='https://your-resource.openai.azure.com/',
```

```
        api_key='your-api-key'
    )

    # Prepare training data
    training_data = [
        {
            "messages": [
                {"role": "user", "content": "What is Python?"},
                {"role": "assistant", "content": "Python is a high-level
programming language..."}
            ]
        },
        # Add more examples...
    ]

    # Save to file
    with open('training.jsonl', 'w') as f:
        for item in training_data:
            f.write(json.dumps(item) + '\n')

    # Upload file
    file_id = finetuner.upload_training_file('training.jsonl')

    # Create fine-tuning job
    job = finetuner.create_fine_tuning_job(
        training_file_id=file_id,
        model='gpt-35-turbo',
        suffix='my-model'
    )

    # Wait for completion
    model_id = finetuner.wait_for_completion(job.id)

    # Use fine-tuned model
    messages = [{"role": "user", "content": "Explain decorators"}]
    response = finetuner.use_fine_tuned_model(model_id, messages)
    print(f"\nFine-tuned model response:\n{response}")
```

Azure Cost Comparison

```
"""
Azure OpenAI Service Cost Comparison
"""

print("Azure OpenAI Service Cost Comparison:")
print("="*70)

print("\n1. PAY-AS-YOU-GO PRICING")
print("-"*70)
print("Model                    Input               Output")
print("GPT-4                    $0.03 / 1K tokens   $0.06 / 1K tokens")
```

```
print("GPT-4 Turbo              $0.01 / 1K tokens   $0.03 / 1K tokens")
print("GPT-3.5 Turbo            $0.0005 / 1K tokens $0.0015 / 1K tokens")
print("GPT-3.5 Turbo 16K        $0.003 / 1K tokens  $0.004 / 1K tokens")
print("Embeddings (ada-002)     $0.0001 / 1K tokens")
print("DALL-E 3 (1024x1024)     $0.04 / image")
print("Whisper                  $0.006 / minute")
print("\nExample: 1M input + 1M output GPT-4 = $90")

print("\n2. PROVISIONED THROUGHPUT (PTU)")
print("-"*70)
print("Model                    Monthly Cost (100 PTU)")
print("GPT-4                    $36,400")
print("GPT-3.5 Turbo            $7,300")
print("\nNote: PTU for consistent high-volume workloads")

print("\n3. FINE-TUNING")
print("-"*70)
print("Model                    Training            Hosting
Input/Output")
print("GPT-3.5 Turbo            $0.008 / 1K tokens  $0.001 / hour       $0.012 /
$0.016 per 1K")
print("\nExample: Training on 100K tokens = $0.80")

print("\n4. COST OPTIMIZATION TIPS")
print("-"*70)
print("✓ Use GPT-3.5 Turbo for most tasks (20x cheaper than GPT-4)")
print("✓ Use content filtering to reduce wasted tokens")
print("✓ Implement caching for repeated queries")
print("✓ Use PTU for >50M tokens/month (better economics)")
print("✓ Fine-tune GPT-3.5 for specialized tasks (better than GPT-4)")
```

LLM Deployment Strategies

Multi-Cloud LLM Comparison

Feature	AWS Bedrock	GCP Vertex AI	Azure OpenAI
Available Models	Claude, Llama 2, Titan	PaLM 2, Gemini, Llama 2	GPT-4, GPT-3.5, DALL-E
Fine-tuning	Limited (Titan only)	Yes (PaLM 2)	Yes (GPT-3.5)
Custom Training	SageMaker	Vertex AI Training	Azure ML
Context Window	Up to 200K (Claude)	Up to 32K (PaLM 2)	Up to 128K (GPT-4 Turbo)
Pricing Model	Pay-per-token	Pay-per-char	Pay-per-token
Free Tier	No	$300 credit*	No

Feature	AWS Bedrock	GCP Vertex AI	Azure OpenAI
Best For	Enterprise, AWS ecosystem	Research, multimodal	GPT-4 access, Azure ecosystem
Latency	Low	Low	Medium

Note: *Pricing and free tier credits are subject to change. Always verify current rates and offerings from the respective cloud providers.* | **RAG Support** | Knowledge Bases | Grounding | Azure AI Search |

Deployment Best Practices

```
"""
LLM Deployment Best Practices
"""

class LLMDeploymentBestPractices:
    """
    Production LLM deployment patterns
    """

    @staticmethod
    def implement_caching():
        """
        Cache responses to reduce costs
        """
        import hashlib
        import json
        from functools import lru_cache

        class LLMCache:
            def __init__(self):
                self.cache = {}

            def get_cache_key(self, prompt, params):
                """Generate cache key"""
                cache_str = json.dumps({'prompt': prompt, 'params': params},
sort_keys=True)
                return hashlib.md5(cache_str.encode()).hexdigest()

            def get(self, prompt, params):
                """Get cached response"""
                key = self.get_cache_key(prompt, params)
                return self.cache.get(key)

            def set(self, prompt, params, response):
                """Cache response"""
                key = self.get_cache_key(prompt, params)
                self.cache[key] = response

        cache = LLMCache()
```

```
        print("✓ Caching enabled")
        print("  Typical savings: 30-50% on repeated queries")

        return cache

    @staticmethod
    def implement_rate_limiting():
        """
        Rate limiting to control costs
        """
        from collections import deque
        import time

        class RateLimiter:
            def __init__(self, max_requests_per_minute=60):
                self.max_requests = max_requests_per_minute
                self.requests = deque()

            def allow_request(self):
                """Check if request allowed"""
                now = time.time()

                # Remove old requests
                while self.requests and now - self.requests[0] > 60:
                    self.requests.popleft()

                if len(self.requests) < self.max_requests:
                    self.requests.append(now)
                    return True

                return False

        limiter = RateLimiter(max_requests_per_minute=100)

        print("✓ Rate limiting enabled")
        print("  Max: 100 requests/minute")

        return limiter

    @staticmethod
    def implement_prompt_optimization():
        """
        Optimize prompts to reduce token usage
        """
        class PromptOptimizer:
            @staticmethod
            def compress_prompt(prompt):
                """Remove unnecessary words"""
                # Remove extra whitespace
                prompt = ' '.join(prompt.split())

                # Remove filler words
                fillers = ['um', 'uh', 'like', 'you know', 'basically',
'actually']
```

```
                for filler in fillers:
                    prompt = prompt.replace(f' {filler} ', ' ')

                return prompt

            @staticmethod
            def use_few_shot(task, examples, query):
                """Efficient few-shot prompting"""
                prompt = f"Task: {task}\n\n"

                # Use only 2-3 examples
                for i, ex in enumerate(examples[:3]):
                    prompt += f"Example {i+1}:\n"
                    prompt += f"Input: {ex['input']}\n"
                    prompt += f"Output: {ex['output']}\n\n"

                prompt += f"Now do:\nInput: {query}\nOutput:"

                return prompt

        optimizer = PromptOptimizer()

        print("✓ Prompt optimization enabled")
        print("  Typical savings: 20-30% tokens")

        return optimizer

    @staticmethod
    def implement_fallback_strategy():
        """
        Fallback to cheaper model if possible
        """
        class SmartLLMRouter:
            def __init__(self):
                self.complexity_threshold = 0.5

            def estimate_complexity(self, prompt):
                """Estimate query complexity"""
                # Simple heuristic
                complexity_score = 0

                # Length
                if len(prompt) > 500:
                    complexity_score += 0.3

                # Technical keywords
                technical_words = ['algorithm', 'theory', 'mathematical',
'complex']
                if any(word in prompt.lower() for word in technical_words):
                    complexity_score += 0.4

                # Code
                if '```' in prompt or 'def ' in prompt:
                    complexity_score += 0.3
```

```
                return min(complexity_score, 1.0)

            def route_request(self, prompt):
                """Route to appropriate model"""
                complexity = self.estimate_complexity(prompt)

                if complexity < self.complexity_threshold:
                    return 'gpt-3.5-turbo'  # Cheaper
                else:
                    return 'gpt-4'  # More capable

        router = SmartLLMRouter()

        print("✓ Smart routing enabled")
        print("  Routes simple queries to cheaper models")

        return router

    @staticmethod
    def implement_monitoring():
        """
        Monitor LLM usage and costs
        """
        class LLMMonitor:
            def __init__(self):
                self.stats = {
                    'total_requests': 0,
                    'total_tokens': 0,
                    'total_cost': 0.0,
                    'by_model': {}
                }

            def log_request(self, model, input_tokens, output_tokens, cost):
                """Log request metrics"""
                self.stats['total_requests'] += 1
                self.stats['total_tokens'] += input_tokens + output_tokens
                self.stats['total_cost'] += cost

                if model not in self.stats['by_model']:
                    self.stats['by_model'][model] = {
                        'requests': 0,
                        'tokens': 0,
                        'cost': 0.0
                    }

                self.stats['by_model'][model]['requests'] += 1
                self.stats['by_model'][model]['tokens'] += input_tokens +
output_tokens
                self.stats['by_model'][model]['cost'] += cost

            def get_report(self):
                """Generate usage report"""
                print("\n" + "="*60)
```

```
                print("LLM USAGE REPORT")
                print("="*60)
                print(f"Total Requests: {self.stats['total_requests']}")
                print(f"Total Tokens: {self.stats['total_tokens']:,}")
                print(f"Total Cost: ${self.stats['total_cost']:.2f}")
                print("\nBy Model:")
                for model, stats in self.stats['by_model'].items():
                    print(f"  {model}:")
                    print(f"    Requests: {stats['requests']}")
                    print(f"    Tokens: {stats['tokens']:,}")
                    print(f"    Cost: ${stats['cost']:.2f}")
                print("="*60)

        monitor = LLMMonitor()

        print("✓ Monitoring enabled")

        return monitor

# Example: Complete production setup
if __name__ == "__main__":
    practices = LLMDeploymentBestPractices()

    # Setup all components
    cache = practices.implement_caching()
    limiter = practices.implement_rate_limiting()
    optimizer = practices.implement_prompt_optimization()
    router = practices.implement_fallback_strategy()
    monitor = practices.implement_monitoring()

    print("\n✓ Production LLM system configured")
    print("\nExpected benefits:")
    print("  - 30-50% cost reduction (caching)")
    print("  - 20-30% token savings (prompt optimization)")
    print("  - 40% cost reduction (smart routing)")
    print("  - Complete visibility (monitoring)")
```

Part 4: Computer Vision & Deep Learning

Computer Vision Terminology

Term	Definition
CNN	Convolutional Neural Network - architecture for image processing
Convolution	Operation that applies filters to extract features
Pooling	Downsampling operation (max pool, average pool)
Kernel/Filter	Small matrix for feature detection

Term	Definition
Stride	Step size for moving filter across image
Padding	Adding borders to maintain dimensions
Receptive Field	Area of input that affects a neuron
Feature Map	Output of convolution operation
Transfer Learning	Using pre-trained models on new tasks
Data Augmentation	Artificially expanding dataset (rotation, flip, etc.)
IoU	Intersection over Union - object detection metric
mAP	Mean Average Precision - detection quality metric
Backbone	Feature extraction network in detection models
Anchor Boxes	Pre-defined bounding boxes for detection
NMS	Non-Maximum Suppression - removing duplicate detections

CNN Architectures

Classic CNN Architectures

```
"""
Classic CNN Architectures for Image Classification
"""

import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision import models, transforms, datasets
from torch.utils.data import DataLoader
import numpy as np

class LeNet5(nn.Module):
    """
    LeNet-5 (1998) - First successful CNN
    Use case: Handwritten digit recognition (MNIST)
    """
    def __init__(self, num_classes=10):
        super(LeNet5, self).__init__()
        self.conv1 = nn.Conv2d(1, 6, kernel_size=5, padding=2)
        self.conv2 = nn.Conv2d(6, 16, kernel_size=5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, num_classes)

    def forward(self, x):
        x = F.max_pool2d(F.relu(self.conv1(x)), 2)
        x = F.max_pool2d(F.relu(self.conv2(x)), 2)
```

```
        x = x.view(-1, 16 * 5 * 5)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x

class AlexNet_Custom(nn.Module):
    """
    AlexNet (2012) - ImageNet winner
    Use case: General image classification
    """
    def __init__(self, num_classes=1000):
        super(AlexNet_Custom, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=11, stride=4, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),

            nn.Conv2d(64, 192, kernel_size=5, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),

            nn.Conv2d(192, 384, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),

            nn.Conv2d(384, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),

            nn.Conv2d(256, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
        )

        self.classifier = nn.Sequential(
            nn.Dropout(),
            nn.Linear(256 * 6 * 6, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(),
            nn.Linear(4096, 4096),
            nn.ReLU(inplace=True),
            nn.Linear(4096, num_classes),
        )

    def forward(self, x):
        x = self.features(x)
        x = x.view(x.size(0), -1)
        x = self.classifier(x)
        return x

class VGGBlock(nn.Module):
    """VGG Block: Conv → ReLU → Conv → ReLU → MaxPool"""
    def __init__(self, in_channels, out_channels, num_convs):
        super(VGGBlock, self).__init__()
        layers = []
```

```
        for _ in range(num_convs):
            layers.append(nn.Conv2d(in_channels, out_channels, kernel_size=3,
padding=1))
            layers.append(nn.ReLU(inplace=True))
            in_channels = out_channels
        layers.append(nn.MaxPool2d(kernel_size=2, stride=2))
        self.block = nn.Sequential(*layers)

    def forward(self, x):
        return self.block(x)

class VGG16(nn.Module):
    """
    VGG-16 (2014) - Very Deep CNN
    Use case: Feature extraction, transfer learning
    """
    def __init__(self, num_classes=1000):
        super(VGG16, self).__init__()
        self.features = nn.Sequential(
            VGGBlock(3, 64, 2),      # 64 channels, 2 conv layers
            VGGBlock(64, 128, 2),    # 128 channels, 2 conv layers
            VGGBlock(128, 256, 3),   # 256 channels, 3 conv layers
            VGGBlock(256, 512, 3),   # 512 channels, 3 conv layers
            VGGBlock(512, 512, 3),   # 512 channels, 3 conv layers
        )

        self.classifier = nn.Sequential(
            nn.Linear(512 * 7 * 7, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(),
            nn.Linear(4096, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(),
            nn.Linear(4096, num_classes)
        )

    def forward(self, x):
        x = self.features(x)
        x = x.view(x.size(0), -1)
        x = self.classifier(x)
        return x

class ResidualBlock(nn.Module):
    """
    Residual Block with skip connection
    Key innovation: x_out = F(x) + x
    """
    def __init__(self, in_channels, out_channels, stride=1):
        super(ResidualBlock, self).__init__()

        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3,
                               stride=stride, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_channels)
```

```
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3,
                               stride=1, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_channels)

        # Shortcut connection
        self.shortcut = nn.Sequential()
        if stride != 1 or in_channels != out_channels:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_channels, out_channels, kernel_size=1, stride=stride,
bias=False),
                nn.BatchNorm2d(out_channels)
            )

    def forward(self, x):
        identity = x

        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))

        out += self.shortcut(identity)  # Skip connection
        out = F.relu(out)

        return out

class ResNet18(nn.Module):
    """
    ResNet-18 (2015) - Residual Networks
    Use case: Deep networks without vanishing gradients
    """
    def __init__(self, num_classes=1000):
        super(ResNet18, self).__init__()

        self.conv1 = nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3,
bias=False)
        self.bn1 = nn.BatchNorm2d(64)
        self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)

        # Residual layers
        self.layer1 = self._make_layer(64, 64, 2, stride=1)
        self.layer2 = self._make_layer(64, 128, 2, stride=2)
        self.layer3 = self._make_layer(128, 256, 2, stride=2)
        self.layer4 = self._make_layer(256, 512, 2, stride=2)

        self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
        self.fc = nn.Linear(512, num_classes)

    def _make_layer(self, in_channels, out_channels, num_blocks, stride):
        layers = []
        layers.append(ResidualBlock(in_channels, out_channels, stride))
        for _ in range(1, num_blocks):
            layers.append(ResidualBlock(out_channels, out_channels, 1))
        return nn.Sequential(*layers)

    def forward(self, x):
```

```
        x = F.relu(self.bn1(self.conv1(x)))
        x = self.maxpool(x)

        x = self.layer1(x)
        x = self.layer2(x)
        x = self.layer3(x)
        x = self.layer4(x)

        x = self.avgpool(x)
        x = x.view(x.size(0), -1)
        x = self.fc(x)

        return x

class CVTrainer:
    """
    Computer Vision Model Trainer
    """

    def __init__(self, model, device='cuda'):
        """
        Initialize trainer

        Args:
            model: PyTorch model
            device: 'cuda' or 'cpu'
        """
        self.model = model.to(device)
        self.device = device

        print(f"✓ Model initialized on {device}")
        print(f"  Parameters: {sum(p.numel() for p in model.parameters()):,}")

    def train(self, train_loader, val_loader, epochs=10, lr=0.001):
        """Train model"""
        criterion = nn.CrossEntropyLoss()
        optimizer = torch.optim.Adam(self.model.parameters(), lr=lr)
        scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, 'min',
patience=2)

        best_val_acc = 0.0

        for epoch in range(epochs):
            # Training
            self.model.train()
            train_loss = 0.0
            train_correct = 0
            train_total = 0

            for images, labels in train_loader:
                images, labels = images.to(self.device), labels.to(self.device)

                optimizer.zero_grad()
                outputs = self.model(images)
```

```
                loss = criterion(outputs, labels)
                loss.backward()
                optimizer.step()

                train_loss += loss.item()
                _, predicted = outputs.max(1)
                train_total += labels.size(0)
                train_correct += predicted.eq(labels).sum().item()

            train_acc = 100. * train_correct / train_total

            # Validation
            self.model.eval()
            val_loss = 0.0
            val_correct = 0
            val_total = 0

            with torch.no_grad():
                for images, labels in val_loader:
                    images, labels = images.to(self.device),
labels.to(self.device)

                    outputs = self.model(images)
                    loss = criterion(outputs, labels)

                    val_loss += loss.item()
                    _, predicted = outputs.max(1)
                    val_total += labels.size(0)
                    val_correct += predicted.eq(labels).sum().item()

            val_acc = 100. * val_correct / val_total

            # Learning rate scheduling
            scheduler.step(val_loss)

            print(f"Epoch {epoch+1}/{epochs}:")
            print(f"  Train Loss: {train_loss/len(train_loader):.4f}, Acc:
{train_acc:.2f}%")
            print(f"  Val Loss: {val_loss/len(val_loader):.4f}, Acc:
{val_acc:.2f}%")

            # Save best model
            if val_acc > best_val_acc:
                best_val_acc = val_acc
                torch.save(self.model.state_dict(), 'best_model.pth')
                print(f"  ✓ Best model saved (Val Acc: {val_acc:.2f}%)")

        print(f"\n✓ Training complete!")
        print(f"  Best Validation Accuracy: {best_val_acc:.2f}%")

# Example usage
if __name__ == "__main__":
    # Load dataset (CIFAR-10)
    transform = transforms.Compose([
```

```
        transforms.RandomHorizontalFlip(),
        transforms.RandomCrop(32, padding=4),
        transforms.ToTensor(),
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
    ])

    train_dataset = datasets.CIFAR10(root='./data', train=True, download=True,
transform=transform)
    val_dataset = datasets.CIFAR10(root='./data', train=False, download=True,
transform=transform)

    train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True,
num_workers=4)
    val_loader = DataLoader(val_dataset, batch_size=128, shuffle=False,
num_workers=4)

    # Train ResNet-18
    model = ResNet18(num_classes=10)
    trainer = CVTrainer(model, device='cuda')
    trainer.train(train_loader, val_loader, epochs=50, lr=0.001)
```

Modern Vision Architectures

Vision Transformer (ViT) & EfficientNet

```
"""
Modern Vision Architectures
Vision Transformer (ViT) and EfficientNet
"""

import torch
import torch.nn as nn
import math

class PatchEmbedding(nn.Module):
    """
    Split image into patches and embed them
    """
    def __init__(self, img_size=224, patch_size=16, in_channels=3, embed_dim=768):
        super().__init__()
        self.img_size = img_size
        self.patch_size = patch_size
        self.num_patches = (img_size // patch_size) ** 2

        # Convolution to create patches and embed
        self.proj = nn.Conv2d(in_channels, embed_dim, kernel_size=patch_size,
stride=patch_size)

    def forward(self, x):
        # x: (B, C, H, W)
        x = self.proj(x)  # (B, embed_dim, H/P, W/P)
```

```
        x = x.flatten(2)  # (B, embed_dim, num_patches)
        x = x.transpose(1, 2)  # (B, num_patches, embed_dim)
        return x

class MultiHeadAttention(nn.Module):
    """Multi-head self-attention"""
    def __init__(self, embed_dim, num_heads, dropout=0.1):
        super().__init__()
        self.embed_dim = embed_dim
        self.num_heads = num_heads
        self.head_dim = embed_dim // num_heads

        self.qkv = nn.Linear(embed_dim, embed_dim * 3)
        self.proj = nn.Linear(embed_dim, embed_dim)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        B, N, C = x.shape

        # Generate Q, K, V
        qkv = self.qkv(x).reshape(B, N, 3, self.num_heads, self.head_dim)
        qkv = qkv.permute(2, 0, 3, 1, 4)  # (3, B, num_heads, N, head_dim)
        q, k, v = qkv[0], qkv[1], qkv[2]

        # Attention
        attn = (q @ k.transpose(-2, -1)) / math.sqrt(self.head_dim)
        attn = attn.softmax(dim=-1)
        attn = self.dropout(attn)

        x = (attn @ v).transpose(1, 2).reshape(B, N, C)
        x = self.proj(x)
        x = self.dropout(x)

        return x

class TransformerBlock(nn.Module):
    """Transformer encoder block"""
    def __init__(self, embed_dim, num_heads, mlp_ratio=4.0, dropout=0.1):
        super().__init__()

        self.norm1 = nn.LayerNorm(embed_dim)
        self.attn = MultiHeadAttention(embed_dim, num_heads, dropout)

        self.norm2 = nn.LayerNorm(embed_dim)
        mlp_hidden_dim = int(embed_dim * mlp_ratio)
        self.mlp = nn.Sequential(
            nn.Linear(embed_dim, mlp_hidden_dim),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(mlp_hidden_dim, embed_dim),
            nn.Dropout(dropout)
        )

    def forward(self, x):
```

```
        # Attention with residual
        x = x + self.attn(self.norm1(x))

        # MLP with residual
        x = x + self.mlp(self.norm2(x))

        return x

class VisionTransformer(nn.Module):
    """
    Vision Transformer (ViT) - 2020

    Key innovation: Apply Transformers directly to image patches
    Use case: State-of-the-art image classification
    """
    def __init__(self, img_size=224, patch_size=16, in_channels=3,
num_classes=1000,
                 embed_dim=768, depth=12, num_heads=12, mlp_ratio=4.0,
dropout=0.1):
        super().__init__()

        # Patch embedding
        self.patch_embed = PatchEmbedding(img_size, patch_size, in_channels,
embed_dim)
        num_patches = self.patch_embed.num_patches

        # Class token
        self.cls_token = nn.Parameter(torch.zeros(1, 1, embed_dim))

        # Position embedding
        self.pos_embed = nn.Parameter(torch.zeros(1, num_patches + 1, embed_dim))

        self.dropout = nn.Dropout(dropout)

        # Transformer blocks
        self.blocks = nn.Sequential(*[
            TransformerBlock(embed_dim, num_heads, mlp_ratio, dropout)
            for _ in range(depth)
        ])

        self.norm = nn.LayerNorm(embed_dim)

        # Classification head
        self.head = nn.Linear(embed_dim, num_classes)

        # Initialize weights
        nn.init.trunc_normal_(self.pos_embed, std=0.02)
        nn.init.trunc_normal_(self.cls_token, std=0.02)

    def forward(self, x):
        B = x.shape[0]

        # Patch embedding
        x = self.patch_embed(x)
```

```
        # Prepend class token
        cls_tokens = self.cls_token.expand(B, -1, -1)
        x = torch.cat((cls_tokens, x), dim=1)

        # Add position embedding
        x = x + self.pos_embed
        x = self.dropout(x)

        # Transformer blocks
        x = self.blocks(x)
        x = self.norm(x)

        # Classification (use class token)
        x = x[:, 0]
        x = self.head(x)

        return x

class EfficientNet_Block(nn.Module):
    """
    Mobile Inverted Bottleneck (MBConv) block
    """
    def __init__(self, in_channels, out_channels, expand_ratio, stride):
        super().__init__()

        hidden_dim = in_channels * expand_ratio
        self.use_residual = (stride == 1 and in_channels == out_channels)

        layers = []

        # Expansion
        if expand_ratio != 1:
            layers.append(nn.Conv2d(in_channels, hidden_dim, 1, bias=False))
            layers.append(nn.BatchNorm2d(hidden_dim))
            layers.append(nn.SiLU(inplace=True))

        # Depthwise conv
        layers.extend([
            nn.Conv2d(hidden_dim, hidden_dim, 3, stride=stride, padding=1,
groups=hidden_dim, bias=False),
            nn.BatchNorm2d(hidden_dim),
            nn.SiLU(inplace=True)
        ])

        # Projection
        layers.extend([
            nn.Conv2d(hidden_dim, out_channels, 1, bias=False),
            nn.BatchNorm2d(out_channels)
        ])

        self.conv = nn.Sequential(*layers)

    def forward(self, x):
```

```
        if self.use_residual:
            return x + self.conv(x)
        else:
            return self.conv(x)

class EfficientNetB0(nn.Module):
    """
    EfficientNet-B0 (2019)

    Key innovation: Compound scaling (depth, width, resolution)
    Use case: Efficient high-performance classification
    """
    def __init__(self, num_classes=1000):
        super().__init__()

        # Stem
        self.stem = nn.Sequential(
            nn.Conv2d(3, 32, 3, stride=2, padding=1, bias=False),
            nn.BatchNorm2d(32),
            nn.SiLU(inplace=True)
        )

        # MBConv blocks
        self.blocks = nn.Sequential(
            # Stage 1: MBConv1, k3x3, s1, exp1, c16, se0.25
            EfficientNet_Block(32, 16, expand_ratio=1, stride=1),

            # Stage 2: MBConv6, k3x3, s2, exp6, c24, se0.25
            EfficientNet_Block(16, 24, expand_ratio=6, stride=2),
            EfficientNet_Block(24, 24, expand_ratio=6, stride=1),

            # Stage 3: MBConv6, k5x5, s2, exp6, c40, se0.25
            EfficientNet_Block(24, 40, expand_ratio=6, stride=2),
            EfficientNet_Block(40, 40, expand_ratio=6, stride=1),

            # Stage 4: MBConv6, k3x3, s2, exp6, c80, se0.25
            EfficientNet_Block(40, 80, expand_ratio=6, stride=2),
            EfficientNet_Block(80, 80, expand_ratio=6, stride=1),
            EfficientNet_Block(80, 80, expand_ratio=6, stride=1),

            # Stage 5: MBConv6, k5x5, s1, exp6, c112, se0.25
            EfficientNet_Block(80, 112, expand_ratio=6, stride=1),
            EfficientNet_Block(112, 112, expand_ratio=6, stride=1),
            EfficientNet_Block(112, 112, expand_ratio=6, stride=1),

            # Stage 6: MBConv6, k5x5, s2, exp6, c192, se0.25
            EfficientNet_Block(112, 192, expand_ratio=6, stride=2),
            EfficientNet_Block(192, 192, expand_ratio=6, stride=1),
            EfficientNet_Block(192, 192, expand_ratio=6, stride=1),
            EfficientNet_Block(192, 192, expand_ratio=6, stride=1),

            # Stage 7: MBConv6, k3x3, s1, exp6, c320, se0.25
            EfficientNet_Block(192, 320, expand_ratio=6, stride=1),
        )
```

```
        # Head
        self.head = nn.Sequential(
            nn.Conv2d(320, 1280, 1, bias=False),
            nn.BatchNorm2d(1280),
            nn.SiLU(inplace=True),
            nn.AdaptiveAvgPool2d(1),
            nn.Flatten(),
            nn.Dropout(0.2),
            nn.Linear(1280, num_classes)
        )

    def forward(self, x):
        x = self.stem(x)
        x = self.blocks(x)
        x = self.head(x)
        return x

# Model comparison
print("Modern Vision Architecture Comparison:")
print("="*70)
print("Model                Parameters    ImageNet Top-1    Speed (ms)")
print("-"*70)
print("ViT-Base             86M           81.8%             50")
print("EfficientNet-B0      5.3M          77.1%             10")
print("EfficientNet-B7      66M           84.3%             80")
print("ResNet-50            25M           76.1%             15")
print("ResNet-152           60M           78.3%             45")
```

Object Detection

YOLO, Faster R-CNN, and RetinaNet

```
"""
Object Detection Models
YOLO, Faster R-CNN, RetinaNet
"""

import torch
import torch.nn as nn
import torchvision
from torchvision.models.detection import fasterrcnn_resnet50_fpn,
retinanet_resnet50_fpn
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor

class ObjectDetector:
    """
    Object Detection with Pre-trained Models
    """

    @staticmethod
```

```
    def get_faster_rcnn(num_classes=91, pretrained=True):
        """
        Faster R-CNN with ResNet-50 backbone

        Two-stage detector:
        1. Region Proposal Network (RPN)
        2. Fast R-CNN detector

        Use case: High accuracy object detection
        """
        # Load pre-trained model
        model = fasterrcnn_resnet50_fpn(pretrained=pretrained)

        # Replace classification head
        in_features = model.roi_heads.box_predictor.cls_score.in_features
        model.roi_heads.box_predictor = FastRCNNPredictor(in_features,
num_classes)

        print("✓ Faster R-CNN initialized")
        print(f"  Backbone: ResNet-50 + FPN")
        print(f"  Classes: {num_classes}")

        return model

    @staticmethod
    def get_retinanet(num_classes=91, pretrained=True):
        """
        RetinaNet with ResNet-50 backbone

        Single-stage detector with:
        - Feature Pyramid Network (FPN)
        - Focal Loss for class imbalance

        Use case: Balanced speed and accuracy
        """
        model = retinanet_resnet50_fpn(pretrained=pretrained)

        # Modify for custom classes
        # (Implementation details...)

        print("✓ RetinaNet initialized")
        return model

    @staticmethod
    def get_yolov5(model_size='s', num_classes=80):
        """
        YOLOv5 (You Only Look Once v5)

        Single-stage detector:
        - Very fast inference
        - Good for real-time applications

        Model sizes: n (nano), s (small), m (medium), l (large), x (xlarge)
```

```
        Use case: Real-time object detection
        """
        # Using YOLOv5 from torch.hub
        model = torch.hub.load('ultralytics/yolov5', f'yolov5{model_size}',
                               pretrained=True, classes=num_classes)

        print(f"✓ YOLOv5-{model_size} initialized")
        print(f"  Speed: {'Very Fast' if model_size in ['n', 's'] else 'Fast' if
model_size == 'm' else 'Medium'}")
        print(f"  Accuracy: {'Medium' if model_size in ['n', 's'] else 'High' if
model_size == 'm' else 'Very High'}")

        return model

    @staticmethod
    def detect_objects(model, image_path, confidence_threshold=0.5):
        """
        Detect objects in image

        Returns:
            boxes: Bounding boxes
            labels: Class labels
            scores: Confidence scores
        """
        import cv2
        from PIL import Image

        # Load image
        image = Image.open(image_path).convert('RGB')

        # Convert to tensor
        transform = torchvision.transforms.ToTensor()
        image_tensor = transform(image).unsqueeze(0)

        # Detect
        model.eval()
        with torch.no_grad():
            predictions = model(image_tensor)

        # Filter by confidence
        pred = predictions[0]
        keep = pred['scores'] > confidence_threshold

        boxes = pred['boxes'][keep].cpu().numpy()
        labels = pred['labels'][keep].cpu().numpy()
        scores = pred['scores'][keep].cpu().numpy()

        print(f"Detected {len(boxes)} objects:")
        for i, (box, label, score) in enumerate(zip(boxes, labels, scores)):
            print(f"  {i+1}. Class {label}, Confidence: {score:.2f}, Box: {box}")

        return boxes, labels, scores

    @staticmethod
```

```
    def train_detector(model, train_loader, val_loader, epochs=10, lr=0.005):
        """
        Train object detection model
        """
        device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
        model.to(device)

        # Optimizer
        params = [p for p in model.parameters() if p.requires_grad]
        optimizer = torch.optim.SGD(params, lr=lr, momentum=0.9,
weight_decay=0.0005)

        # Learning rate scheduler
        lr_scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=3,
gamma=0.1)

        for epoch in range(epochs):
            model.train()
            epoch_loss = 0

            for images, targets in train_loader:
                images = list(image.to(device) for image in images)
                targets = [{k: v.to(device) for k, v in t.items()} for t in
targets]

                # Forward pass
                loss_dict = model(images, targets)
                losses = sum(loss for loss in loss_dict.values())

                # Backward pass
                optimizer.zero_grad()
                losses.backward()
                optimizer.step()

                epoch_loss += losses.item()

            lr_scheduler.step()

            print(f"Epoch {epoch+1}/{epochs}, Loss:
{epoch_loss/len(train_loader):.4f}")

        print("✓ Training complete!")

        return model

# Object Detection Comparison
print("\nObject Detection Models Comparison:")
print("="*70)
print("Model            Type         mAP     FPS    Use Case")
print("-"*70)
print("YOLOv5-nano      Single      28.0    238    Real-time edge devices")
print("YOLOv5-small     Single      37.4    143    Real-time applications")
print("YOLOv5-xlarge    Single      50.7    26     High accuracy real-time")
print("Faster R-CNN     Two-stage   42.0    25     High accuracy offline")
```

```
print("RetinaNet        Single      40.8    17     Balanced speed/accuracy")
print("EfficientDet-D0  Single      34.6    98     Mobile deployment")
print("EfficientDet-D7  Single      53.7    10     State-of-the-art accuracy")
```

Generative Adversarial Networks (GANs)

GAN Terminology

Term	Definition
Generator	Network that creates fake samples
Discriminator	Network that distinguishes real vs fake
Adversarial Training	Generator vs Discriminator game
Mode Collapse	Generator produces limited variety
Nash Equilibrium	Balance point in adversarial game
Latent Space	Input noise space for generator
Interpolation	Smooth transitions in latent space
FID Score	Fréchet Inception Distance - quality metric
IS Score	Inception Score - quality and diversity
Conditional GAN	GAN with class labels
Style Transfer	Transferring artistic style to images
Super-resolution	Upscaling image resolution

Basic GAN & DCGAN

```
"""
Generative Adversarial Networks (GANs)
Basic GAN and DCGAN (Deep Convolutional GAN)
"""

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
import numpy as np

class BasicGenerator(nn.Module):
    """
    Basic GAN Generator (Fully Connected)

    Input: Random noise vector (latent_dim)
```

```
    Output: Generated image (flattened)
    """
    def __init__(self, latent_dim=100, img_shape=(1, 28, 28)):
        super(BasicGenerator, self).__init__()

        self.img_shape = img_shape
        img_size = int(np.prod(img_shape))

        self.model = nn.Sequential(
            nn.Linear(latent_dim, 128),
            nn.LeakyReLU(0.2, inplace=True),

            nn.Linear(128, 256),
            nn.BatchNorm1d(256),
            nn.LeakyReLU(0.2, inplace=True),

            nn.Linear(256, 512),
            nn.BatchNorm1d(512),
            nn.LeakyReLU(0.2, inplace=True),

            nn.Linear(512, 1024),
            nn.BatchNorm1d(1024),
            nn.LeakyReLU(0.2, inplace=True),

            nn.Linear(1024, img_size),
            nn.Tanh()  # Output in range [-1, 1]
        )

    def forward(self, z):
        img = self.model(z)
        img = img.view(img.size(0), *self.img_shape)
        return img

class BasicDiscriminator(nn.Module):
    """
    Basic GAN Discriminator (Fully Connected)

    Input: Image (flattened)
    Output: Probability of being real [0, 1]
    """
    def __init__(self, img_shape=(1, 28, 28)):
        super(BasicDiscriminator, self).__init__()

        img_size = int(np.prod(img_shape))

        self.model = nn.Sequential(
            nn.Linear(img_size, 512),
            nn.LeakyReLU(0.2, inplace=True),

            nn.Linear(512, 256),
            nn.LeakyReLU(0.2, inplace=True),

            nn.Linear(256, 1),
            nn.Sigmoid()
```

```
        )

    def forward(self, img):
        img_flat = img.view(img.size(0), -1)
        validity = self.model(img_flat)
        return validity

class DCGAN_Generator(nn.Module):
    """
    DCGAN Generator (Deep Convolutional GAN) - 2016

    Key features:
    - Use transposed convolutions (deconvolutions)
    - No fully connected layers
    - Batch normalization
    - ReLU activations (except output: Tanh)

    Input: Noise vector (latent_dim)
    Output: Generated image (3, 64, 64)
    """
    def __init__(self, latent_dim=100, channels=3):
        super(DCGAN_Generator, self).__init__()

        self.init_size = 4
        self.l1 = nn.Sequential(
            nn.Linear(latent_dim, 512 * self.init_size ** 2)
        )

        self.conv_blocks = nn.Sequential(
            nn.BatchNorm2d(512),

            # 4x4 -> 8x8
            nn.Upsample(scale_factor=2),
            nn.Conv2d(512, 256, 3, stride=1, padding=1),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),

            # 8x8 -> 16x16
            nn.Upsample(scale_factor=2),
            nn.Conv2d(256, 128, 3, stride=1, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),

            # 16x16 -> 32x32
            nn.Upsample(scale_factor=2),
            nn.Conv2d(128, 64, 3, stride=1, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),

            # 32x32 -> 64x64
            nn.Upsample(scale_factor=2),
            nn.Conv2d(64, channels, 3, stride=1, padding=1),
            nn.Tanh()
        )
```

```
    def forward(self, z):
        out = self.l1(z)
        out = out.view(out.shape[0], 512, self.init_size, self.init_size)
        img = self.conv_blocks(out)
        return img

class DCGAN_Discriminator(nn.Module):
    """
    DCGAN Discriminator

    Key features:
    - Use strided convolutions (no pooling)
    - Batch normalization
    - LeakyReLU activations

    Input: Image (3, 64, 64)
    Output: Probability of being real
    """
    def __init__(self, channels=3):
        super(DCGAN_Discriminator, self).__init__()

        self.model = nn.Sequential(
            # 64x64 -> 32x32
            nn.Conv2d(channels, 64, 4, 2, 1, bias=False),
            nn.LeakyReLU(0.2, inplace=True),

            # 32x32 -> 16x16
            nn.Conv2d(64, 128, 4, 2, 1, bias=False),
            nn.BatchNorm2d(128),
            nn.LeakyReLU(0.2, inplace=True),

            # 16x16 -> 8x8
            nn.Conv2d(128, 256, 4, 2, 1, bias=False),
            nn.BatchNorm2d(256),
            nn.LeakyReLU(0.2, inplace=True),

            # 8x8 -> 4x4
            nn.Conv2d(256, 512, 4, 2, 1, bias=False),
            nn.BatchNorm2d(512),
            nn.LeakyReLU(0.2, inplace=True),

            # 4x4 -> 1x1
            nn.Conv2d(512, 1, 4, 1, 0, bias=False),
            nn.Sigmoid()
        )

    def forward(self, img):
        validity = self.model(img)
        return validity.view(-1, 1)

class GANTrainer:
    """
    GAN Training Loop
```

```
    """

    def __init__(self, generator, discriminator, latent_dim=100, device='cuda'):
        """
        Initialize GAN trainer

        Args:
            generator: Generator network
            discriminator: Discriminator network
            latent_dim: Dimension of noise vector
            device: 'cuda' or 'cpu'
        """
        self.generator = generator.to(device)
        self.discriminator = discriminator.to(device)
        self.latent_dim = latent_dim
        self.device = device

        # Loss function
        self.adversarial_loss = nn.BCELoss()

        # Optimizers
        self.optimizer_G = optim.Adam(generator.parameters(), lr=0.0002, betas=
(0.5, 0.999))
        self.optimizer_D = optim.Adam(discriminator.parameters(), lr=0.0002,
betas=(0.5, 0.999))

        print(f"✓ GAN initialized on {device}")

    def train(self, dataloader, epochs=100, sample_interval=500):
        """
        Train GAN

        Training loop:
        1. Train Discriminator: maximize log(D(x)) + log(1 - D(G(z)))
        2. Train Generator: maximize log(D(G(z))) (or minimize log(1 - D(G(z))))
        """
        for epoch in range(epochs):
            for i, (imgs, _) in enumerate(dataloader):
                batch_size = imgs.size(0)

                # Ground truth labels
                real_labels = torch.ones(batch_size, 1).to(self.device)
                fake_labels = torch.zeros(batch_size, 1).to(self.device)

                # Real images
                real_imgs = imgs.to(self.device)

                # -----------------
                #  Train Generator
                # -----------------
                self.optimizer_G.zero_grad()

                # Sample noise
                z = torch.randn(batch_size, self.latent_dim).to(self.device)
```

```
                # Generate fake images
                gen_imgs = self.generator(z)

                # Generator loss: fool discriminator
                g_loss = self.adversarial_loss(
                    self.discriminator(gen_imgs),
                    real_labels
                )

                g_loss.backward()
                self.optimizer_G.step()

                # ---------------------
                #  Train Discriminator
                # ---------------------
                self.optimizer_D.zero_grad()

                # Loss on real images
                real_loss = self.adversarial_loss(
                    self.discriminator(real_imgs),
                    real_labels
                )

                # Loss on fake images
                fake_loss = self.adversarial_loss(
                    self.discriminator(gen_imgs.detach()),
                    fake_labels
                )

                # Total discriminator loss
                d_loss = (real_loss + fake_loss) / 2

                d_loss.backward()
                self.optimizer_D.step()

                # Logging
                batches_done = epoch * len(dataloader) + i
                if batches_done % sample_interval == 0:
                    print(f"[Epoch {epoch}/{epochs}] [Batch {i}/{len(dataloader)}]
"
                          f"[D loss: {d_loss.item():.4f}] [G loss:
{g_loss.item():.4f}]")

                    # Save sample images
                    self.save_sample_images(batches_done)

        print("✓ GAN training complete!")

    def save_sample_images(self, batches_done, n_row=5):
        """Generate and save sample images"""
        from torchvision.utils import save_image

        # Sample noise
```

```
        z = torch.randn(n_row ** 2, self.latent_dim).to(self.device)

        # Generate images
        gen_imgs = self.generator(z)

        # Save
        save_image(gen_imgs.data, f"images/{batches_done}.png", nrow=n_row,
normalize=True)

# Example usage
if __name__ == "__main__":
    # Setup
    latent_dim = 100
    img_shape = (1, 28, 28)  # MNIST

    # Create models
    generator = DCGAN_Generator(latent_dim=latent_dim, channels=1)
    discriminator = DCGAN_Discriminator(channels=1)

    # Load data (MNIST)
    transform = transforms.Compose([
        transforms.Resize(64),
        transforms.ToTensor(),
        transforms.Normalize([0.5], [0.5])
    ])

    dataset = datasets.MNIST(root='./data', train=True, download=True,
transform=transform)
    dataloader = DataLoader(dataset, batch_size=64, shuffle=True)

    # Train
    trainer = GANTrainer(generator, discriminator, latent_dim, device='cuda')
    trainer.train(dataloader, epochs=100, sample_interval=200)
```

4.5 Advanced GANs

Terminology for Advanced GANs

Term	Definition
StyleGAN	GAN with style-based generator (produces high-quality faces)
StyleGAN2	Improved StyleGAN (better quality, fewer artifacts)
CycleGAN	Unpaired image-to-image translation (horse ↔ zebra)
Pix2Pix	Paired image-to-image translation (sketch → photo)
Conditional GAN (cGAN)	GAN with class labels (generate specific digits)
Progressive GAN	Grows from low to high resolution

Term	Definition
WGAN	Wasserstein GAN (more stable training)
WGAN-GP	WGAN with Gradient Penalty
BigGAN	Large-scale GAN for ImageNet (512×512)
StyleGAN3	Translation and rotation equivariant

4.5.1 Conditional GAN (cGAN)

```
"""
Conditional GAN - Generate specific classes
Example: Generate specific MNIST digits (0-9)
"""

import torch
import torch.nn as nn
import torch.nn.functional as F

class ConditionalGenerator(nn.Module):
    """
    Generator with class conditioning
    """
    def __init__(self, latent_dim=100, n_classes=10, img_size=28):
        super(ConditionalGenerator, self).__init__()

        self.latent_dim = latent_dim
        self.n_classes = n_classes
        self.img_size = img_size

        # Embedding for class labels
        self.label_emb = nn.Embedding(n_classes, n_classes)

        # Generator network
        self.model = nn.Sequential(
            nn.Linear(latent_dim + n_classes, 256),
            nn.LeakyReLU(0.2),
            nn.BatchNorm1d(256),

            nn.Linear(256, 512),
            nn.LeakyReLU(0.2),
            nn.BatchNorm1d(512),

            nn.Linear(512, 1024),
            nn.LeakyReLU(0.2),
            nn.BatchNorm1d(1024),

            nn.Linear(1024, img_size * img_size),
            nn.Tanh()
        )
```

```
    def forward(self, noise, labels):
        """
        Args:
            noise: (batch_size, latent_dim)
            labels: (batch_size,) integer labels
        """
        # Embed labels
        label_input = self.label_emb(labels)

        # Concatenate noise and labels
        gen_input = torch.cat([noise, label_input], dim=1)

        # Generate
        img = self.model(gen_input)
        img = img.view(img.size(0), 1, self.img_size, self.img_size)

        return img


class ConditionalDiscriminator(nn.Module):
    """
    Discriminator with class conditioning
    """
    def __init__(self, n_classes=10, img_size=28):
        super(ConditionalDiscriminator, self).__init__()

        self.n_classes = n_classes
        self.img_size = img_size

        # Embedding for class labels
        self.label_emb = nn.Embedding(n_classes, n_classes)

        # Discriminator network
        self.model = nn.Sequential(
            nn.Linear(img_size * img_size + n_classes, 512),
            nn.LeakyReLU(0.2),
            nn.Dropout(0.3),

            nn.Linear(512, 256),
            nn.LeakyReLU(0.2),
            nn.Dropout(0.3),

            nn.Linear(256, 1),
            nn.Sigmoid()
        )

    def forward(self, img, labels):
        """
        Args:
            img: (batch_size, 1, img_size, img_size)
            labels: (batch_size,) integer labels
        """
        # Flatten image
```

```
        img_flat = img.view(img.size(0), -1)

        # Embed labels
        label_input = self.label_emb(labels)

        # Concatenate image and labels
        d_input = torch.cat([img_flat, label_input], dim=1)

        # Discriminate
        validity = self.model(d_input)

        return validity


def train_conditional_gan(generator, discriminator, dataloader, epochs=100,
device='cuda'):
    """
    Train conditional GAN
    """
    import torch.optim as optim
    from tqdm import tqdm

    generator = generator.to(device)
    discriminator = discriminator.to(device)

    # Optimizers
    optimizer_G = optim.Adam(generator.parameters(), lr=0.0002, betas=(0.5,
0.999))
    optimizer_D = optim.Adam(discriminator.parameters(), lr=0.0002, betas=(0.5,
0.999))

    # Loss
    adversarial_loss = nn.BCELoss()

    for epoch in range(epochs):
        for i, (imgs, labels) in enumerate(tqdm(dataloader, desc=f"Epoch
{epoch+1}/{epochs}")):
            batch_size = imgs.size(0)

            # Move to device
            real_imgs = imgs.to(device)
            labels = labels.to(device)

            # Real and fake labels
            valid = torch.ones(batch_size, 1).to(device)
            fake = torch.zeros(batch_size, 1).to(device)

            # ---------------------
            #  Train Discriminator
            # ---------------------
            optimizer_D.zero_grad()

            # Real images
            real_loss = adversarial_loss(discriminator(real_imgs, labels), valid)
```

```
            # Fake images
            z = torch.randn(batch_size, generator.latent_dim).to(device)
            gen_labels = torch.randint(0, generator.n_classes,
(batch_size,)).to(device)
            gen_imgs = generator(z, gen_labels)
            fake_loss = adversarial_loss(discriminator(gen_imgs.detach(),
gen_labels), fake)

            # Total discriminator loss
            d_loss = (real_loss + fake_loss) / 2
            d_loss.backward()
            optimizer_D.step()

            # -----------------
            #  Train Generator
            # -----------------
            optimizer_G.zero_grad()

            # Generate images
            z = torch.randn(batch_size, generator.latent_dim).to(device)
            gen_labels = torch.randint(0, generator.n_classes,
(batch_size,)).to(device)
            gen_imgs = generator(z, gen_labels)

            # Generator loss
            g_loss = adversarial_loss(discriminator(gen_imgs, gen_labels), valid)
            g_loss.backward()
            optimizer_G.step()

        print(f"[Epoch {epoch+1}/{epochs}] [D loss: {d_loss.item():.4f}] [G loss:
{g_loss.item():.4f}]")

    return generator, discriminator


# Example: Generate specific digits
def generate_specific_digits(generator, digits=[0, 1, 2, 3, 4, 5, 6, 7, 8, 9],
device='cuda'):
    """Generate specific MNIST digits"""
    from torchvision.utils import save_image

    generator.eval()
    with torch.no_grad():
        # Noise
        z = torch.randn(len(digits), generator.latent_dim).to(device)

        # Labels
        labels = torch.tensor(digits).to(device)

        # Generate
        gen_imgs = generator(z, labels)

        # Save
```

```
        save_image(gen_imgs, 'conditional_samples.png', nrow=len(digits),
normalize=True)
        print(f"✓ Generated digits: {digits}")
```

4.5.2 CycleGAN - Unpaired Image Translation

```
"""
CycleGAN - Image-to-Image Translation without Paired Data
Example: Horse → Zebra, Summer → Winter, Photo → Monet
"""

import torch
import torch.nn as nn

class ResidualBlock(nn.Module):
    """Residual block for CycleGAN generator"""
    def __init__(self, in_features):
        super(ResidualBlock, self).__init__()

        self.block = nn.Sequential(
            nn.ReflectionPad2d(1),
            nn.Conv2d(in_features, in_features, 3),
            nn.InstanceNorm2d(in_features),
            nn.ReLU(inplace=True),
            nn.ReflectionPad2d(1),
            nn.Conv2d(in_features, in_features, 3),
            nn.InstanceNorm2d(in_features)
        )

    def forward(self, x):
        return x + self.block(x)


class CycleGAN_Generator(nn.Module):
    """
    CycleGAN Generator (ResNet-based)
    """
    def __init__(self, input_channels=3, num_residual_blocks=9):
        super(CycleGAN_Generator, self).__init__()

        # Initial convolution
        model = [
            nn.ReflectionPad2d(3),
            nn.Conv2d(input_channels, 64, 7),
            nn.InstanceNorm2d(64),
            nn.ReLU(inplace=True)
        ]

        # Downsampling
        in_features = 64
```

```
        out_features = in_features * 2
        for _ in range(2):
            model += [
                nn.Conv2d(in_features, out_features, 3, stride=2, padding=1),
                nn.InstanceNorm2d(out_features),
                nn.ReLU(inplace=True)
            ]
            in_features = out_features
            out_features = in_features * 2

        # Residual blocks
        for _ in range(num_residual_blocks):
            model += [ResidualBlock(in_features)]

        # Upsampling
        out_features = in_features // 2
        for _ in range(2):
            model += [
                nn.ConvTranspose2d(in_features, out_features, 3, stride=2,
padding=1, output_padding=1),
                nn.InstanceNorm2d(out_features),
                nn.ReLU(inplace=True)
            ]
            in_features = out_features
            out_features = in_features // 2

        # Output layer
        model += [
            nn.ReflectionPad2d(3),
            nn.Conv2d(64, input_channels, 7),
            nn.Tanh()
        ]

        self.model = nn.Sequential(*model)

    def forward(self, x):
        return self.model(x)


class CycleGAN_Discriminator(nn.Module):
    """
    CycleGAN Discriminator (PatchGAN)
    """
    def __init__(self, input_channels=3):
        super(CycleGAN_Discriminator, self).__init__()

        def discriminator_block(in_filters, out_filters, normalize=True):
            """Discriminator block"""
            layers = [nn.Conv2d(in_filters, out_filters, 4, stride=2, padding=1)]
            if normalize:
                layers.append(nn.InstanceNorm2d(out_filters))
            layers.append(nn.LeakyReLU(0.2, inplace=True))
            return layers
```

```
            self.model = nn.Sequential(
                *discriminator_block(input_channels, 64, normalize=False),
                *discriminator_block(64, 128),
                *discriminator_block(128, 256),
                *discriminator_block(256, 512),
                nn.ZeroPad2d((1, 0, 1, 0)),
                nn.Conv2d(512, 1, 4, padding=1)
            )

        def forward(self, img):
            return self.model(img)


    class CycleGANTrainer:
        """
        CycleGAN Training with Cycle Consistency Loss
        """
        def __init__(self, generator_AB, generator_BA, discriminator_A,
    discriminator_B, device='cuda'):
            """
            Args:
                generator_AB: Transforms A → B (e.g., Horse → Zebra)
                generator_BA: Transforms B → A (e.g., Zebra → Horse)
                discriminator_A: Discriminates domain A
                discriminator_B: Discriminates domain B
            """
            self.G_AB = generator_AB.to(device)
            self.G_BA = generator_BA.to(device)
            self.D_A = discriminator_A.to(device)
            self.D_B = discriminator_B.to(device)
            self.device = device

            # Losses
            self.criterion_GAN = nn.MSELoss()
            self.criterion_cycle = nn.L1Loss()
            self.criterion_identity = nn.L1Loss()

            # Optimizers
            self.optimizer_G = torch.optim.Adam(
                list(self.G_AB.parameters()) + list(self.G_BA.parameters()),
                lr=0.0002, betas=(0.5, 0.999)
            )
            self.optimizer_D_A = torch.optim.Adam(self.D_A.parameters(), lr=0.0002,
    betas=(0.5, 0.999))
            self.optimizer_D_B = torch.optim.Adam(self.D_B.parameters(), lr=0.0002,
    betas=(0.5, 0.999))

        def train(self, dataloader_A, dataloader_B, epochs=100, lambda_cyc=10.0,
    lambda_id=5.0):
            """
            Train CycleGAN

            Args:
                dataloader_A: DataLoader for domain A images
```

```
            dataloader_B: DataLoader for domain B images
            epochs: Number of training epochs
            lambda_cyc: Weight for cycle consistency loss
            lambda_id: Weight for identity loss
        """
        from itertools import cycle

        for epoch in range(epochs):
            for i, (batch_A, batch_B) in enumerate(zip(dataloader_A,
cycle(dataloader_B))):
                # Move to device
                real_A = batch_A.to(self.device)
                real_B = batch_B.to(self.device)

                batch_size = real_A.size(0)

                # Real and fake labels
                valid = torch.ones(batch_size, 1, 16, 16).to(self.device)  #
PatchGAN output size
                fake = torch.zeros(batch_size, 1, 16, 16).to(self.device)

                # ------------------
                #  Train Generators
                # ------------------
                self.G_AB.train()
                self.G_BA.train()

                self.optimizer_G.zero_grad()

                # Identity loss
                loss_id_A = self.criterion_identity(self.G_BA(real_A), real_A)
                loss_id_B = self.criterion_identity(self.G_AB(real_B), real_B)
                loss_identity = (loss_id_A + loss_id_B) / 2

                # GAN loss
                fake_B = self.G_AB(real_A)
                loss_GAN_AB = self.criterion_GAN(self.D_B(fake_B), valid)

                fake_A = self.G_BA(real_B)
                loss_GAN_BA = self.criterion_GAN(self.D_A(fake_A), valid)

                loss_GAN = (loss_GAN_AB + loss_GAN_BA) / 2

                # Cycle consistency loss
                recovered_A = self.G_BA(fake_B)
                loss_cycle_A = self.criterion_cycle(recovered_A, real_A)

                recovered_B = self.G_AB(fake_A)
                loss_cycle_B = self.criterion_cycle(recovered_B, real_B)

                loss_cycle = (loss_cycle_A + loss_cycle_B) / 2

                # Total generator loss
                loss_G = loss_GAN + lambda_cyc * loss_cycle + lambda_id *
```

```
    loss_identity

                loss_G.backward()
                self.optimizer_G.step()

                # -----------------------
                #  Train Discriminator A
                # -----------------------
                self.optimizer_D_A.zero_grad()

                loss_real_A = self.criterion_GAN(self.D_A(real_A), valid)
                loss_fake_A = self.criterion_GAN(self.D_A(fake_A.detach()), fake)
                loss_D_A = (loss_real_A + loss_fake_A) / 2

                loss_D_A.backward()
                self.optimizer_D_A.step()

                # -----------------------
                #  Train Discriminator B
                # -----------------------
                self.optimizer_D_B.zero_grad()

                loss_real_B = self.criterion_GAN(self.D_B(real_B), valid)
                loss_fake_B = self.criterion_GAN(self.D_B(fake_B.detach()), fake)
                loss_D_B = (loss_real_B + loss_fake_B) / 2

                loss_D_B.backward()
                self.optimizer_D_B.step()

                if i % 100 == 0:
                    print(f"[Epoch {epoch}/{epochs}] [Batch {i}] "
                          f"[D loss: {(loss_D_A + loss_D_B).item():.4f}] "
                          f"[G loss: {loss_G.item():.4f}] "
                          f"[Cycle: {loss_cycle.item():.4f}]")

        print("✓ CycleGAN training complete!")
        return self.G_AB, self.G_BA


# Example usage
if __name__ == "__main__":
    # Create models
    G_AB = CycleGAN_Generator(input_channels=3, num_residual_blocks=9)  # A → B
    G_BA = CycleGAN_Generator(input_channels=3, num_residual_blocks=9)  # B → A
    D_A = CycleGAN_Discriminator(input_channels=3)
    D_B = CycleGAN_Discriminator(input_channels=3)

    # Train
    trainer = CycleGANTrainer(G_AB, G_BA, D_A, D_B, device='cuda')

    # Load data (horse2zebra dataset from public sources)
    # trainer.train(horse_loader, zebra_loader, epochs=200)
```

Public Datasets for CycleGAN:

- Horse2Zebra: Available from UC Berkeley's CycleGAN project
- Summer2Winter Yosemite: Available from academic sources
- Monet2Photo: Available from the original CycleGAN paper

4.5.3 Pix2Pix - Paired Image Translation

```
"""
Pix2Pix - Image-to-Image Translation with Paired Data
Example: Edges → Photo, Sketch → Color, Satellite → Map
"""

import torch
import torch.nn as nn

class UNetBlock(nn.Module):
    """U-Net encoder/decoder block"""
    def __init__(self, in_size, out_size, normalize=True, dropout=0.0):
        super(UNetBlock, self).__init__()
        layers = [nn.Conv2d(in_size, out_size, 4, 2, 1, bias=False)]
        if normalize:
            layers.append(nn.BatchNorm2d(out_size))
        layers.append(nn.LeakyReLU(0.2))
        if dropout:
            layers.append(nn.Dropout(dropout))
        self.model = nn.Sequential(*layers)

    def forward(self, x):
        return self.model(x)


class Pix2Pix_Generator(nn.Module):
    """
    U-Net Generator for Pix2Pix
    """
    def __init__(self, input_channels=3, output_channels=3):
        super(Pix2Pix_Generator, self).__init__()

        # Encoder (downsampling)
        self.down1 = nn.Conv2d(input_channels, 64, 4, 2, 1)
        self.down2 = UNetBlock(64, 128)
        self.down3 = UNetBlock(128, 256)
        self.down4 = UNetBlock(256, 512, dropout=0.5)
        self.down5 = UNetBlock(512, 512, dropout=0.5)
        self.down6 = UNetBlock(512, 512, dropout=0.5)
        self.down7 = UNetBlock(512, 512, dropout=0.5)
        self.down8 = UNetBlock(512, 512, normalize=False, dropout=0.5)

        # Decoder (upsampling)
        self.up1 = nn.ConvTranspose2d(512, 512, 4, 2, 1)
```

```
        self.up2 = nn.ConvTranspose2d(1024, 512, 4, 2, 1)
        self.up3 = nn.ConvTranspose2d(1024, 512, 4, 2, 1)
        self.up4 = nn.ConvTranspose2d(1024, 512, 4, 2, 1)
        self.up5 = nn.ConvTranspose2d(1024, 256, 4, 2, 1)
        self.up6 = nn.ConvTranspose2d(512, 128, 4, 2, 1)
        self.up7 = nn.ConvTranspose2d(256, 64, 4, 2, 1)
        self.up8 = nn.ConvTranspose2d(128, output_channels, 4, 2, 1)

        self.norm = nn.BatchNorm2d(512)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.5)
        self.tanh = nn.Tanh()

    def forward(self, x):
        # Encoder
        d1 = self.down1(x)
        d2 = self.down2(d1)
        d3 = self.down3(d2)
        d4 = self.down4(d3)
        d5 = self.down5(d4)
        d6 = self.down6(d5)
        d7 = self.down7(d6)
        d8 = self.down8(d7)

        # Decoder with skip connections
        u1 = self.dropout(self.norm(self.relu(self.up1(d8))))
        u1 = torch.cat([u1, d7], 1)

        u2 = self.dropout(self.norm(self.relu(self.up2(u1))))
        u2 = torch.cat([u2, d6], 1)

        u3 = self.dropout(self.norm(self.relu(self.up3(u2))))
        u3 = torch.cat([u3, d5], 1)

        u4 = self.norm(self.relu(self.up4(u3)))
        u4 = torch.cat([u4, d4], 1)

        u5 = self.norm(self.relu(self.up5(u4)))
        u5 = torch.cat([u5, d3], 1)

        u6 = self.norm(self.relu(self.up6(u5)))
        u6 = torch.cat([u6, d2], 1)

        u7 = self.norm(self.relu(self.up7(u6)))
        u7 = torch.cat([u7, d1], 1)

        u8 = self.up8(u7)

        return self.tanh(u8)


class Pix2PixTrainer:
    """
    Pix2Pix Training with L1 + GAN Loss
```

```
    """
    def __init__(self, generator, discriminator, device='cuda'):
        self.generator = generator.to(device)
        self.discriminator = discriminator.to(device)
        self.device = device

        # Losses
        self.criterion_GAN = nn.MSELoss()
        self.criterion_pixelwise = nn.L1Loss()

        # Optimizers
        self.optimizer_G = torch.optim.Adam(generator.parameters(), lr=0.0002,
betas=(0.5, 0.999))
        self.optimizer_D = torch.optim.Adam(discriminator.parameters(), lr=0.0002,
betas=(0.5, 0.999))

    def train(self, dataloader, epochs=100, lambda_pixel=100):
        """
        Train Pix2Pix

        Args:
            dataloader: Returns (input_image, target_image) pairs
            lambda_pixel: Weight for L1 loss (encourages realistic details)
        """
        for epoch in range(epochs):
            for i, (input_imgs, target_imgs) in enumerate(dataloader):
                input_imgs = input_imgs.to(self.device)
                target_imgs = target_imgs.to(self.device)

                batch_size = input_imgs.size(0)

                # Labels
                valid = torch.ones(batch_size, 1, 30, 30).to(self.device)  #
PatchGAN
                fake = torch.zeros(batch_size, 1, 30, 30).to(self.device)

                # ------------------
                #  Train Generator
                # ------------------
                self.optimizer_G.zero_grad()

                # Generate fake images
                fake_imgs = self.generator(input_imgs)

                # GAN loss
                pred_fake = self.discriminator(fake_imgs, input_imgs)
                loss_GAN = self.criterion_GAN(pred_fake, valid)

                # Pixel-wise loss
                loss_pixel = self.criterion_pixelwise(fake_imgs, target_imgs)

                # Total generator loss
                loss_G = loss_GAN + lambda_pixel * loss_pixel
```

```
                loss_G.backward()
                self.optimizer_G.step()

                # ----------------------
                #  Train Discriminator
                # ----------------------
                self.optimizer_D.zero_grad()

                # Real images
                pred_real = self.discriminator(target_imgs, input_imgs)
                loss_real = self.criterion_GAN(pred_real, valid)

                # Fake images
                pred_fake = self.discriminator(fake_imgs.detach(), input_imgs)
                loss_fake = self.criterion_GAN(pred_fake, fake)

                # Total discriminator loss
                loss_D = (loss_real + loss_fake) / 2

                loss_D.backward()
                self.optimizer_D.step()

                if i % 100 == 0:
                    print(f"[Epoch {epoch}/{epochs}] [Batch {i}] "
                          f"[D loss: {loss_D.item():.4f}] "
                          f"[G loss: {loss_G.item():.4f}] "
                          f"[Pixel loss: {loss_pixel.item():.4f}]")

        print("✓ Pix2Pix training complete!")
        return self.generator
```

Public Datasets for Pix2Pix:

- Edges2Shoes: Available from UC Berkeley
- Facades: CMP Facade Database
- Maps (Satellite→Map): Available from the original Pix2Pix paper

---

4.5.4 StyleGAN & StyleGAN2

```
"""
StyleGAN2 - High-Quality Face Generation
Note: StyleGAN2 is complex. Here's a simplified architecture overview.
For production, use: https://github.com/NVlabs/stylegan2-ada-pytorch
"""

import torch
import torch.nn as nn
import torch.nn.functional as F

class MappingNetwork(nn.Module):
```

```
    """
    Maps latent code z → intermediate latent w
    """
    def __init__(self, latent_dim=512, mapping_layers=8):
        super(MappingNetwork, self).__init__()

        layers = []
        for i in range(mapping_layers):
            layers.append(nn.Linear(latent_dim, latent_dim))
            layers.append(nn.LeakyReLU(0.2))

        self.mapping = nn.Sequential(*layers)

    def forward(self, z):
        """
        Args:
            z: (batch_size, latent_dim) random noise
        Returns:
            w: (batch_size, latent_dim) disentangled latent
        """
        return self.mapping(z)


class StyleBlock(nn.Module):
    """
    Style-based synthesis block (AdaIN)
    """
    def __init__(self, in_channels, out_channels, latent_dim=512):
        super(StyleBlock, self).__init__()

        self.conv = nn.Conv2d(in_channels, out_channels, 3, padding=1)
        self.noise = nn.Parameter(torch.randn(1, 1, 1, 1))

        # Style modulation (Adaptive Instance Normalization)
        self.style_scale = nn.Linear(latent_dim, out_channels)
        self.style_shift = nn.Linear(latent_dim, out_channels)

        self.activation = nn.LeakyReLU(0.2)

    def forward(self, x, w):
        """
        Args:
            x: (batch, in_channels, H, W)
            w: (batch, latent_dim) style vector
        """
        # Convolution
        x = self.conv(x)

        # Add noise
        noise = self.noise.expand(x.size(0), 1, x.size(2), x.size(3))
        x = x + noise

        # Instance normalization
        x_mean = x.mean(dim=[2, 3], keepdim=True)
```

```
        x_std = x.std(dim=[2, 3], keepdim=True) + 1e-8
        x = (x - x_mean) / x_std

        # Apply style (AdaIN)
        style_scale = self.style_scale(w).unsqueeze(2).unsqueeze(3)
        style_shift = self.style_shift(w).unsqueeze(2).unsqueeze(3)

        x = x * style_scale + style_shift

        x = self.activation(x)

        return x


class StyleGAN2_Generator_Simplified(nn.Module):
    """
    Simplified StyleGAN2 Generator
    Real StyleGAN2 has progressive growth, skip connections, etc.
    """
    def __init__(self, latent_dim=512, img_channels=3):
        super(StyleGAN2_Generator_Simplified, self).__init__()

        self.latent_dim = latent_dim

        # Mapping network
        self.mapping = MappingNetwork(latent_dim)

        # Constant input (learned)
        self.const_input = nn.Parameter(torch.randn(1, 512, 4, 4))

        # Style blocks (progressive)
        self.style_blocks = nn.ModuleList([
            StyleBlock(512, 512, latent_dim),  # 4x4
            StyleBlock(512, 512, latent_dim),  # 8x8
            StyleBlock(512, 256, latent_dim),  # 16x16
            StyleBlock(256, 128, latent_dim),  # 32x32
            StyleBlock(128, 64, latent_dim),   # 64x64
            StyleBlock(64, 32, latent_dim),    # 128x128
        ])

        # RGB output layers
        self.to_rgb = nn.ModuleList([
            nn.Conv2d(512, img_channels, 1) for _ in range(6)
        ])

        self.upsample = nn.Upsample(scale_factor=2, mode='bilinear',
align_corners=False)

    def forward(self, z):
        """
        Args:
            z: (batch_size, latent_dim) random noise
        Returns:
            img: (batch_size, img_channels, 128, 128)
```

```
        """
        # Map to intermediate latent
        w = self.mapping(z)

        # Start with constant input
        x = self.const_input.expand(z.size(0), -1, -1, -1)

        # Progressive synthesis
        for i, style_block in enumerate(self.style_blocks):
            x = style_block(x, w)
            if i < len(self.style_blocks) - 1:
                x = self.upsample(x)

        # To RGB
        img = self.to_rgb[-1](x)
        img = torch.tanh(img)

        return img


# Example: Pre-trained StyleGAN2 usage
def use_pretrained_stylegan2():
    """
    Use official pre-trained StyleGAN2 from NVIDIA
    """
    import pickle
    import numpy as np

    # Download pre-trained model
    # wget https://nvlabs-fi-cdn.nvidia.com/stylegan2-ada-
pytorch/pretrained/ffhq.pkl

    with open('ffhq.pkl', 'rb') as f:
        G = pickle.load(f)['G_ema'].cuda()  # Generator

    # Generate random face
    z = torch.randn([1, G.z_dim]).cuda()
    w = G.mapping(z, None)
    img = G.synthesis(w, noise_mode='const')

    print(f"Generated image shape: {img.shape}")

    return img


# Style mixing example
def style_mixing(G, z1, z2, mix_layer=4):
    """
    Mix styles from two latent codes
    Creates interesting hybrids (e.g., face structure from z1, colors from z2)
    """
    w1 = G.mapping(z1)
    w2 = G.mapping(z2)
```

```
    # Use w1 for early layers, w2 for later layers
    w_mix = w1.clone()
    w_mix[:, mix_layer:] = w2[:, mix_layer:]

    img = G.synthesis(w_mix)
    return img
```

Training StyleGAN2 on Cloud:

AWS SageMaker:

```
# Instance: ml.p3.8xlarge (4x V100 GPUs)
# Cost: ~$14/hour
# Training time: 2-3 days for FFHQ (1024×1024)

# Upload dataset to S3
aws s3 sync ./ffhq_dataset s3://my-bucket/stylegan2/data/

# Start training job
python train.py \
    --outdir=s3://my-bucket/stylegan2/results \
    --data=s3://my-bucket/stylegan2/data/ffhq \
    --gpus=4 \
    --kimg=25000
```

GCP Vertex AI:

```
# Instance: n1-standard-16 + 4x V100
# Cost: ~$8-12/hour
# Training time: Similar to AWS

gcloud ai custom-jobs create \
    --region=us-central1 \
    --display-name=stylegan2-training \
    --worker-pool-spec=machine-type=n1-standard-16,replica-count=1,accelerator-
type=NVIDIA_TESLA_V100,accelerator-count=4 \
    --python-package-uris=gs://my-bucket/stylegan2_trainer.tar.gz \
    --args=--outdir=gs://my-bucket/results,--data=gs://my-bucket/ffhq
```

Azure ML:

```
# Azure ML training
from azureml.core import Workspace, Experiment, ScriptRunConfig
from azureml.core.compute import ComputeTarget

ws = Workspace.from_config()
```

```
# Use NC24s_v3 (4x V100)
compute_target = ComputeTarget(workspace=ws, name='gpu-cluster')

config = ScriptRunConfig(
    source_directory='./stylegan2',
    script='train.py',
    arguments=['--outdir', './results', '--gpus', 4],
    compute_target=compute_target
)

experiment = Experiment(ws, 'stylegan2-training')
run = experiment.submit(config)
```

4.6 GAN Training on Cloud Providers

Comparison Table - GAN Training Costs

Provider	Instance Type	GPUs	RAM	Cost/Hour	Best For
AWS	ml.p3.2xlarge	1x V100	61 GB	$3.06	Basic GAN, DCGAN
AWS	ml.p3.8xlarge	4x V100	244 GB	$12.24	StyleGAN2, BigGAN
AWS	ml.p4d.24xlarge	8x A100	1152 GB	$32.77	StyleGAN3, large-scale
GCP	n1-standard-8 + 1x V100	1x V100	30 GB	$2.48	Basic GAN
GCP	a2-highgpu-4g	4x A100	680 GB	$14.73	StyleGAN2
Azure	Standard_NC6s_v3	1x V100	112 GB	$3.06	Basic GAN
Azure	Standard_ND96asr_v4	8x A100	900 GB	$27.20	Large-scale GANs

Note: Costs are approximate and subject to change. Verify current pricing with cloud providers.

Training GANs on AWS SageMaker

```
"""
Train GANs on AWS SageMaker
"""

import sagemaker
from sagemaker.pytorch import PyTorch

def train_gan_sagemaker(train_data_s3, gan_type='dcgan'):
    """
    Train GAN on AWS SageMaker

    Args:
        train_data_s3: S3 path to training images
```

```
        gan_type: 'dcgan', 'stylegan2', 'cyclegan', 'pix2pix'
    """
    session = sagemaker.Session()
    role = sagemaker.get_execution_role()

    # PyTorch estimator
    estimator = PyTorch(
        entry_point='train_gan.py',
        role=role,
        instance_type='ml.p3.2xlarge',  # 1x V100
        instance_count=1,
        framework_version='2.0',
        py_version='py310',
        hyperparameters={
            'gan-type': gan_type,
            'epochs': 100,
            'batch-size': 64,
            'latent-dim': 128
        },
        use_spot_instances=True,
        max_run=3600 * 12,  # 12 hours
        max_wait=3600 * 24   # 24 hours
    )

    # Start training
    estimator.fit({'train': train_data_s3})

    print(f"✓ GAN training complete: {estimator.model_data}")

    return estimator
```

Training GANs on GCP Vertex AI

```
"""
Train GANs on GCP Vertex AI
"""

from google.cloud import aiplatform

def train_gan_vertex_ai(dataset_uri, gan_type='dcgan'):
    """
    Train GAN on GCP Vertex AI
    """
    aiplatform.init(project='my-project', location='us-central1')

    # Custom training job
    job = aiplatform.CustomContainerTrainingJob(
        display_name=f'{gan_type}-training',
        container_uri='gcr.io/my-project/gan-trainer:latest'
    )
```

```
    # Run training
    model = job.run(
        dataset=dataset_uri,
        replica_count=1,
        machine_type='n1-standard-8',
        accelerator_type='NVIDIA_TESLA_V100',
        accelerator_count=1,
        args=[
            f'--gan-type={gan_type}',
            '--epochs=100',
            '--batch-size=64'
        ]
    )

    print(f"✓ GAN training complete")

    return model
```

Training GANs on Azure ML

```
"""
Train GANs on Azure ML
"""

from azureml.core import Workspace, Experiment, ScriptRunConfig
from azureml.core.compute import AmlCompute

def train_gan_azure_ml(ws, dataset_path, gan_type='dcgan'):
    """
    Train GAN on Azure ML
    """
    # Get/create compute target
    compute_target = AmlCompute(ws, 'gpu-cluster')

    # Script config
    config = ScriptRunConfig(
        source_directory='./gan_training',
        script='train_gan.py',
        arguments=[
            '--data-path', dataset_path,
            '--gan-type', gan_type,
            '--epochs', 100,
            '--batch-size', 64
        ],
        compute_target=compute_target
    )

    # Submit experiment
    experiment = Experiment(ws, f'{gan_type}-training')
```

```
    run = experiment.submit(config)

    run.wait_for_completion(show_output=True)

    print("✓ GAN training complete")

    return run
```

4.7 Computer Vision & GAN Applications

Use Cases by GAN Type

GAN Type	Use Case	Public Dataset Sources
Basic GAN	Simple image generation	MNIST, Fashion-MNIST
DCGAN	Face generation, texture synthesis	CelebA, LSUN Bedrooms
Conditional GAN	Class-specific generation	CIFAR-10, ImageNet
StyleGAN2	High-quality faces, art	FFHQ, MetFaces, AFHQ
CycleGAN	Style transfer, domain adaptation	Horse2Zebra, Apple2Orange
Pix2Pix	Image colorization, super-resolution	Edges2Shoes, Maps, Facades
Progressive GAN	High-resolution generation	CelebA-HQ

Popular Public Datasets:

- **FFHQ (Flickr-Faces-HQ):** 70,000 high-quality face images
- **CelebA:** 200,000+ celebrity face images
- **LSUN Bedrooms:** Large-scale scene understanding benchmark
- **MetFaces:** Artistic face paintings from The Met museum

Part 5: Cross-Cloud Comparison & Best Practices

5.1 Cross-Cloud Comparison

Service Comparison Table

Feature	AWS	GCP	Azure
ML Platform	SageMaker	Vertex AI	Azure ML
LLM Service	Bedrock	Vertex AI (PaLM/Gemini)	Azure OpenAI
AutoML	SageMaker Autopilot	Vertex AI AutoML	Azure AutoML
GPU Instances	P3, P4, P5	A2, N1 + GPU	NC, ND, NV series

Feature	AWS	GCP	Azure
Custom AI Chips	Trainium, Inferentia	TPU v2-v5	None
Spot/Preemptible	Spot Instances	Preemptible VMs	Spot VMs
Notebooks	SageMaker Studio	Vertex AI Workbench	Azure ML Studio
Model Registry	SageMaker Registry	Vertex AI Model Registry	Azure ML Registry
MLOps	SageMaker Pipelines	Vertex AI Pipelines	Azure ML Pipelines
Edge Deployment	SageMaker Edge	TensorFlow Lite	Azure IoT Edge

---

Cost Comparison - Training Instances

> **Important:** The pricing information below serves as a general comparison guide based on available data at the time of publication. Cloud providers frequently update their pricing. For accurate, up-to-date costs:
>
> - **AWS:** https://aws.amazon.com/sagemaker/pricing/
> - **GCP:** https://cloud.google.com/vertex-ai/pricing
> - **Azure:** https://azure.microsoft.com/en-us/pricing/details/machine-learning/

GPU Training (per hour)

Instance Type	GPUs	vCPUs	RAM	AWS	GCP	Azure
Single V100	1x V100 (16GB)	8	61 GB	$3.06	$2.48	$3.06
Quad V100	4x V100	32	244 GB	$12.24	$9.20	$12.24
Single A100	1x A100 (40GB)	12	85 GB	$4.10	$3.67	$3.67
Quad A100	4x A100	48	680 GB	$16.40	$14.73	$14.40
Octa A100	8x A100	96	1152 GB	$32.77	$28.47	$27.20

Note: Costs per hour are approximate and subject to change. Verify current pricing with cloud providers.

TPU Training (GCP only)

TPU Type	TPU Cores	Memory	Cost/Hour
TPU v2-8	8 cores	64 GB	$4.50
TPU v3-8	8 cores	128 GB	$8.00
TPU v4-8	8 cores	256 GB	$11.00
TPU v5-8	8 cores	256 GB	$12.00

Note: TPU costs are approximate and subject to change. Available only on GCP.

Cloud Provider Selection - Practical Considerations

> **Reality Check:** In most cases, your cloud provider is already chosen based on your organization's existing infrastructure, contracts, and IT policies. This section helps you understand the ML/LLM-specific strengths of each platform when you're working within those constraints.

If You're Already on AWS:

- ✅ Use **SageMaker** for comprehensive ML platform
- ✅ Access **Amazon Bedrock** for foundation models (Claude, Llama, Titan)
- ✅ Leverage **Trainium/Inferentia** for custom AI accelerators
- ✅ Deploy on **Lambda/ECS/EKS** for serverless or container workloads
- ✅ Benefit from wide range of GPU instance types (P3, P4, P5)

If You're Already on GCP:

- ✅ Use **TPUs** for cost-effective training of large models
- ✅ Access **Vertex AI** for unified ML platform
- ✅ Leverage **PaLM 2/Gemini** foundation models
- ✅ Use **TensorFlow/JAX** with native optimization
- ✅ Benefit from **BigQuery ML** for SQL-based machine learning
- ✅ Typically **10-15% cheaper GPU costs** than competitors

If You're Already on Azure:

- ✅ Use **Azure OpenAI Service** for GPT-4, GPT-3.5 access
- ✅ Integrate with **Microsoft 365, Power BI, Dynamics**
- ✅ Leverage **Azure ML** for enterprise-grade platform
- ✅ Use **Azure Active Directory** for authentication/authorization
- ✅ Build with **.NET/C#** ML.NET framework if preferred

If You're Choosing a Cloud Provider (Greenfield Projects):

Consider AWS if:

- You need the most mature ML ecosystem with broadest service coverage
- You want access to Amazon Bedrock's model marketplace

Consider GCP if:

- You need TPUs for training large models (most cost-effective)
- You prefer TensorFlow/JAX ecosystem
- You want lowest GPU costs

Consider Azure if:

- You need GPT-4/GPT-3.5 via Azure OpenAI Service
- Your organization is Microsoft-centric
- You require strong enterprise compliance features

Multi-Cloud Deployment Strategy

```
"""
Deploy ML models across multiple clouds
"""

class MultiCloudDeployer:
    """
    Deploy to AWS, GCP, and Azure simultaneously
    """
    def __init__(self, model_path):
        self.model_path = model_path

    def deploy_aws(self, endpoint_name='ml-endpoint'):
        """Deploy to AWS SageMaker"""
        import sagemaker
        from sagemaker.pytorch import PyTorchModel

        role = sagemaker.get_execution_role()

        model = PyTorchModel(
            model_data=self.model_path,
            role=role,
            framework_version='2.0',
            py_version='py310',
            entry_point='inference.py'
        )

        predictor = model.deploy(
            initial_instance_count=1,
            instance_type='ml.g4dn.xlarge',
            endpoint_name=endpoint_name
        )

        print(f"✓ AWS deployment: {endpoint_name}")
        return predictor

    def deploy_gcp(self, endpoint_name='ml-endpoint'):
        """Deploy to GCP Vertex AI"""
        from google.cloud import aiplatform

        aiplatform.init(project='my-project', location='us-central1')

        model = aiplatform.Model.upload(
            display_name=endpoint_name,
            artifact_uri=self.model_path,
            serving_container_image_uri='gcr.io/my-project/predictor:latest'
        )

        endpoint = model.deploy(
            machine_type='n1-standard-4',
            accelerator_type='NVIDIA_TESLA_T4',
```

```
            accelerator_count=1
        )

        print(f"✓ GCP deployment: {endpoint.resource_name}")
        return endpoint

    def deploy_azure(self, endpoint_name='ml-endpoint'):
        """Deploy to Azure ML"""
        from azureml.core import Workspace, Model
        from azureml.core.model import InferenceConfig
        from azureml.core.webservice import AciWebservice

        ws = Workspace.from_config()

        model = Model.register(
            workspace=ws,
            model_path=self.model_path,
            model_name=endpoint_name
        )

        inference_config = InferenceConfig(
            entry_script='score.py',
            environment=ws.environments['my-env']
        )

        deployment_config = AciWebservice.deploy_configuration(
            cpu_cores=2,
            memory_gb=4,
            gpu_cores=1
        )

        service = Model.deploy(
            workspace=ws,
            name=endpoint_name,
            models=[model],
            inference_config=inference_config,
            deployment_config=deployment_config
        )

        service.wait_for_deployment(show_output=True)

        print(f"✓ Azure deployment: {service.scoring_uri}")
        return service

    def deploy_all(self, base_name='ml-model'):
        """Deploy to all three clouds"""
        print("Deploying to all clouds...")

        aws_endpoint = self.deploy_aws(f'{base_name}-aws')
        gcp_endpoint = self.deploy_gcp(f'{base_name}-gcp')
        azure_endpoint = self.deploy_azure(f'{base_name}-azure')

        print("\n✓ Multi-cloud deployment complete!")
```

```
        return {
            'aws': aws_endpoint,
            'gcp': gcp_endpoint,
            'azure': azure_endpoint
        }


# Example usage
deployer = MultiCloudDeployer(model_path='s3://my-bucket/model.tar.gz')
endpoints = deployer.deploy_all(base_name='recommendation-model')
```

5.3 Cost Optimization Strategies

> **Pricing Note:** Cost examples and savings percentages represent typical scenarios based on available information at publication time. Actual savings may vary based on current cloud provider rates, region, commitment levels, and promotional offers. Always calculate costs using current pricing calculators.

1. Use Spot/Preemptible Instances

```
"""
Cost savings: 60-90% discount
"""

# AWS Spot Instances
estimator = PyTorch(
    entry_point='train.py',
    role=role,
    instance_type='ml.p3.2xlarge',
    instance_count=1,
    use_spot_instances=True,  # ← 70% discount
    max_run=3600,
    max_wait=7200
)

# GCP Preemptible VMs
job = aiplatform.CustomContainerTrainingJob(
    display_name='training-job',
    container_uri='gcr.io/my-project/trainer:latest'
)

model = job.run(
    replica_count=1,
    machine_type='n1-standard-8',
    accelerator_type='NVIDIA_TESLA_V100',
    accelerator_count=1,
    base_output_dir='gs://my-bucket/output',
    preemptible=True  # ← 80% discount
)

# Azure Spot VMs
```

```
config = ScriptRunConfig(
    source_directory='./training',
    script='train.py',
    compute_target='gpu-cluster'
)

# Configure cluster with Spot VMs
from azureml.core.compute import AmlCompute

compute_config = AmlCompute.provisioning_configuration(
    vm_size='Standard_NC6s_v3',
    min_nodes=0,
    max_nodes=4,
    vm_priority='lowpriority'  # ← Spot VMs
)
```

Savings Example:

- Regular V100: $3.06/hour
- Spot V100: $0.92/hour (70% off)
- **100 hours training: Save $214**

---

2. Choose Right Instance Size

Training Task	GPU Needed	Instance	Cost/Hour
Small model (<100M params)	T4 (16GB)	g4dn.xlarge	$0.526
Medium model (100M-1B)	V100 (16GB)	p3.2xlarge	$3.06
Large model (1B-10B)	A100 (40GB)	p4d.24xlarge	$32.77
Huge model (>10B)	8x A100 (80GB)	p5.48xlarge	$98.32

Note: Costs are approximate and subject to change.

Tip: Start with smaller instance, scale up only if OOM

---

3. Mixed Precision Training

```
"""
Use FP16 instead of FP32
→ 2x faster training
→ 2x less memory
→ Same accuracy
"""

# PyTorch with Automatic Mixed Precision
from torch.cuda.amp import autocast, GradScaler
```

```
scaler = GradScaler()

for epoch in range(epochs):
    for inputs, labels in dataloader:
        optimizer.zero_grad()

        # Mixed precision forward pass
        with autocast():
            outputs = model(inputs)
            loss = criterion(outputs, labels)

        # Scaled backward pass
        scaler.scale(loss).backward()
        scaler.step(optimizer)
        scaler.update()

print("✓ Training with FP16 - 2x faster!")
```

Savings:

- Without FP16: 10 hours = $30.60 (p3.2xlarge)
- With FP16: 5 hours = $15.30
- **Save: $15.30 (50%)**

---

4. Gradient Accumulation

```
"""
Train large batches on small GPUs
Simulate batch_size=128 on GPU that fits batch_size=32
"""

accumulation_steps = 4  # 32 * 4 = 128 effective batch size

for i, (inputs, labels) in enumerate(dataloader):
    outputs = model(inputs)
    loss = criterion(outputs, labels) / accumulation_steps

    loss.backward()

    # Update every N steps
    if (i + 1) % accumulation_steps == 0:
        optimizer.step()
        optimizer.zero_grad()

print("✓ Large batch training on small GPU - save $$")
```

---

5. Dataset Optimization

```
"""
Optimize data loading → reduce training time → save money
"""

# Efficient DataLoader
from torch.utils.data import DataLoader

dataloader = DataLoader(
    dataset,
    batch_size=64,
    shuffle=True,
    num_workers=4,  # Parallel data loading
    pin_memory=True,  # Faster GPU transfer
    persistent_workers=True,  # Keep workers alive
    prefetch_factor=2  # Pre-load batches
)

# Cache preprocessed data
import functools

@functools.lru_cache(maxsize=10000)
def load_and_preprocess(image_path):
    img = Image.open(image_path)
    return transform(img)
```

---

6. Early Stopping

```
"""
Stop training when validation stops improving
"""

class EarlyStopping:
    def __init__(self, patience=5, min_delta=0.001):
        self.patience = patience
        self.min_delta = min_delta
        self.counter = 0
        self.best_loss = None
        self.early_stop = False

    def __call__(self, val_loss):
        if self.best_loss is None:
            self.best_loss = val_loss
        elif val_loss > self.best_loss - self.min_delta:
            self.counter += 1
            if self.counter >= self.patience:
                self.early_stop = True
        else:
            self.best_loss = val_loss
            self.counter = 0
```

```
        return self.early_stop


# Usage
early_stopping = EarlyStopping(patience=5)

for epoch in range(100):
    train_loss = train_one_epoch()
    val_loss = validate()

    if early_stopping(val_loss):
        print(f"Early stopping at epoch {epoch}")
        break
```

Savings:

- Without early stopping: 100 epochs × 0.5 hours = 50 hours = $153
- With early stopping: 35 epochs × 0.5 hours = 17.5 hours = $53.55
- **Save: $99.45 (65%)**

---

7. Model Distillation

```
"""
Train large model → distill to small model → deploy small model
Inference cost: 10x cheaper
"""

# Step 1: Train large teacher model (expensive)
teacher = LargeModel()  # 1B parameters
train(teacher)  # Cost: $100

# Step 2: Distill to small student model
student = SmallModel()  # 100M parameters

for inputs, labels in dataloader:
    # Teacher predictions (soft labels)
    with torch.no_grad():
        teacher_logits = teacher(inputs)

    # Student learns from teacher
    student_logits = student(inputs)

    # Distillation loss
    loss = distillation_loss(student_logits, teacher_logits, temperature=3.0)
    loss.backward()
    optimizer.step()
```

```
# Step 3: Deploy small model
deploy(student)  # Inference: $0.01/1K requests vs $0.10/1K for teacher
```

Cost Optimization Summary

Strategy	Savings	Trade-off
Spot Instances	60-90%	Can be interrupted
Right-sized instances	30-50%	Need to benchmark
Mixed Precision (FP16)	40-50%	Slight accuracy loss (negligible)
Gradient Accumulation	20-40%	Longer training time
Efficient Data Loading	10-20%	Code complexity
Early Stopping	20-60%	Risk of under training
Model Distillation	80-90% (inference)	Initial training cost

Combined savings: 70-85% total cost reduction

5.4 Production Best Practices

1. Model Versioning

```
"""
Track model versions with MLflow
"""

import mlflow
import mlflow.pytorch

# Start MLflow run
with mlflow.start_run():
    # Log parameters
    mlflow.log_param("learning_rate", 0.001)
    mlflow.log_param("batch_size", 64)
    mlflow.log_param("epochs", 10)

    # Train model
    model = train_model()

    # Log metrics
    mlflow.log_metric("train_accuracy", 0.95)
    mlflow.log_metric("val_accuracy", 0.93)

    # Log model
    mlflow.pytorch.log_model(model, "model")
```

```
    # Tag
    mlflow.set_tag("model_type", "resnet50")
    mlflow.set_tag("dataset", "imagenet")

print("✓ Model version logged to MLflow")
```

---

2. A/B Testing

```
"""
A/B test new model vs production model
"""

class ModelABTester:
    def __init__(self, model_a_endpoint, model_b_endpoint, split=0.5):
        """
        Args:
            model_a_endpoint: Production model
            model_b_endpoint: New model
            split: Traffic split (0.5 = 50/50)
        """
        self.model_a = model_a_endpoint
        self.model_b = model_b_endpoint
        self.split = split
        self.metrics = {'a': [], 'b': []}

    def predict(self, input_data):
        """Route request to A or B"""
        import random

        if random.random() < self.split:
            # Route to model A
            prediction = self.model_a.predict(input_data)
            model_used = 'a'
        else:
            # Route to model B
            prediction = self.model_b.predict(input_data)
            model_used = 'b'

        return prediction, model_used

    def log_metric(self, model_used, metric_value):
        """Log performance metric"""
        self.metrics[model_used].append(metric_value)

    def analyze_results(self):
        """Statistical significance test"""
        from scipy import stats

        a_metrics = self.metrics['a']
        b_metrics = self.metrics['b']
```

```
        # T-test
        t_stat, p_value = stats.ttest_ind(a_metrics, b_metrics)

        print(f"Model A: mean={np.mean(a_metrics):.4f}, n={len(a_metrics)}")
        print(f"Model B: mean={np.mean(b_metrics):.4f}, n={len(b_metrics)}")
        print(f"P-value: {p_value:.4f}")

        if p_value < 0.05:
            if np.mean(b_metrics) > np.mean(a_metrics):
                print("✓ Model B is significantly better - promote to
production")
            else:
                print("Model A is still better - keep in production")
        else:
            print("No significant difference - collect more data")


# Example usage
ab_tester = ModelABTester(
    model_a_endpoint='production-model-v1',
    model_b_endpoint='candidate-model-v2',
    split=0.1  # 10% traffic to new model
)

for input_data in test_inputs:
    prediction, model_used = ab_tester.predict(input_data)
    accuracy = evaluate(prediction, ground_truth)
    ab_tester.log_metric(model_used, accuracy)

ab_tester.analyze_results()
```

---

3. Monitoring & Alerting

```
"""
Monitor model performance in production
"""

import prometheus_client
from prometheus_client import Counter, Histogram, Gauge

# Define metrics
prediction_counter = Counter('predictions_total', 'Total predictions')
prediction_latency = Histogram('prediction_latency_seconds', 'Prediction latency')
model_accuracy = Gauge('model_accuracy', 'Model accuracy')
error_counter = Counter('prediction_errors_total', 'Total errors')

def monitored_predict(model, input_data):
    """Prediction with monitoring"""
    import time
```

```
    try:
        # Measure latency
        start_time = time.time()

        prediction = model.predict(input_data)

        latency = time.time() - start_time

        # Log metrics
        prediction_counter.inc()
        prediction_latency.observe(latency)

        return prediction

    except Exception as e:
        error_counter.inc()
        raise e

# Alert if latency > 1 second
def check_latency_alert():
    avg_latency = prediction_latency._sum.get() / prediction_latency._count.get()

    if avg_latency > 1.0:
        send_alert(f"High latency detected: {avg_latency:.2f}s")

# Alert if accuracy drops
def check_accuracy_alert(current_accuracy, threshold=0.85):
    model_accuracy.set(current_accuracy)

    if current_accuracy < threshold:
        send_alert(f"Accuracy dropped: {current_accuracy:.2%} < {threshold:.2%}")
```

---

4. Canary Deployment

```
"""
Gradual rollout of new model
"""

class CanaryDeployment:
    def __init__(self, old_endpoint, new_endpoint):
        self.old_endpoint = old_endpoint
        self.new_endpoint = new_endpoint
        self.canary_percentage = 0

    def gradual_rollout(self, duration_hours=24, steps=10):
        """
        Gradual increase from 0% to 100% over duration
        """
        import time
```

```
        increment = 100 / steps
        sleep_time = (duration_hours * 3600) / steps

        for step in range(steps + 1):
            self.canary_percentage = step * increment

            print(f"Canary at {self.canary_percentage}% traffic")

            # Monitor for issues
            if self.detect_issues():
                print("❌ Issues detected - rolling back!")
                self.rollback()
                return False

            if step < steps:
                time.sleep(sleep_time)

        print("✓ Canary deployment successful - new model at 100%")
        return True

    def predict(self, input_data):
        """Route to old or new model based on canary percentage"""
        import random

        if random.random() * 100 < self.canary_percentage:
            return self.new_endpoint.predict(input_data)
        else:
            return self.old_endpoint.predict(input_data)

    def detect_issues(self):
        """Check for errors, latency spikes, accuracy drops"""
        # Implement monitoring logic
        error_rate = get_error_rate(self.new_endpoint)
        latency = get_avg_latency(self.new_endpoint)

        return error_rate > 0.05 or latency > 2.0

    def rollback(self):
        """Instant rollback to old model"""
        self.canary_percentage = 0
        print("✓ Rolled back to old model")


# Example
canary = CanaryDeployment(
    old_endpoint='model-v1',
    new_endpoint='model-v2'
)

# Gradual rollout over 24 hours
canary.gradual_rollout(duration_hours=24, steps=10)
```

5. Model Caching

```
"""
Cache predictions for common inputs
"""

from functools import lru_cache
import hashlib
import json

class CachedPredictor:
    def __init__(self, model, cache_size=10000):
        self.model = model
        self.cache = {}
        self.cache_size = cache_size
        self.hits = 0
        self.misses = 0

    def _hash_input(self, input_data):
        """Create hash of input"""
        input_str = json.dumps(input_data, sort_keys=True)
        return hashlib.md5(input_str.encode()).hexdigest()

    def predict(self, input_data):
        """Predict with caching"""
        input_hash = self._hash_input(input_data)

        # Check cache
        if input_hash in self.cache:
            self.hits += 1
            return self.cache[input_hash]

        # Cache miss - run model
        self.misses += 1
        prediction = self.model.predict(input_data)

        # Store in cache
        if len(self.cache) >= self.cache_size:
            # Remove oldest entry (FIFO)
            self.cache.pop(next(iter(self.cache)))

        self.cache[input_hash] = prediction

        return prediction

    def get_cache_stats(self):
        """Cache hit rate"""
        total = self.hits + self.misses
        hit_rate = self.hits / total if total > 0 else 0

        print(f"Cache Stats:")
        print(f"  Hits: {self.hits}")
```

```
        print(f"  Misses: {self.misses}")
        print(f"  Hit Rate: {hit_rate:.2%}")

        return hit_rate


# Example
cached_predictor = CachedPredictor(model, cache_size=10000)

for input_data in requests:
    prediction = cached_predictor.predict(input_data)

cached_predictor.get_cache_stats()
# Output: Cache Hit Rate: 73% → 73% fewer model calls!
```

Production Checklist

```
## Pre-Production Checklist

### Model Quality
- [ ] Accuracy/metrics meet requirements
- [ ] Tested on diverse inputs
- [ ] Bias/fairness evaluation complete
- [ ] Safety testing done (adversarial examples)

### Performance
- [ ] Latency < 100ms (or requirement)
- [ ] Throughput tested (requests/second)
- [ ] Memory usage profiled
- [ ] GPU utilization optimized

### Reliability
- [ ] Error handling implemented
- [ ] Graceful degradation for edge cases
- [ ] Timeout handling
- [ ] Retry logic for transient failures

### Monitoring
- [ ] Metrics logged (latency, accuracy, errors)
- [ ] Alerts configured
- [ ] Dashboards created
- [ ] On-call runbook documented

### Security
- [ ] Input validation
- [ ] Authentication/authorization
- [ ] Data encryption (in-transit, at-rest)
- [ ] PII handling compliant

### Deployment
```

```
- [ ] Canary/blue-green strategy planned
- [ ] Rollback procedure tested
- [ ] Load testing completed
- [ ] Disaster recovery plan

### Documentation
- [ ] API documentation
- [ ] Model card (training data, limitations)
- [ ] Runbook for incidents
- [ ] User guide
```

Final Summary

Key Takeaways

Traditional ML

- ✅ Use **XGBoost/Random Forest** for tabular data
- ✅ **AutoML** for fast prototyping
- ✅ **HPO** (Hyperparameter Optimization) for best results
- ✅ Deploy to **SageMaker/Vertex AI/Azure ML**

LLMs

- ✅ Start with **LoRA/QLoRA** (cost-effective fine-tuning)
- ✅ Use **Bedrock/Azure OpenAI** for no-training solutions
- ✅ **RLHF/DPO** for alignment
- ✅ **RAG** for knowledge grounding

Computer Vision

- ✅ **ResNet/EfficientNet** for image classification
- ✅ **YOLO/Faster R-CNN** for object detection
- ✅ **ViT (Vision Transformer)** for state-of-the-art

GANs

- ✅ **DCGAN** for basic image generation
- ✅ **StyleGAN2** for high-quality faces
- ✅ **CycleGAN** for unpaired translation
- ✅ **Pix2Pix** for paired translation

Cost Optimization

- ✅ **Spot instances:** 60-90% discount
- ✅ **Mixed precision:** 2x faster
- ✅ **Early stopping:** Stop when done
- ✅ **Model distillation:** 10x cheaper inference

Production

- ✅ **A/B testing** before full rollout
- ✅ **Canary deployment** for safety
- ✅ **Monitoring** for reliability
- ✅ **Caching** for performance

Quick Reference Commands

AWS CLI

```
# Train model
aws sagemaker create-training-job --training-job-name my-job ...

# Deploy endpoint
aws sagemaker create-endpoint --endpoint-name my-endpoint ...

# Invoke endpoint
aws sagemaker-runtime invoke-endpoint --endpoint-name my-endpoint --body '...'
```

GCP CLI

```
# Train model
gcloud ai custom-jobs create --region=us-central1 ...

# Deploy model
gcloud ai endpoints deploy-model --endpoint=ENDPOINT_ID ...

# Predict
gcloud ai endpoints predict ENDPOINT_ID --json-request=request.json
```

Azure CLI

```
# Train model
az ml job create --file job.yml

# Deploy model
az ml online-endpoint create --name my-endpoint ...

# Invoke endpoint
az ml online-endpoint invoke --name my-endpoint --request-file request.json
```

Document Complete! 🎉

This master guide covers:

- ✅ Traditional ML training & testing
- ✅ LLM fine-tuning & evaluation
- ✅ Computer Vision (CNNs, Vision Transformers, Object Detection)
- ✅ GANs (Basic → DCGAN → StyleGAN2 → CycleGAN → Pix2Pix)
- ✅ AWS, GCP, Azure implementations
- ✅ Public dataset integration
- ✅ Cost optimization strategies
- ✅ Production best practices
- ✅ Complete code examples

Last Updated: September 2026
Version: 1.0
For: Professional Skill Enhancement
Authors: AI Collaboration & Pramod Kumar

Disclaimer: This document contains technical guidance and code examples for informational purposes only. Cloud provider pricing information, free tier details, and service availability reflect the best available information at the time of publication and may have changed. Always:

- Verify current pricing with cloud providers before deployment
- Check service availability in your region
- Review terms of service and compliance requirements
- Test thoroughly in development environments before production use